

```

1  /* =====
2  NaqshAI — O'zbek milliy naqshlari studiyasi
3  Elektron hisoblash mashinalari (EHM) uchun yaratilgan dastur
4  -----
5  Dasturlash tili : JavaScript (ECMAScript, ECMA-262 / ISO/IEC 16262 standarti)
6  Muallif       : EGAMNAZAROVA SEVARAXON XASANBOY QIZI
7  Talabnoma     : DT 202610311/2
8  Fayl         : naqshai.js
9  Hajmi        : 9 008 qator (ushbu sarlavha bilan birga)
10 -----
11 IZOH. Ushbu matn — dasturning BOSHLANG'ICH MATNI (source code).
12 U hech qanday kompilyator yoki obfuskator tomonidan qayta ishlanmagan:
13 o'zgaruvchi va funksiya nomlari to'liq, izohlar (kommentariyalar) saqlangan,
14 qator tuzilishi asl holida.
15
16 Ushbu faylga FAQAT muallifning shaxsiy kodi kiritilgan. Dastur tarkibidagi
17 uchinchi tomon kutubxonalari — Three.js r128 (MIT litsenziyasi) va
18 Supabase JS v2 (MIT litsenziyasi) — muallifning ijodiy mehnati natijasi
19 emas, shuning uchun bu yerga KIRITILMAGAN.
20 ===== */
21
22 'use strict';
23 /* =====
24 NaqshAI — O'zbek milliy naqshlari studiyasi
25 Yagona fayl: naqsh yadrosi, 3D, AR, Chat, Eksport
26 ===== */
27
28 const TAU = Math.PI * 2;
29
30 /* ----- Global holat ----- */
31 const P = {
32   type:'girihi', sym:8, cmp:4, sw:2.2, ir:0.38,
33   spir:0.34, dens:0.55, lay:2, skew:0, rot:0,
34   pal:'gold', fill:'dual',
35   dvar:0, weave:true, /* PIC: kontakt burchagi varianti va to'qish */
36   _bg:'#f7f1e3'
37 };
38 let TESS = { on:false, group:'p4', cell:150, showCell:false, bleed:true, noring:true };
39 let NORING = 0; /* >0 bo'lsa: markazdagi shu radiusdan katta doiralari (ramkalar) chizilmaydi */
40 const T3D = { tess:false, group:'p4' };
41
42 /* ----- Palitralar (6 ta, har biri 5 rang) ----- */
43 const PAL = {
44   gold: {nm:"Oltin",      bg:'#f7f1e3', c:['#b8912f', '#1d3a63', '#6e5312', '#3f6a95', '#efe2bc']},
45   turquoise: {nm:"Feruza", bg:'#f0f7f6', c:['#0f8b8b', '#b8912f', '#0a5a60', '#6cc0b6', '#e2f2ee']},
46   emerald: {nm:"Zumrad",  bg:'#eff5ec', c:['#1e7a4c', '#b8912f', '#0d4227', '#5cb886', '#d7ecdd']},
47   ruby: {nm:"Yoqut",     bg:'#f9f0ec', c:['#9c2331', '#b8912f', '#57101a', '#cf6572', '#f2d9d6']},
48   navy: {nm:"To'q ko'k",  bg:'#eef1f6', c:['#173263', '#b8912f', '#0c1e40', '#5f7cb4', '#e6e0cb']},
49   earth: {nm:"Tuproq",    bg:'#f6f0e6', c:['#7a5230', '#5f6b4a', '#452e14', '#b08050', '#e6d3af']}
50 };
51
52 /* ----- Rang helperlari ----- */
53 function hex2rgb(h){
54   h = (h||'#000').replace('#','');
55   if(h.length===3) h = h[0]+h[0]+h[1]+h[1]+h[2]+h[2];
56   return [parseInt(h.slice(0,2),16)|0, parseInt(h.slice(2,4),16)|0, parseInt(h.slice(4,6),16)|0];
57 }
58 function lerpColor(c1,c2,t){
59   const a=hex2rgb(c1), b=hex2rgb(c2);
60   const m=i=>Math.round(a[i]+(b[i]-a[i])*t);
61   return 'rgb('+m(0)+' '+m(1)+' '+m(2)+' )';
62 }
63 /* Qoraytirish — lenta konturlari uchun */
64 function shade(c,t){ return lerpColor(c,'#201a0c',t); }
65
66 /* ----- Geometriya helperlari ----- */
67 function pol(a,r){ return [Math.cos(a)*r, Math.sin(a)*r]; }
68 function rayX(p,a,q,b){
69   const dx=Math.cos(a), dy=Math.sin(a), ex=Math.cos(b), ey=Math.sin(b);
70   const den = dx*ey - dy*ex;
71   if(Math.abs(den) < 1e-9) return null;
72   const t = ((q[0]-p[0])*ey - (q[1]-p[1])*ex) / den;
73   return [p[0]+dx*t, p[1]+dy*t];
74 }
75 /* Ikki kesma kesishmasi */
76 function segX(A,B,C,D){
77   const dx=B[0]-A[0], dy=B[1]-A[1], d2x=D[0]-C[0], d2y=D[1]-C[1];
78   const den = dx*d2y - dly*d2x;
79   if(Math.abs(den)<1e-9) return null;
80   const t = ((C[0]-A[0])*d2y - (C[1]-A[1])*d2x)/den;
81   return [A[0]+dx*t, A[1]+dy*t];
82 }
83 /* Doira — 4 kubik Bezier bilan */
84 function circle(ctx,x,y,r){
85   if(NORING && x===0 && y===0 && r >= NORING) return; /* tessellatsiyada doira ramka olib tashlanadi */
86   const k = 0.5522847498*r;
87   ctx.moveTo(x+r,y);
88   ctx.bezierCurveTo(x+r,y+k, x+k,y+r, x,y+r);
89   ctx.bezierCurveTo(x-k,y+r, x-r,y+k, x-r,y);
90   ctx.bezierCurveTo(x-r,y-k, x-k,y-r, x,y-r);
91   ctx.bezierCurveTo(x+k,y-r, x+r,y-k, x+r,y);

```

```

92   ctx.closePath();
93 }
94 /* Yoy — Bezier segmentlar bilan (4/3·tan(Δ/4) formulasi) */
95 function arcB(ctx,cx,cy,r,a0,a1,noMove){
96   let da = a1-a0;
97   const segs = Math.max(1, Math.ceil(Math.abs(da)/(Math.PI/2)));
98   const step = da/segs;
99   const s0 = [cx+Math.cos(a0)*r, cy+Math.sin(a0)*r];
100  if(!noMove) ctx.moveTo(s0[0],s0[1]);
101  for(let i=0;i<segs;i++){
102    const t0=a0+step*i, t1=t0+step;
103    const k = 4/3*Math.tan((t1-t0)/4)*r;
104    ctx.bezierCurveTo(
105      cx+Math.cos(t0)*r - Math.sin(t0)*k, cy+Math.sin(t0)*r + Math.cos(t0)*k,
106      cx+Math.cos(t1)*r + Math.sin(t1)*k, cy+Math.sin(t1)*r - Math.cos(t1)*k,
107      cx+Math.cos(t1)*r, cy+Math.sin(t1)*r);
108  }
109 }
110 function ellipsePath(ctx,x,y,rx,ry,rot){
111   ctx.save(); ctx.translate(x,y); ctx.rotate(rot||0); ctx.scale(rx,ry);
112   circle(ctx,0,0,1);
113   ctx.restore();
114 }
115 /* Catmull-Rom → kubik Bezier */
116 function smoothPath(ctx,pts,closed){
117   const n = pts.length;
118   if(n < 2) return;
119   const get = i => closed ? pts[((i%n)+n)%n] : pts[Math.max(0,Math.min(n-1,i))];
120   ctx.moveTo(pts[0][0], pts[0][1]);
121   const last = closed ? n : n-1;
122   for(let i=0;i<last;i++){
123     const p0=get(i-1), p1=get(i), p2=get(i+1), p3=get(i+2);
124     ctx.bezierCurveTo(
125       p1[0]+(p2[0]-p0[0])/6, p1[1]+(p2[1]-p0[1])/6,
126       p2[0]-(p3[0]-p1[0])/6, p2[1]-(p3[1]-p1[1])/6,
127       p2[0], p2[1]);
128   }
129   if(closed) ctx.closePath();
130 }
131 function polyPath(ctx,pts){
132   if(!pts.length) return;
133   ctx.moveTo(pts[0][0],pts[0][1]);
134   for(let i=1;i<pts.length;i++) ctx.lineTo(pts[i][0],pts[i][1]);
135   ctx.closePath();
136 }
137 function starPath(ctx,n,rOut,rIn,rot0){
138   rot0 = rot0||0;
139   const pts=[];
140   for(let i=0;i<n*2;i++) pts.push(pol(rot0 + Math.PI*i/n, i%2 ? rIn : rOut));
141   polyPath(ctx,pts);
142 }
143 /* Barg — yopiq tomchi (2 kubik Bezier) */
144 function leaf(ctx,x,y,ang,len,wid){
145   ctx.save(); ctx.translate(x,y); ctx.rotate(ang);
146   ctx.moveTo(0,0);
147   ctx.bezierCurveTo(len*0.22,-wid, len*0.72,-wid*0.7, len,0);
148   ctx.bezierCurveTo(len*0.72, wid*0.7, len*0.22, wid, 0,0);
149   ctx.closePath();
150   ctx.restore();
151 }
152 /* Gulbarg — egri yonli, o'tkir uchli */
153 function petalPath(ctx,a,rIn,rOut,halfw){
154   const bl = pol(a-halfw, rIn), br = pol(a+halfw, rIn), tip = pol(a, rOut);
155   const cl1 = pol(a-halfw*1.12, rIn+(rOut-rIn)*0.42), cl2 = pol(a-halfw*0.5, rOut*0.93);
156   const cr2 = pol(a+halfw*1.12, rIn+(rOut-rIn)*0.42), cr1 = pol(a+halfw*0.5, rOut*0.93);
157   ctx.moveTo(bl[0],bl[1]);
158   ctx.bezierCurveTo(cl1[0],cl1[1], cl2[0],cl2[1], tip[0],tip[1]);
159   ctx.bezierCurveTo(cr1[0],cr1[1], cr2[0],cr2[1], br[0],br[1]);
160   const cb = pol(a, rIn*0.86);
161   ctx.bezierCurveTo(cb[0],cb[1], cb[0],cb[1], bl[0],bl[1]);
162   ctx.closePath();
163 }
164 /* Logarifmik spiral: r(θ)=a·e^(bθ) */
165 function spiralPts(b,thMax,L,mirror,phase){
166   const pts=[], a = L/Math.exp(b*thMax), mir = mirror ? -1 : 1;
167   const steps = Math.max(10, Math.round(thMax/0.14));
168   for(let i=0;i<=steps;i++){
169     const th = thMax*i/steps;
170     const r = a*Math.exp(b*th);
171     pts.push([Math.cos(mir*th+(phase||0))*r, Math.sin(mir*th+(phase||0))*r]);
172   }
173   return pts;
174 }
175 function cubicAt(p0,p1,p2,p3,t){
176   const u=1-t;
177   return [
178     u*u*u*p0[0] + 3*u*u*t*p1[0] + 3*u*t*t*p2[0] + t*t*t*p3[0],
179     u*u*u*p0[1] + 3*u*u*t*p1[1] + 3*u*t*t*p2[1] + t*t*t*p3[1]
180   ];
181 }
182

```

```

183  /* ----- Chizish helperlari ----- */
184  function st(ctx,col,w){
185      ctx.save();
186      ctx.strokeStyle=col; ctx.lineWidth=Math.max(0.15,w);
187      ctx.lineCap='round'; ctx.lineJoin='round';
188      ctx.stroke();
189      ctx.restore();
190  }
191  /* Interlacing: pastki chiziq kesishuvda uziladi */
192  function iStroke(ctx,col,w){
193      if(!ctx._svg){
194          ctx.save();
195          ctx.strokeStyle=P._bg; ctx.lineWidth=Math.max(0.4,w)*2.6;
196          ctx.lineCap='round'; ctx.lineJoin='round';
197          ctx.stroke();
198          ctx.restore();
199      }
200      st(ctx,col,w);
201  }
202  /* == LENTA (RIBBON) – sanoat sifatidagi tasma: ikki kontur orasida band ==
203      Haqiqiy girih/knotwork ana shunday ko'rinadi */
204  function ribbon(ctx,band,edge,w){
205      ctx.save();
206      ctx.lineCap='round'; ctx.lineJoin='round';
207      ctx.strokeStyle = edge;
208      ctx.lineWidth = w;
209      ctx.stroke();
210      ctx.strokeStyle = band;
211      ctx.lineWidth = Math.max(0.6, w - Math.max(1.8, w*0.42));
212      ctx.stroke();
213      /* glazur yaltirog'i – lenta o'rtasidagi nozik yorug' chiziq */
214      ctx.strokeStyle = lerpColor(band, 'ffffff', 0.5);
215      ctx.lineWidth = Math.max(0.35, w*0.11);
216      ctx.stroke();
217      ctx.restore();
218  }
219  /* Lenta + interlacing (pastdagi lentani uzadi → ustidan o'tish effekti) */
220  function ribbonI(ctx,band,edge,w){
221      if(!ctx._svg){
222          ctx.save();
223          ctx.strokeStyle = P._bg;
224          ctx.lineWidth = w + Math.max(3, w*0.55);
225          ctx.lineCap='round'; ctx.lineJoin='round';
226          ctx.stroke();
227          ctx.restore();
228      }
229      ribbon(ctx,band,edge,w);
230  }
231  function fillA(ctx,col,alpha){
232      if(P.fill==='line') return;
233      ctx.save();
234      ctx.globalAlpha = (P.fill==='fill' ? 0.94 : 0.62) * (alpha===undefined?1:alpha);
235      ctx.fillStyle = col;
236      ctx.fill();
237      ctx.restore();
238  }
239  function fillHard(ctx,col,alpha){
240      ctx.save();
241      ctx.globalAlpha = alpha===undefined?1:alpha;
242      ctx.fillStyle = col;
243      ctx.fill();
244      ctx.restore();
245  }
246  function dot(ctx,x,y,r,col){
247      ctx.beginPath(); circle(ctx,x,y,r);
248      ctx.save(); ctx.fillStyle=col; ctx.fill(); ctx.restore();
249  }
250
251  /* =====
252      A. O'ZBEK GEOMETRIK NAQSHLARI (8) – kanonik qurilish
253      ===== */
254
255  /* Rozetta geometriyasi: yulduz o'zagi + kite-gulbarglar (klassik islomiy rozetta) */
256  function rosetteGeo(n,R,ir){
257      const R0 = R*0.92; /* gulbarg tashqi radiusi */
258      const r2 = R*(0.36 + ir*0.28); /* yulduz uchlari */
259      const r1 = r2*0.55; /* yulduz ichki */
260      const rm = r2 + (R0-r2)*0.52; /* gulbarg yelkasi */
261      const w = (Math.PI/n)*0.86; /* gulbarg yarim eni */
262      return {R0,r2,r1,rm,w};
263  }
264  /* Kite-gulbarg – biroz bo'rtgan yonlar bilan */
265  function kitePath(ctx,a,g){
266      const A = pol(a, g.r2), L = pol(a-g.w, g.rm), T = pol(a, g.R0), Rr = pol(a+g.w, g.rm);
267      const bow = 0.14;
268      const q=(P1,P2)=>{ /* biroz tashqariga bo'rtgan Bezier yon */
269          const mx=(P1[0]+P2[0])/2, my=(P1[1]+P2[1])/2;
270          const d=Math.hypot(mx,my)||1;
271          const cx2=mx*(1+bow*40/d), cy2=my*(1+bow*40/d);
272          ctx.bezierCurveTo(P1[0]+(cx2-P1[0])*0.6,P1[1]+(cy2-P1[1])*0.6, P2[0]+(cx2-P2[0])*0.6,P2[1]+(cy2-P2[1])*0.6, P2
[0],P2[1]);

```

```

273     };
274     ctx.moveTo(A[0],A[1]); q(A,L); q(L,T); q(T,Rr); q(Rr,A);
275     ctx.closePath();
276 }
277
278 /* 1. GIRIH – rozetta strapwork: to'qilgan lentali yulduz + kite-gulbarglar */
279 function drawGirih(ctx,R,p,sw,c0,c1,c2,c3){
280     const n = Math.max(6, p.sym);
281     const g = rosetteGeo(n,R,p.ir);
282     g.w *= (0.72 + p.dens*0.55); /* zichlik → gulbarg eni */
283     g.r1 = g.r2*(0.66 - p.spir*0.28); /* spiral → yulduz o'tkirligi */
284     const edge = shade(c0,0.5);
285     const bw = sw*2.2 + 1.5;
286     /* 1) kite-gulbarglar foni */
287     for(let i=0;i<n;i++){
288         const a = TAU*i/n - Math.PI/2;
289         ctx.beginPath(); kitePath(ctx,a,g);
290         fillA(ctx, i%2? c3 : lerpColor(c3,c1,0.25), 0.75);
291         st(ctx, shade(c2,0.25), sw*0.45);
292         /* o'rta tomir – chizmachil aniqlik chizig'i */
293         const v0 = pol(a, g.r2*1.05), v1 = pol(a, g.R0*0.93);
294         ctx.beginPath();
295         ctx.moveTo(v0[0],v0[1]);
296         ctx.bezierCurveTo(v0[0],v0[1], v1[0],v1[1], v1[0],v1[1]);
297         st(ctx, shade(c2,0.12), sw*0.3);
298     }
299     /* 2) yulduz o'zagi to'ldirilgan */
300     ctx.beginPath(); starPath(ctx, n, g.r2, g.r1, -Math.PI/2);
301     fillA(ctx, c1, 0.9);
302     /* 3) LENTALAR: {n/k} vatarlar, ikki o'tishda to'qiladi */
303     const k = Math.max(2, Math.min(n-2, Math.floor(n/2)-1));
304     const pts=[]; for(let i=0;i<n;i++) pts.push(pol(TAU*i/n - Math.PI/2, g.R0));
305     const chord=(i)>=>{ const A=pts[i], B=pts[(i+k)%n]; ctx.beginPath(); ctx.moveTo(A[0],A[1]); ctx.lineTo(B[0],B[1])
; };
306     for(let pass=0; pass<2; pass++){
307         for(let i=pass; i<n; i+=2){ chord(i); ribbonI(ctx, c0, edge, bw); }
308     }
309     /* 4) markaziy vatarlarning o'rta bo'laklari – to'qishni yakunlash */
310     for(let i=0;i<n;i+=2){
311         const A=pts[i], B=pts[(i+k)%n];
312         ctx.beginPath();
313         ctx.moveTo(A[0]+(B[0]-A[0])*0.42, A[1]+(B[1]-A[1])*0.42);
314         ctx.lineTo(A[0]+(B[0]-A[0])*0.58, A[1]+(B[1]-A[1])*0.58);
315         ribbonI(ctx, c0, edge, bw);
316     }
317     /* 4b) gulbarglar orasidagi interstitsial nayzalar */
318     for(let i=0;i<n;i++){
319         const a = TAU*(i+0.5)/n - Math.PI/2;
320         ctx.beginPath();
321         const A2 = pol(a-g.w*0.5, g.rm), B2 = pol(a+g.w*0.5, g.rm), T2 = pol(a, g.R0*0.995), I2 = pol(a, g.r2*1.12);
322         ctx.moveTo(I2[0],I2[1]);
323         ctx.bezierCurveTo(A2[0],A2[1], A2[0],A2[1], T2[0],T2[1]);
324         ctx.bezierCurveTo(B2[0],B2[1], B2[0],B2[1], I2[0],I2[1]);
325         ctx.closePath();
326         fillA(ctx, c0, 0.4);
327         st(ctx, shade(c2,0.2), sw*0.35);
328     }
329     /* 5) tashqi halqa lentasi + tugunlarda mini-yulduzlar */
330     ctx.beginPath(); circle(ctx,0,0,g.R0); ribbon(ctx, c1, shade(c1,0.45), bw*0.85);
331     ctx.beginPath(); circle(ctx,0,0,R); st(ctx, c2, sw*0.6);
332     for(let i=0;i<n;i++){
333         const q = pol(TAU*i/n - Math.PI/2, g.R0);
334         ctx.beginPath(); starPath(ctx, 8, bw*1.05, bw*0.5, 0);
335         ctx.save(); ctx.translate(q[0],q[1]);
336         ctx.beginPath(); starPath(ctx, 8, bw*1.05, bw*0.5, 0);
337         fillHard(ctx, c3, 0.95); st(ctx, shade(c0,0.35), sw*0.35);
338         ctx.restore();
339     }
340     dot(ctx,0,0,g.r1*0.35,c0);
341     /* murakkablik ≥5 → ichki mini-rozetta halqasi */
342     if(p.cmp>=5){
343         ctx.save();
344         ctx.scale(0.4,0.4);
345         ctx.rotate(Math.PI/n);
346         const g2 = rosetteGeo(n, R, p.ir);
347         for(let i=0;i<n;i++){
348             ctx.beginPath(); kitePath(ctx, TAU*i/n - Math.PI/2, g2);
349             fillA(ctx, i%2? c1 : c3, 0.65);
350             st(ctx, shade(c2,0.25), sw*0.9);
351         }
352         ctx.beginPath(); starPath(ctx, n, g2.r2, g2.r2*0.5, -Math.PI/2);
353         fillA(ctx, c0, 0.85);
354         st(ctx, edge, sw*0.8);
355         ctx.restore();
356     }
357 }
358
359 /* 2. XATAM – 8 qirrali yulduz: ikki to'qilgan kvadrat lenta */
360 function drawXatam(ctx,R,p,sw,c0,c1,c2,c3){
361     const edge = shade(c0,0.5);
362     const bw = sw*2.4 + 1.5;

```

```

363 const levels = 1 + (p.cmp>=5 ? 1 : 0);
364 for(let lev=0; lev<levels; lev++){
365   const r = R*0.9*Math.pow(0.5, lev);
366   const rot = -Math.PI/4 + lev*Math.PI/8;
367   const sq = (rr,rt)=>{ const q=[]; for(let i=0;i<4;i++) q.push(pol(rt+TAU*i/4, rr)); return q; };
368   const A = sq(r, rot), B = sq(r, rot+Math.PI/4);
369   /* 8 qirrali yulduz foni */
370   ctx.beginPath(); starPath(ctx, 8, r, r*0.765, rot);
371   fillA(ctx, lev? c3 : c1, 0.55);
372   /* ichki sakkizburchak */
373   ctx.beginPath();
374   const oct=[]; for(let i=0;i<8;i++) oct.push(pol(rot+Math.PI/8+TAU*i/8, r*0.545));
375   polyPath(ctx,oct);
376   fillA(ctx, lev? c1 : c3, 0.8);
377   st(ctx, shade(c2,0.25), sw*0.5);
378   /* TO'QILGAN kvadratlar: A pastda, B ustida, so'ng A ning 2 qarama-qarshi
379      burchagi qayta ustiga – navbatlashgan haqiqiy weave */
380   ctx.beginPath(); polyPath(ctx,A); ribbon(ctx, c0, edge, bw);
381   ctx.beginPath(); polyPath(ctx,B); ribbonI(ctx, lev? c0 : c1, shade(lev? c0 : c1, 0.5), bw);
382   for(const j of [0,2]){
383     ctx.beginPath();
384     const P1=A[j], P2=A[(j+1)%4];
385     ctx.moveTo(P1[0]+(P2[0]-P1[0])*0.28, P1[1]+(P2[1]-P1[1])*0.28);
386     ctx.lineTo(P1[0]+(P2[0]-P1[0])*0.72, P1[1]+(P2[1]-P1[1])*0.72);
387     const P3=A[(j+2)%4], P4=A[(j+3)%4];
388     ctx.moveTo(P3[0]+(P4[0]-P3[0])*0.28, P3[1]+(P4[1]-P3[1])*0.28);
389     ctx.lineTo(P3[0]+(P4[0]-P3[0])*0.72, P3[1]+(P4[1]-P3[1])*0.72);
390     ribbonI(ctx, c0, edge, bw);
391     break;
392   }
393 }
394 /* yulduz uchlaridan tashqi halqaga bog'lovchi nurlar */
395 ctx.beginPath();
396 for(let i=0;i<8;i++){
397   const a = -Math.PI/4 + TAU*i/8;
398   const A2 = pol(a, R*0.9), B2 = pol(a, R);
399   ctx.moveTo(A2[0],A2[1]); ctx.lineTo(B2[0],B2[1]);
400 }
401 st(ctx, c2, sw*0.6);
402 ctx.beginPath(); circle(ctx,0,0,R); ribbon(ctx, c1, shade(c1,0.45), sw*1.6+1);
403 dot(ctx,0,0,R*0.05,c0);
404 }
405
406 /* 3. ROZETTA – gulbarg halqalari, konsentrik bandlar (loyqasiz) */
407 function drawRozetta(ctx,R,p,sw,c0,c1,c2,c3){
408   const n = Math.max(5,p.sym);
409   const w = (Math.PI/n)*(0.55 + p.dens*0.35);
410   /* tashqi gulbarg halqasi */
411   for(let i=0;i<n;i++){
412     const a = TAU*i/n - Math.PI/2;
413     ctx.beginPath(); petalPath(ctx, a, R*0.46, R*0.95, w);
414     fillA(ctx, i%2? c1 : c3, 0.65);
415     st(ctx, shade(c0,0.35), sw*0.7);
416     /* o'rta tomir */
417     const b0 = pol(a, R*0.5), b1 = pol(a, R*0.86);
418     ctx.beginPath(); ctx.moveTo(b0[0],b0[1]);
419     ctx.bezierCurveTo(b0[0],b0[1], b1[0],b1[1], b1[0],b1[1]);
420     st(ctx, shade(c2,0.15), sw*0.35);
421   }
422   /* oraliq kichik gulbarglar – o'z bandida */
423   for(let i=0;i<n;i++){
424     const a = TAU*(i+0.5)/n - Math.PI/2;
425     ctx.beginPath(); petalPath(ctx, a, R*0.24, R*0.5, w*0.62);
426     fillA(ctx, c0, 0.55);
427     st(ctx, shade(c0,0.4), sw*0.5);
428   }
429   /* markaz: halqa + yulduzcha – ichki radius slayderiga bog'liq */
430   const rc = R*(0.13 + p.ir*0.28);
431   ctx.beginPath(); circle(ctx,0,0,rc); ribbon(ctx, c1, shade(c1,0.45), sw*1.6+1);
432   ctx.beginPath(); starPath(ctx, n, rc*0.72, rc*0.38, -Math.PI/2);
433   fillHard(ctx, c0, 0.9); st(ctx, shade(c0,0.4), sw*0.4);
434   /* murakkablik → nuqta halqalari */
435   const dotRings = Math.max(0, Math.round(p.cmp/2) - 1);
436   for(let k=0;k<dotRings;k++){
437     const rd = rc + (R*0.46 - rc)*(k+1)/(dotRings+1);
438     for(let i=0;i<n*2;i++){
439       const q = pol(TAU*(i+0.5)/(n*2) - Math.PI/2, rd);
440       dot(ctx, q[0], q[1], sw*0.7, k%2? c2 : c1);
441     }
442   }
443   ctx.beginPath(); circle(ctx,0,0,R); st(ctx, c2, sw*0.55);
444   dot(ctx,0,0,rc*0.16,c1);
445 }
446
447 /* 4. YULDUZ – to'qilgan {n/k} lenta yulduz */
448 function drawYulduz(ctx,R,p,sw,c0,c1,c2,c3){
449   const n = Math.max(5,p.sym);
450   const k = Math.max(2, Math.min(n-2, Math.floor(n/2)-1 + Math.round(p.spir*1.5)));
451   const r = R*0.9;
452   const edge = shade(c0,0.5);
453   const bw = sw*2.4 + 1.5;

```

```

454 const pts=[]; for(let i=0;i<n;i++) pts.push(pol(TAU*i/n - Math.PI/2, r));
455 /* markaziy n-burchak o'zak: vatarlar ichki kesishmalaridan */
456 const inner=[];
457 for(let i=0;i<n;i++){
458   const X = segX(pts[i], pts[(i+k)%n], pts[(i+1)%n], pts[(i+1-k+n)%n]);
459   if(X) inner.push(X);
460 }
461 if(inner.length>=3){
462   ctx.beginPath(); polyPath(ctx, inner);
463   fillA(ctx, c1, 0.85);
464 }
465 /* to'qilgan lenta vatarlar — ikki o'tish + o'rta segment ustidan */
466 const chord=(i)=>{ const A=pts[i],B=pts[(i+k)%n]; ctx.beginPath(); ctx.moveTo(A[0],A[1]); ctx.lineTo(B[0],B[1]);
};
467 for(let pass=0; pass<2; pass++){
468   for(let i=pass; i<n; i+=2){ chord(i); ribbonI(ctx, i%2? c1 : c0, shade(i%2? c1 : c0, 0.5), bw); }
469 }
470 for(let i=0;i<n;i+=2){
471   const A=pts[i], B=pts[(i+k)%n];
472   ctx.beginPath();
473   ctx.moveTo(A[0]+(B[0]-A[0])*0.44, A[1]+(B[1]-A[1])*0.44);
474   ctx.lineTo(A[0]+(B[0]-A[0])*0.56, A[1]+(B[1]-A[1])*0.56);
475   ribbonI(ctx, c0, edge, bw);
476 }
477 /* tashqi halqa lentasi + tugun nuqtalar */
478 ctx.beginPath(); circle(ctx,0,0,r); ribbon(ctx, c1, shade(c1,0.45), bw*0.8);
479 for(const q of pts) dot(ctx,q[0],q[1], bw*0.45, c0);
480 ctx.beginPath(); circle(ctx,0,0,R); st(ctx, c2, sw*0.5);
481 dot(ctx,0,0,R*0.04,c0);
482 }
483
484 /* 5. MUQARNAS — toza pog'onali arkada: ravoq lentalar + tokchalar */
485 function drawMuqarnas(ctx,R,p,sw,c0,c1,c2,c3){
486   const levels = Math.max(2, Math.min(6, 1 + p.cmp));
487   for(let l=0;l<levels;l++){
488     const r = R*0.94*Math.pow(0.74, l);
489     const m = Math.max(6, p.sym + l*2);
490     const drop = r*(0.14 + p.ir*0.2);
491     const t = l/(levels-1||1);
492     const band = lerpColor(c0,c1,t);
493     /* tokcha (nisha) fonlari */
494     for(let i=0;i<m;i++){
495       const a0 = TAU*i/m - Math.PI/2, a1 = TAU*(i+1)/m - Math.PI/2;
496       const A = pol(a0,r), B = pol(a1,r), M = pol((a0+a1)/2, r-drop);
497       ctx.beginPath();
498       ctx.moveTo(A[0],A[1]);
499       ctx.bezierCurveTo(A[0]*(1-drop/r*0.5),A[1]*(1-drop/r*0.5), M[0]+(A[0]-B[0])*0.15,M[1]+(A[1]-B[1])*0.15, M[0]
,M[1]);
500       ctx.bezierCurveTo(M[0]+(B[0]-A[0])*0.15,M[1]+(B[1]-A[1])*0.15, B[0]*(1-drop/r*0.5),B[1]*(1-drop/r*0.5), B[0]
,B[1]);
501       arcB(ctx,0,0,r,a1,a0,true);
502       ctx.closePath();
503       fillA(ctx, i%2? c3 : lerpColor(c3,c1,0.3), 0.4 + t*0.2);
504     }
505     /* ravoq lentasi */
506     ctx.beginPath();
507     for(let i=0;i<m;i++){
508       const a0 = TAU*i/m - Math.PI/2, a1 = TAU*(i+1)/m - Math.PI/2;
509       const A = pol(a0,r), B = pol(a1,r), M = pol((a0+a1)/2, r-drop);
510       ctx.moveTo(A[0],A[1]);
511       ctx.bezierCurveTo(A[0]*(1-drop/r*0.5),A[1]*(1-drop/r*0.5), M[0]+(A[0]-B[0])*0.15,M[1]+(A[1]-B[1])*0.15, M[0]
,M[1]);
512       ctx.bezierCurveTo(M[0]+(B[0]-A[0])*0.15,M[1]+(B[1]-A[1])*0.15, B[0]*(1-drop/r*0.5),B[1]*(1-drop/r*0.5), B[0]
,B[1]);
513     }
514     ribbon(ctx, band, shade(band,0.45), sw*1.5+0.8);
515     /* osilgan tomchi nuqtalari */
516     for(let i=0;i<m;i++){
517       const M = pol(TAU*(i+0.5)/m - Math.PI/2, r-drop);
518       dot(ctx, M[0], M[1], sw*0.9+0.6, shade(band,0.3));
519     }
520   }
521   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.55);
522   const core = R*Math.pow(0.74, levels);
523   ctx.beginPath(); starPath(ctx, p.sym, core, core*0.5, -Math.PI/2);
524   fillHard(ctx, c0, 0.9);
525 }
526
527 /* 6. KATAK-8 — 4.8.8 Arximed kafeli, aniq hizalangan */
528 function drawKatak8(ctx,R,p,sw,c0,c1,c2,c3){
529   const FIELD = !!p.field;
530   const cells = FIELD ? Math.max(2, Math.round(R*1.7/(p._cellpx||150))) : Math.max(2, Math.round(2 + p.dens*2));
531   const s = R*1.7/cells;
532   const oR = s*0.5/Math.cos(Math.PI/8); /* sakkizburchak "radiusi" */
533   const half = cells*s/2;
534   ctx.save();
535   if(!FIELD){
536     ctx.beginPath(); circle(ctx,0,0,R*0.98);
537     if(!ctx._svg) ctx.clip();
538   }
539   for(let gy=0; gy<=cells; gy++){

```

```

540     for(let gx=0; gx<=cells; gx++){
541         const x = -half + gx*s, y = -half + gy*s;
542         if(Math.hypot(x,y) > R*1.05) continue;
543         /* sakkizburchak */
544         const oct=[]; for(let i=0;i<8;i++) oct.push([x+Math.cos(Math.PI/8+TAU*i/8)*oR, y+Math.sin(Math.PI/8+TAU*i/8)
*oR]);
545         ctx.beginPath(); polyPath(ctx,oct);
546         fillA(ctx, (gx+gy)%2? c3 : lerpColor(c3,'#ffffff',0.4), 0.8);
547         ribbon(ctx, c0, shade(c0,0.45), sw*1.4+0.8);
548         /* ichki yulduzcha */
549         if(p.cmp>=3){
550             ctx.beginPath();
551             const sp=[]; for(let i=0;i<16;i++) sp.push([x+Math.cos(Math.PI/8+Math.PI*i/8)*(i%2? oR*0.22 : oR*0.5), y+M
ath.sin(Math.PI/8+Math.PI*i/8)*(i%2? oR*0.22 : oR*0.5)]);
552             polyPath(ctx,sp);
553             fillA(ctx, c1, 0.8); st(ctx, shade(c1,0.35), sw*0.4);
554         }
555     }
556 }
557 /* kichik kvadratlar — panjara tugunlari */
558 const q = s*0.5 - oR*Math.cos(Math.PI/8);
559 for(let gy=0; gy<cells; gy++){
560     for(let gx=0; gx<cells; gx++){
561         const x = -half + gx*s + s/2, y = -half + gy*s + s/2;
562         if(Math.hypot(x,y) > R*1.05) continue;
563         ctx.beginPath();
564         polyPath(ctx,[[x,y-q*1.35],[x+q*1.35,y],[x,y+q*1.35],[x-q*1.35,y]]);
565         fillHard(ctx, c0, 0.9);
566         st(ctx, shade(c0,0.45), sw*0.5);
567     }
568 }
569 ctx.restore();
570 if(!FIELD){
571     ctx.beginPath(); circle(ctx,0,0,R*0.98); ribbon(ctx, c1, shade(c1,0.45), sw*1.8+1);
572 }
573 }
574
575 /* 7. OLTİ BURCHAK — asal uyasi, toza lenta konturlar */
576 function drawHexa(ctx,R,p,sw,c0,c1,c2,c3){
577     const FIELD = !!p._field;
578     const s = FIELD ? Math.max(26,(p._cellpx||150)*0.42) : R/(2.0 + p.dens*2.0);
579     const dx = s*Math.sqrt(3), dy = s*1.5;
580     ctx.save();
581     if(!FIELD){
582         ctx.beginPath(); circle(ctx,0,0,R*0.98);
583         if(!ctx._svg) ctx.clip();
584     }
585     const NN = FIELD ? Math.ceil(R/(s*1.5))+1 : 5;
586     for(let gy=-NN; gy<=NN; gy++){
587         for(let gx=-NN; gx<=NN; gx++){
588             const x = gx*dx + (gy%2 ? dx/2 : 0), y = gy*dy;
589             if(Math.hypot(x,y) > R*1.08) continue;
590             const pts=[]; for(let i=0;i<6;i++) pts.push([x+Math.cos(Math.PI/6+TAU*i/6)*s, y+Math.sin(Math.PI/6+TAU*i/6)*
s]);
591             ctx.beginPath(); polyPath(ctx,pts);
592             const ci = ((gx%3)+3+(gy%3)*2)%3;
593             fillA(ctx, [c3, lerpColor(c3,'#ffffff',0.45), lerpColor(c3,c1,0.25)][ci], 0.7);
594             ribbon(ctx, c0, shade(c0,0.45), sw*1.3+0.7);
595             if(p.cmp>=4){
596                 for(let i=0;i<6;i++){
597                     ctx.beginPath();
598                     leaf(ctx, x, y, Math.PI/6+TAU*i/6, s*0.52, s*0.14);
599                     fillA(ctx, c1, 0.75); st(ctx, shade(c1,0.35), sw*0.35);
600                 }
601                 dot(ctx,x,y,s*0.1,c0);
602             }
603         }
604     }
605     ctx.restore();
606     if(!FIELD){
607         ctx.beginPath(); circle(ctx,0,0,R*0.98); ribbon(ctx, c1, shade(c1,0.45), sw*1.8+1);
608     }
609 }
610
611 /* 8. ZANJIRA — bir-biriga o'tkazilgan halqalar zanjiri (haqiqiy interlock) */
612 function drawZanjira(ctx,R,p,sw,c0,c1,c2,c3){
613     const n = Math.max(6, p.sym);
614     const rc = R*0.68;
615     const rr = Math.PI*rc/n * (0.92 + p.dens*0.35); /* qo'shnilar bilan kesishadi */
616     const bw = sw*2.2 + 1.5;
617     const centers=[]; for(let i=0;i<n;i++) centers.push(pol(TAU*i/n - Math.PI/2, rc));
618     /* har halqa — lenta; keyingisi oldingisini kesib o'tadi */
619     for(let i=0;i<n;i++){
620         const c = centers[i], col = i%2? c1 : c0;
621         ctx.beginPath(); circle(ctx, c[0], c[1], rr);
622         ribbonI(ctx, col, shade(col,0.5), bw);
623     }
624     /* birinchi halqaning oxirgi halqa tomonidagi yoyi — zanjirni ulash */
625     const a0 = Math.atan2(centers[0][1], centers[0][0]) + Math.PI; /* markazga qaragan */
626     const dirLast = Math.atan2(centers[n-1][1]-centers[0][1], centers[n-1][0]-centers[0][0]);
627     ctx.beginPath();

```

```

628 arcB(ctx, centers[0][0], centers[0][1], rr, dirLast-0.7, dirLast+0.7);
629 ribbonI(ctx, c0, shade(c0,0.5), bw);
630 /* markaziy medalyon */
631 const ri = rc - rr - bw;
632 if(ri > R*0.1){
633   ctx.beginPath(); circle(ctx,0,0,ri*0.9); ribbon(ctx, c2, shade(c2,0.4), sw*1.3+0.7);
634   ctx.beginPath(); starPath(ctx, n, ri*0.62, ri*0.34, -Math.PI/2);
635   fillA(ctx, c1, 0.8); st(ctx, shade(c1,0.4), sw*0.5);
636   dot(ctx,0,0,ri*0.14,c0);
637 }
638 ctx.beginPath(); circle(ctx,0,0,R); st(ctx, c2, sw*0.5);
639 }
640
641 /* =====
642 B. ISLIMIY ORGANIK NAQSHLAR (8) – nafis novda va barglar
643 ===== */
644
645 /* Nozik torayuvchi novda: 3 o'tishli qalinlik gradatsiyasi */
646 function stem(ctx,pts,col,w){
647   const edge = shade(col,0.35);
648   ctx.beginPath(); smoothPath(ctx,pts,false);
649   st(ctx, edge, w);
650   const half = pts.slice(0, Math.ceil(pts.length*0.6));
651   ctx.beginPath(); smoothPath(ctx,half,false);
652   st(ctx, edge, w*1.55);
653   ctx.beginPath(); smoothPath(ctx,pts,false);
654   st(ctx, col, Math.max(0.4,w*0.5));
655 }
656 /* Nafis barg: to'ldirish + kontur + o'rta tomir */
657 function fineLeaf(ctx,x,y,ang,len,wid,fillC,edgeC,sw){
658   ctx.beginPath(); leaf(ctx,x,y,ang,len,wid);
659   fillA(ctx, fillC, 0.85);
660   st(ctx, edgeC, sw);
661   ctx.beginPath();
662   ctx.save(); ctx.translate(x,y); ctx.rotate(ang);
663   ctx.moveTo(len*0.08,0);
664   ctx.bezierCurveTo(len*0.3,-wid*0.12, len*0.55,-wid*0.1, len*0.8,0);
665   ctx.restore();
666   st(ctx, edgeC, sw*0.55);
667 }
668 /* Uch bargli palmetta kurtak */
669 function budAt(ctx,x,y,ang,L,fillC,edgeC,sw){
670   for(const [da,f] of [[-0.55,0.72],[0.55,0.72],[0,1]]){
671     ctx.beginPath(); leaf(ctx,x,y,ang+da,L*f,L*f*0.3);
672     fillA(ctx, f===1? fillC : lerpColor(fillC,edgeC,0.25), 0.9);
673     st(ctx, shade(edgeC,0.2), sw*0.5);
674   }
675 }
676
677 /* 9. ISLIMIY – nafis spiral novdalar */
678 function drawIslimiy(ctx,R,p,sw,c0,c1,c2,c3){
679   const n = Math.max(3, Math.min(10, p.sym));
680   const b = 0.1 + p.spir*0.28;
681   const thMax = Math.PI*(1.2 + p.cmp*0.18);
682   for(let i=0;i<n;i++){
683     ctx.save(); ctx.rotate(TAU*i/n);
684     const pts = spiralPts(b, thMax, R*0.88, i%2===1, 0);
685     stem(ctx, pts, c0, sw*1.1);
686     /* barglar – kamroq, kattaroq, uchga qarab o'sadi */
687     const every = Math.max(5, 11 - Math.round(p.dens*5));
688     for(let j=every; j<pts.length-3; j+=every){
689       const t2 = j/pts.length;
690       const a = pts[j], b2 = pts[j+1];
691       const ang = Math.atan2(b2[1]-a[1], b2[0]-a[0]);
692       const side = (Math.floor(j/every)%2)? 1 : -1;
693       fineLeaf(ctx, a[0], a[1], ang + side*Math.PI/2.5, R*0.1*(0.5+t2), R*0.03*(0.5+t2), c1, shade(c0,0.25), sw*0.
45);
694     }
695     const e = pts[pts.length-1], e2 = pts[pts.length-3];
696     budAt(ctx, e[0], e[1], Math.atan2(e[1]-e2[1],e[0]-e2[0]), R*0.16, c1, c0, sw);
697     /* markazdagi kichik qarshi gajak */
698     if(p.cmp>=3){
699       const pts2 = spiralPts(b*1.35, thMax*0.42, R*0.3, i%2===0, 0.4);
700       stem(ctx, pts2, c2, sw*0.7);
701       const f = pts2[pts2.length-1], f2 = pts2[pts2.length-2];
702       fineLeaf(ctx, f[0], f[1], Math.atan2(f[1]-f2[1],f[0]-f2[0]), R*0.09, R*0.028, c3, shade(c2,0.2), sw*0.4);
703     }
704     ctx.restore();
705   }
706   ctx.beginPath(); circle(ctx,0,0,R*0.05+sw); fillHard(ctx,c0,0.95);
707   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
708 }
709
710 /* -----
711 O'YMA PANEL (kartush) – Xiva/Farg'ona yog'och o'ymakorligi uslubi
712 -----
713 Ustun qirralaridagi va eshik tabaqalaridagi naqsh radial medalyon EMAS:
714 u tik KARTUSH – ramkali panel, ichida o'qqa nisbatan KO'ZGU-SIMMETRIK
715 islamiy. Uch xususiyati bu uslubni tanitadi va shu yerda takrorlanadi:
716
717 1) kartush ramkasi – yon tomonlari tik, uchlari siniq (ogee) yoy;

```

```

718     2) NUQTALI FON ("chizma" / pargori) – o'yilgan pastki qatlam;
719     3) RELYEF – har kontur ostiga yorug' va soya nusxasi, shunda naqsh
720         yassi chiziq emas, o'yilgan sirt bo'lib ko'rinadi.
721
722     Barcha shakllar oddiy yo'l buyruqlari bilan chiziladi (offscreen canvas
723     yoki filtr ishlatilmaydi), shuning uchun SVG/DXF eksporti ham ishlaydi.
724     ----- */
725     function oymaCartouche(ctx, w, h){
726         const tip = h*0.22;                /* uchdagi yoy balandligi */
727         ctx.moveTo(-w, -h + tip);
728         ctx.lineTo(-w, h - tip);
729         /* yuqori siniq yoy: nazorat nuqtalari cho'qqida qarama-qarshi yo'nalgani
730             uchun uchi yumaloq emas, O'TKIR chiqadi */
731         ctx.bezierCurveTo(-w, h - tip*0.32, -w*0.13, h*1.02, 0, h);
732         ctx.bezierCurveTo(w*0.13, h*1.02, w, h - tip*0.32, w, h - tip);
733         ctx.lineTo(w, -h + tip);
734         ctx.bezierCurveTo(w, -h + tip*0.32, w*0.13, -h*1.02, 0, -h);
735         ctx.bezierCurveTo(-w*0.13, -h*1.02, -w, -h + tip*0.32, -w, -h + tip);
736         ctx.closePath();
737     }
738
739     function drawOyma(ctx, R, p, sw, c0, c1, c2, c3){
740         const hw = R*0.60, hh = R*0.95;
741         const fw = R*0.070;                /* ramka eni */
742         const lw = Math.max(0.6, sw*0.85);
743         const iw = hw - fw*2.1, ih = hh - fw*2.1; /* ichki maydon */
744         const dark = shade(c0, 0.55), light = lerpColor(c1, 'ffffff', 0.72);
745
746         /* --- RELYEF yordamchisi: konturni yorug' va soya nusxasi bilan --- */
747         const d = Math.max(0.55, lw*0.55);
748         /* Vektor eksportida (SVG/DXF/EPS) yorug'-soya nusxalari CHIZILMAYDI:
749             ular ekranda o'yma taassurotini beradi, lekin lazer uchun bu HAR
750             CHIZIQNI UCH MARTA kesish degani – material kuyadi va ish vaqti uch
751             barobar oshadi. SvgCtx o'zini ctx._svg bilan tanitadi. */
752         const vec = !!ctx._svg;
753         const relief = (path, wid)=>{
754             if(!vec){
755                 ctx.save(); ctx.translate(-d, -d); ctx.beginPath(); path(); st(ctx, light, wid); ctx.restore();
756                 ctx.save(); ctx.translate(d, d); ctx.beginPath(); path(); st(ctx, dark, wid); ctx.restore();
757             }
758             ctx.beginPath(); path(); st(ctx, c0, wid);
759         };
760
761         /* --- 1) NUQTALI FON – o'yilgan pastki qatlam ---
762             Bu SIRT FAKTURASI, kesish konturi emas. Vektor eksportida chizilmaydi:
763             aks holda lazerga minglab mayda doira (har biri alohida teshik) ketardi –
764             o'lchandi: 180x120 mm o'yma panelda 3019 kontur, 2.5 MB DXF. */
765         if(!vec && p.dens > 0.15){
766             ctx.save();
767             ctx.beginPath(); oymaCartouche(ctx, iw, ih); ctx.clip();
768             const step = Math.max(R*0.026, R*0.055*(1.15 - p.dens));
769             const rr = Math.max(0.5, step*0.17);
770             /* deterministik siljish – Math.random ishlatilmaydi, aks holda har
771                 chizishda boshqa natija chiqib, eksport takrorlanmas bo'lib qolardi */
772             const jit = (i,j)=>{ const s = Math.sin(i*127.1 + j*311.7)*43758.5453; return s - Math.floor(s); };
773             for(let j = -Math.ceil(ih/step); j <= Math.ceil(ih/step); j++){
774                 for(let i = -Math.ceil(iw/step); i <= Math.ceil(iw/step); i++){
775                     const x = i*step + (j%2 ? step*0.5 : 0) + (jit(i,j)-0.5)*step*0.28;
776                     const y = j*step + (jit(j,i)-0.5)*step*0.28;
777                     ctx.beginPath(); circle(ctx, x, y, rr); fillHard(ctx, shade(c2, 0.30), 0.55);
778                 }
779             }
780             ctx.restore();
781         }
782
783         /* --- 2) MARKAZIY KOMPOZITSIYA – o'qqa nisbatan ko'zgu-simmetrik ---
784             Tuzilma: har yon tomonda ILON-IZI NOVDA pastdan tepaga o'tadi, uning
785             har burilishida palmetta, oralarida barglar. Novda panel enining
786             deyarli hammasini egallagani uchun maydon TEKIS to'ladi.
787
788             (Avvalgi urinishda har tugundan logarifmik spiral chiqarilgandi: katta
789             L da spiralning tashqi o'rami yirik halqaga aylanib, panel o'rtaida
790             chalkash tugun hosil qilardi, yon tomonlar esa bo'sh qolardi. Ilon-izi
791             novda bu muammoni butunlay yo'q qiladi.) */
792         const waves = 3 + Math.min(2, Math.max(0, (p.cmp|0) - 3));
793         const half = (sgn)=>{
794             ctx.save(); ctx.scale(sgn, 1);
795         };
796         /* --- ilon-izi novda --- */
797         const vine = [];
798         const N = 120;
799         for(let i = 0; i <= N; i++){
800             const t = i/N;                /* 0 → pastdan tepaga */
801             const y = -ih*0.86 + t*ih*1.72;
802             /* novda o'qni KESIB O'TMAYDI: aks holda ikki ko'zgu nusxa markazda
803                 chalkashib, tugun hosil qilardi */
804             const x = iw*0.16 + (0.5 + 0.5*Math.sin(t*Math.PI*waves - Math.PI/2))*iw*0.62;
805             vine.push([x, y]);
806         }
807         relief(()=>smoothPath(ctx, vine, false), lw*1.15);
808     }

```

```

809     /* burilish nuqtalari – palmetta va qarshi gajak */
810     for(let w = 0; w < waves; w++){
811         const t = (w + 0.5)/waves;          /* to'liq cho'qqisi */
812         const i = Math.round(t*N);
813         const a = vine[Math.max(0,i-1)], b2 = vine[Math.min(N,i+1)];
814         const ang = Math.atan2(b2[1]-a[1], b2[0]-a[0]);
815         const q = vine[i];
816         /* palmetta TASHQARIGA qaragan – o'q tomonga emas, aks holda ikki
817            ko'zgu nusxaning palmettalari markazda ustma-ust tushadi */
818         budAt(ctx, q[0], q[1], 0, iw*0.30, c1, c0, lw*0.9);
819     }
820
821     /* novda bo'ylab barglar – ikki tomonga navbatma-navbat */
822     const step = Math.max(5, 13 - Math.round(p.dens*7));
823     for(let i = step; i < N - step; i += step){
824         const a = vine[i-1], b2 = vine[i+1], q = vine[i];
825         const ang = Math.atan2(b2[1]-a[1], b2[0]-a[0]);
826         const side = (Math.floor(i/step) % 2) ? 1 : -1;
827         fineLeaf(ctx, q[0], q[1], ang + side*Math.PI/2.3, iw*0.20, iw*0.068, c1, dark, lw*0.42);
828     }
829     ctx.restore();
830 };
831 half(1); half(-1);
832
833     /* o'q bo'ylab ingichka poya – ikki novdani bog'laydi */
834     const spine = [];
835     for(let i = 0; i <= 24; i++) spine.push([0, -ih*0.88 + (i/24)*ih*1.76]);
836     relief(()=>smoothPath(ctx, spine, false), lw*0.8);
837
838     /* uchlardagi kurtaklar va markaziy palmetta */
839     budAt(ctx, 0, ih*0.90, -Math.PI/2, ih*0.13, c1, c0, lw);
840     budAt(ctx, 0, -ih*0.90, Math.PI/2, ih*0.13, c1, c0, lw);
841     for(const s2 of [-1, 1]){
842         fineLeaf(ctx, 0, 0, s2>0 ? 0 : Math.PI, iw*0.34, iw*0.13, c1, dark, lw*0.55);
843         fineLeaf(ctx, 0, 0, s2>0 ? -Math.PI/2 : Math.PI/2, iw*0.22, iw*0.085, c3, dark, lw*0.45);
844     }
845     ctx.beginPath(); circle(ctx, 0, 0, iw*0.075); fillHard(ctx, c3, 0.95); st(ctx, dark, lw*0.7);
846
847     /* --- 3) KARTUSH RAMKASI – eng ustida, relyef bilan --- */
848     relief(()=>oymaCartouche(ctx, iw, ih), lw*0.9);
849     relief(()=>oymaCartouche(ctx, hw, hh), lw*1.35);
850     /* ramka ichidagi ingichka yo'l */
851     ctx.beginPath(); oymaCartouche(ctx, (hw+iw)/2, (hh+ih)/2); st(ctx, c2, lw*0.55);
852 }
853
854     /* Rumi motivi – ilmoqli buralgan barg (klassik "yaprak") */
855     function rumiLeafPath(ctx,L){
856         ctx.moveTo(0,0);
857         ctx.bezierCurveTo(L*0.12,-L*0.36, L*0.68,-L*0.42, L*0.94,-L*0.1);
858         ctx.bezierCurveTo(L*1.04, L*0.04, L*0.9, L*0.2, L*0.7, L*0.13);
859         ctx.bezierCurveTo(L*0.52, L*0.06, L*0.42, L*0.2, L*0.28, L*0.16);
860         ctx.bezierCurveTo(L*0.12, L*0.1, L*0.04, L*0.05, 0,0);
861         ctx.closePath();
862     }
863
864     /* 10. RUMI – halqa bo'ylab ilmoqli barglar + tok chizig'i */
865     function drawRumi(ctx,R,p,sw,c0,c1,c2,c3){
866         const n = Math.max(5, p.sym);
867         const rc = R*0.62;
868         /* tok halqasi (lenta) */
869         ctx.beginPath(); circle(ctx,0,0,rc);
870         ribbon(ctx, c2, shade(c2,0.4), sw*1.4+0.8);
871         const L = R*0.36*(0.8 + p.dens*0.35);
872         for(let i=0;i<n;i++){
873             const a = TAU*i/n - Math.PI/2;
874             const c = pol(a, rc);
875             /* tashqariga qaragan katta rumi barg */
876             ctx.save(); ctx.translate(c[0],c[1]); ctx.rotate(a - Math.PI/2 + 0.35 + p.skew*0.4);
877             ctx.beginPath(); rumiLeafPath(ctx, L);
878             fillA(ctx, i%2? c1 : c0, 0.85);
879             st(ctx, shade(i%2? c1 : c0, 0.4), sw*0.7);
880             /* ichki ilmoq chizig'i */
881             ctx.beginPath();
882             ctx.moveTo(L*0.1,-L*0.06);
883             ctx.bezierCurveTo(L*0.35,-L*0.22, L*0.62,-L*0.24, L*0.78,-L*0.06);
884             st(ctx, shade(c3,0.1), sw*0.4);
885             ctx.restore();
886             /* ichkariga qaragan kichik barg */
887             ctx.save(); ctx.translate(c[0],c[1]); ctx.rotate(a + Math.PI/2 - 0.35 - p.skew*0.4);
888             ctx.beginPath(); rumiLeafPath(ctx, L*0.55);
889             fillA(ctx, c3, 0.85);
890             st(ctx, shade(c2,0.25), sw*0.5);
891             ctx.restore();
892             dot(ctx, c[0], c[1], sw*1.1+0.6, shade(c0,0.2));
893         }
894     }
895
896     /* TASHQI BOG'LOVCHI NOVDALAR – katta barglar orasidan chetgacha S-novda:
897        tessellatsiyada qo'shni naqsh novdalari bilan tutashib UZLUKSIZ gulchambar hosil qiladi */
898     for(let i=0;i<n;i++){
899         const a = TAU*(i+0.5)/n - Math.PI/2;
900         const sgn = i%2 ? 1 : -1;
901         const pts = [];
902         for(let j=0;j<=10;j++){

```

```

900     const t = j/10;
901     const rr = rc*0.96 + (R*1.06 - rc*0.96)*t;
902     const wob = Math.sin(t*Math.PI)*0.16*sgn; /* yengil S-egri */
903     pts.push(pol(a + wob, rr));
904 }
905 stem(ctx, pts, c2, sw*0.85);
906 /* o'rtadagi yon barg */
907 const m1 = pts[5], m2 = pts[6];
908 const mAng = Math.atan2(m2[1]-m1[1], m2[0]-m1[0]);
909 fineLeaf(ctx, m1[0], m1[1], mAng + sgn*Math.PI/2.4, L*0.34, L*0.11, c3, shade(c2,0.2), sw*0.4);
910 /* uchidagi barg – plitka chegarasidan o'tib qo'shnisiga yetadi */
911 const e = pts[10], e2 = pts[9];
912 fineLeaf(ctx, e[0], e[1], Math.atan2(e[1]-e2[1], e[0]-e2[0]), L*0.42, L*0.14, i%2? c1 : c0, shade(c0,0.3), sw*
0.5);
913 }
914 /* markaz rozetkasi */
915 const ri = rc - L*0.75;
916 if(ri > R*0.08){
917     ctx.beginPath(); starPath(ctx, n, ri*0.7, ri*0.4, -Math.PI/2);
918     fillA(ctx, c1, 0.8); st(ctx, shade(c1,0.35), sw*0.5);
919     dot(ctx,0,0,ri*0.16,c0);
920 }
921 ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
922 }
923
924 /* 11. HATAYI – lotus: 3-segmentli gulbarglar, toza bandlar */
925 function hatayiPetal(ctx,a,rIn,rOut,w){
926     const bl=pol(a-w,rIn), br=pol(a+w,rIn);
927     const wl=pol(a-w*0.62, rIn+(rOut-rIn)*0.58), wr=pol(a+w*0.62, rIn+(rOut-rIn)*0.58);
928     const tip=pol(a,rOut);
929     ctx.moveTo(bl[0],bl[1]);
930     const clp=pol(a-w*1.25, rIn+(rOut-rIn)*0.28);
931     ctx.bezierCurveTo(clp[0],clp[1], wl[0],wl[1], wl[0],wl[1]);
932     const c2p=pol(a-w*0.12, rOut*0.99);
933     ctx.bezierCurveTo(wl[0],wl[1], c2p[0],c2p[1], tip[0],tip[1]);
934     const c3p=pol(a+w*0.12, rOut*0.99);
935     ctx.bezierCurveTo(c3p[0],c3p[1], wr[0],wr[1], wr[0],wr[1]);
936     const c4p=pol(a+w*1.25, rIn+(rOut-rIn)*0.28);
937     ctx.bezierCurveTo(wr[0],wr[1], c4p[0],c4p[1], br[0],br[1]);
938     const cb=pol(a, rIn*0.84);
939     ctx.bezierCurveTo(cb[0],cb[1], cb[0],cb[1], bl[0],bl[1]);
940     ctx.closePath();
941 }
942 function drawHatayi(ctx,R,p,sw,c0,c1,c2,c3){
943     const m = 5 + (p.cmp>=5 ? 2 : (p.cmp>=3 ? 1 : 0));
944     const w = (Math.PI/m)*(0.62+p.dens*0.25);
945     /* tashqi gulbarglar */
946     for(let i=0;i<m;i++){
947         const a = TAU*i/m - Math.PI/2;
948         ctx.beginPath(); hatayiPetal(ctx, a, R*0.36, R*0.94, w);
949         fillA(ctx, i%2? c1 : lerpColor(c1,c3,0.4), 0.7);
950         st(ctx, shade(c0,0.35), sw*0.7);
951         const t0 = pol(a, R*0.42), t1 = pol(a, R*0.82);
952         ctx.beginPath(); ctx.moveTo(t0[0],t0[1]);
953         ctx.bezierCurveTo(t0[0],t0[1], t1[0],t1[1], t1[0],t1[1]);
954         st(ctx, shade(c2,0.15), sw*0.35);
955     }
956     /* oraliq kichik gulbarglar */
957     for(let i=0;i<m;i++){
958         ctx.beginPath(); hatayiPetal(ctx, TAU*(i+0.5)/m - Math.PI/2, R*0.2, R*0.46, w*0.66);
959         fillA(ctx, c3, 0.8); st(ctx, shade(c2,0.25), sw*0.5);
960     }
961     /* urug'don halqasi */
962     ctx.beginPath(); circle(ctx,0,0,R*0.2); ribbon(ctx, c1, shade(c1,0.4), sw*1.3+0.7);
963     const seeds = Math.max(6, m);
964     for(let i=0;i<seeds;i++){
965         const c = pol(TAU*i/seeds, R*0.13);
966         dot(ctx,c[0],c[1],R*0.024,shade(c0,0.15));
967     }
968     dot(ctx,0,0,R*0.055,c0);
969     ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
970 }
971
972 /* 12. PALAK – nilufar: 3 toza gulbarg halqasi */
973 function drawPalak(ctx,R,p,sw,c0,c1,c2,c3){
974     const n = Math.max(6, p.sym);
975     const rings = [
976         {rIn:R*0.55, rOut:R*0.95, w:(Math.PI/n)*0.8, off:0, f:c1, e:c0},
977         {rIn:R*0.32, rOut:R*0.6, w:(Math.PI/n)*0.72, off:0.5, f:c3, e:c2},
978         {rIn:R*0.14, rOut:R*0.36, w:(Math.PI/n)*0.62, off:0, f:lerpColor(c1,'#ffffff',0.3), e:c0}
979     ];
980     const cnt = 1 + Math.min(2, Math.round(p.cmp/3)+1);
981     for(let k=0;k<Math.min(rings.length,cnt+1);k++){
982         const rg = rings[k];
983         for(let i=0;i<n;i++){
984             const a = TAU*(i+rg.off)/n - Math.PI/2;
985             ctx.beginPath(); petalPath(ctx, a, rg.rIn, rg.rOut, rg.w);
986             fillA(ctx, i%2? rg.f : lerpColor(rg.f, rg.e, 0.18), 0.75);
987             st(ctx, shade(rg.e,0.3), sw*0.6*(1-k*0.12));
988         }
989     }

```

```

990   ctx.beginPath(); circle(ctx,0,0,R*0.13); ribbon(ctx, c0, shade(c0,0.45), sw*1.2+0.6);
991   dot(ctx,0,0,R*0.05,c1);
992   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
993 }
994
995 /* 13. SHAMSA — quyosh medalyoni: yulduz + nurlar + gulli tashqi band */
996 function drawShamsa(ctx,R,p,sw,c0,c1,c2,c3){
997   const n = Math.max(6, p.sym);
998   /* tashqi bandlar */
999   ctx.beginPath(); circle(ctx,0,0,R*0.98); ribbon(ctx, c1, shade(c1,0.45), sw*1.7+1);
1000  ctx.beginPath(); circle(ctx,0,0,R*0.8); st(ctx, c2, sw*0.5);
1001  /* tashqi gulbarg bandi 0.8R-0.97R */
1002  for(let i=0;i<n*2;i++){
1003    const a = TAU*(i+0.5)/(n*2) - Math.PI/2;
1004    ctx.beginPath(); petalPath(ctx, a, R*0.8, R*0.955, (Math.PI/(n*2))*0.8);
1005    fillA(ctx, i%2? c3 : c1, 0.65);
1006    st(ctx, shade(c0,0.3), sw*0.4);
1007  }
1008  /* nurlar — 0.42R dan 0.76R gacha, uzun/qisqa navbat */
1009  for(let i=0;i<n*2;i++){
1010    const a = TAU*i/(n*2) - Math.PI/2;
1011    const long = i%2===0;
1012    const r0 = R*0.4, len = R*(long? 0.36 : 0.22);
1013    const s0 = pol(a,r0);
1014    ctx.beginPath(); leaf(ctx, s0[0], s0[1], a, len, len*0.14);
1015    fillA(ctx, long? c0 : c2, 0.9);
1016    st(ctx, shade(long? c0 : c2, 0.3), sw*0.4);
1017  }
1018  /* markaziy yulduz o'zagi */
1019  ctx.beginPath(); circle(ctx,0,0,R*0.42); ribbon(ctx, c2, shade(c2,0.4), sw*1.2+0.6);
1020  ctx.beginPath(); starPath(ctx, n, R*0.36, R*0.17, -Math.PI/2);
1021  fillA(ctx, c0, 0.92);
1022  st(ctx, shade(c0,0.45), sw*0.7);
1023  ctx.beginPath(); starPath(ctx, n, R*0.2, R*0.1, -Math.PI/2 + Math.PI/n);
1024  fillA(ctx, c1, 0.95);
1025  dot(ctx,0,0,R*0.055,c3);
1026 }
1027
1028 /* 14. ARABESQUE — ogee to'r: S-egri lentalar + palmettalar */
1029 function drawArabesque(ctx,R,p,sw,c0,c1,c2,c3){
1030   const n = Math.max(5, p.sym);
1031   const rIn = R*0.4, rOut = R*0.88;
1032   /* ogee to'ri: har ichki tugundan ikki qo'shni tashqi tugunga S-egri */
1033   const inner=[], outer=[];
1034   for(let i=0;i<n;i++){
1035     inner.push(pol(TAU*i/n - Math.PI/2, rIn));
1036     outer.push(pol(TAU*(i+0.5)/n - Math.PI/2, rOut));
1037   }
1038   const sCurve=(A,B,dir)=>{
1039     const mx=(A[0]+B[0])/2, my=(A[1]+B[1])/2;
1040     const nx=-(B[1]-A[1]), ny=(B[0]-A[0]);
1041     const L2=Math.hypot(nx,ny)||1;
1042     const off = R*0.09*dir*(0.6+p.spir);
1043     ctx.moveTo(A[0],A[1]);
1044     ctx.bezierCurveTo(
1045       A[0]*0.6+mx*0.4 + nx/L2*off, A[1]*0.6+my*0.4 + ny/L2*off,
1046       B[0]*0.6+mx*0.4 - nx/L2*off, B[1]*0.6+my*0.4 - ny/L2*off,
1047       B[0],B[1]);
1048   };
1049   ctx.beginPath();
1050   for(let i=0;i<n;i++){ sCurve(inner[i], outer[i], 1); }
1051   ribbon(ctx, c0, shade(c0,0.45), sw*1.6+0.8);
1052   ctx.beginPath();
1053   for(let i=0;i<n;i++){ sCurve(inner[(i+1)%n], outer[i], -1); }
1054   ribbonI(ctx, c1, shade(c1,0.45), sw*1.6+0.8);
1055   /* tashqi tugunlarda palmetta */
1056   for(let i=0;i<n;i++){
1057     const a = TAU*(i+0.5)/n - Math.PI/2;
1058     budAt(ctx, outer[i][0], outer[i][1], a, R*0.14, c1, c0, sw);
1059   }
1060   /* ichki tugunlarda barg juftligi */
1061   for(let i=0;i<n;i++){
1062     const a = TAU*i/n - Math.PI/2;
1063     fineLeaf(ctx, inner[i][0], inner[i][1], a+Math.PI*0.82, R*0.1, R*0.03, c3, shade(c2,0.2), sw*0.4);
1064     fineLeaf(ctx, inner[i][0], inner[i][1], a-Math.PI*0.82+Math.PI*2, R*0.1, R*0.03, c3, shade(c2,0.2), sw*0.4);
1065   }
1066   /* markaz */
1067   ctx.beginPath(); circle(ctx,0,0,rIn*0.55); ribbon(ctx, c2, shade(c2,0.4), sw*1.2+0.6);
1068   ctx.beginPath(); starPath(ctx, n, rIn*0.4, rIn*0.22, -Math.PI/2);
1069   fillA(ctx,c0,0.9);
1070   dot(ctx,0,0,rIn*0.12,c1);
1071   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
1072 }
1073
1074 /* 15. SCROLL — bargli novda: S-poya + gajak uchi */
1075 function drawScroll(ctx,R,p,sw,c0,c1,c2,c3){
1076   const n = Math.max(3, Math.min(9, p.sym));
1077   for(let i=0;i<n;i++){
1078     ctx.save(); ctx.rotate(TAU*i/n);
1079     const p0=[R*0.16,0], p1=[R*0.36,-R*0.3*(1+p.skew*0.4)], p2=[R*0.58,R*0.3*(1+p.skew*0.4)], p3=[R*0.8,0];
1080     /* poya nuqtalari */

```

```

1081     const stemPts=[]; for(let j=0;j<=16;j++) stemPts.push(cubicAt(p0,p1,p2,p3,j/16));
1082     stem(ctx, stemPts, i%2? c1 : c0, sw*1.05);
1083     /* gajak uchi — kichik spiral */
1084     const endAng = Math.atan2(stemPts[16][1]-stemPts[15][1], stemPts[16][0]-stemPts[15][0]);
1085     const curl = spiralPts(0.22, Math.PI*1.6, R*0.14, i%2===1, 0)
1086     .map(q=>{
1087         const ca=Math.cos(endAng), sa=Math.sin(endAng);
1088         return [p3[0]+q[0]*ca-q[1]*sa, p3[1]+q[0]*sa+q[1]*ca];
1089     });
1090     stem(ctx, curl, c2, sw*0.7);
1091     /* navbatlashgan barglar */
1092     const leaves = 2 + Math.round(p.dens*3);
1093     for(let j=1;j<=leaves;j++){
1094         const t = j/(leaves+1);
1095         const q = cubicAt(p0,p1,p2,p3,t);
1096         const q2 = cubicAt(p0,p1,p2,p3,Math.min(1,t+0.02));
1097         const ang = Math.atan2(q2[1]-q[1], q2[0]-q[0]);
1098         fineLeaf(ctx, q[0], q[1], ang + (j%2? 1:-1)*Math.PI/2.3, R*0.12*(0.7+t*0.5), R*0.035, j%2? c3:c1, shade(c0,0
1099     .25), sw*0.45);
1100     }
1101     ctx.restore();
1102     }
1103     ctx.beginPath(); circle(ctx,0,0,R*0.1); ribbon(ctx, c0, shade(c0,0.45), sw*1.2+0.6);
1104     dot(ctx,0,0,R*0.04,c1);
1105     ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
1106 }
1107 /* 16. MEDALYON — ko'p bandli markaziy medalyon */
1108 function drawMedallion(ctx,R,p,sw,c0,c1,c2,c3){
1109     const n = Math.max(6, p.sym);
1110     /* tashqi band + scallop */
1111     ctx.beginPath(); circle(ctx,0,0,R*0.98); ribbon(ctx, c1, shade(c1,0.45), sw*1.7+1);
1112     ctx.beginPath();
1113     for(let i=0;i<n*2;i++){
1114         const a0 = TAU*i/(n*2)-Math.PI/2, a1 = TAU*(i+1)/(n*2)-Math.PI/2;
1115         const A=pol(a0,R*0.9), M=pol((a0+a1)/2, R*0.82);
1116         if(i===0) ctx.moveTo(A[0],A[1]);
1117         const B=pol(a1,R*0.9);
1118         ctx.bezierCurveTo(A[0]*0.94,A[1]*0.94, M[0],M[1], M[0],M[1]);
1119         ctx.bezierCurveTo(M[0],M[1], B[0]*0.94,B[1]*0.94, B[0],B[1]);
1120     }
1121     ctx.closePath();
1122     ribbon(ctx, c2, shade(c2,0.4), sw*1.1+0.5);
1123     /* gulbarg bandi */
1124     for(let i=0;i<n;i++){
1125         const a = TAU*i/n - Math.PI/2;
1126         ctx.beginPath(); petalPath(ctx, a, R*0.5, R*0.8, (Math.PI/n)*0.7);
1127         fillA(ctx, i%2? c1 : c3, 0.65);
1128         st(ctx, shade(c0,0.3), sw*0.5);
1129     }
1130     /* girih o'zagi */
1131     ctx.save(); ctx.scale(0.5,0.5);
1132     drawGirih(ctx, R*0.95, {...p, sym:n}, sw*1.6, c0, c1, c2, c3);
1133     ctx.restore();
1134 }
1135
1136 /* — CHINNI HOSHIYA BANDI — Rishton lagani uslubidagi ikki qatorli hoshiya — */
1137 function ceramicBorder(ctx,R,p,pal){
1138     const sw = p.sw;
1139     /* tashqi qalin band */
1140     ctx.beginPath(); circle(ctx,0,0,R*0.985); ribbon(ctx, pal.c[1], shade(pal.c[1],0.45), sw*2.6+2);
1141     ctx.beginPath(); circle(ctx,0,0,R*0.885); ribbon(ctx, pal.c[0], shade(pal.c[0],0.45), sw*1.4+0.8);
1142     /* band ichida: mini rozetta + romb navbatlashuvi */
1143     const m = Math.max(12, p.sym*3);
1144     const rb = R*0.935;
1145     for(let i=0;i<m;i++){
1146         const a = TAU*i/m - Math.PI/2;
1147         const q = pol(a, rb);
1148         ctx.save(); ctx.translate(q[0],q[1]); ctx.rotate(a+Math.PI/2);
1149         if(i%2===0){
1150             ctx.beginPath(); starPath(ctx, 8, R*0.033, R*0.015, 0);
1151             fillHard(ctx, pal.c[4], 0.95); st(ctx, shade(pal.c[0],0.35), sw*0.35);
1152             dot(ctx,0,0,R*0.008, pal.c[0]);
1153         } else {
1154             ctx.beginPath();
1155             polyPath(ctx, [[0, -R*0.028], [R*0.02,0], [0,R*0.028], [-R*0.02,0]]);
1156             fillHard(ctx, pal.c[3], 0.9); st(ctx, shade(pal.c[2],0.25), sw*0.3);
1157         }
1158         ctx.restore();
1159     }
1160     /* ichki nozik band: scallop + nuqta zanjiri */
1161     ctx.beginPath();
1162     const ms = m;
1163     for(let i=0;i<ms;i++){
1164         const a0 = TAU*i/ms - Math.PI/2, a1 = TAU*(i+1)/ms - Math.PI/2;
1165         const A = pol(a0, R*0.84), M = pol((a0+a1)/2, R*0.8), B = pol(a1, R*0.84);
1166         if(i===0) ctx.moveTo(A[0],A[1]);
1167         ctx.bezierCurveTo(A[0]*0.97,A[1]*0.97, M[0],M[1], M[0],M[1]);
1168         ctx.bezierCurveTo(M[0],M[1], B[0]*0.97,B[1]*0.97, B[0],B[1]);
1169     }
1170     ctx.closePath();

```

```

1171   st(ctx, shade(pal.c[2],0.15), sw*0.6);
1172   for(let i=0;i<m;i++){
1173     const q = pol(TAU*(i+0.5)/ms - Math.PI/2, R*0.815);
1174     dot(ctx, q[0], q[1], sw*0.65, pal.c[1]);
1175   }
1176   ctx.beginPath(); circle(ctx,0,0,R*0.72); st(ctx, pal.c[2], sw*0.5);
1177 }
1178 /* Chinni (lagan/piyola) kompozitsiyasi: hoshiya bandlari + markaziy medalyon */
1179 function drawPlateComposition(ctx,S,p){
1180   const pal = PAL[p.pal] || PAL.gold;
1181   ctx.save();
1182   ctx.translate(S/2, S/2);
1183   ceramicBorder(ctx, S*0.49, p, pal);
1184   ctx.restore();
1185   drawPattern(ctx, S/2, S/2, S*0.315, p);
1186 }
1187
1188 /* =====
1189 HANKIN USULI — "poligonlar aloqasi" (polygons-in-contact)
1190 Har plitka qirrasining O'RTASIDAN kontakt burchagi ostida ikki nur
1191 chiqadi; qo'shni nurlar kesishmasi strapwork chizig'ini beradi.
1192 ===== */
1193 function polyArea(poly){
1194   let a=0;
1195   for(let i=0;i<poly.length;i++){
1196     const A=poly[i], B=poly[(i+1)%poly.length];
1197     a += A[0]*B[1] - B[0]*A[1];
1198   }
1199   return a/2;
1200 }
1201 function hankinStraps(polyIn, ang){
1202   const poly = polyArea(polyIn) < 0 ? polyIn.slice().reverse() : polyIn;
1203   const m = poly.length, straps=[];
1204   const mids=[], dirs=[];
1205   for(let i=0;i<m;i++){
1206     const A=poly[i], B=poly[(i+1)%m];
1207     mids.push([(A[0]+B[0])/2, (A[1]+B[1])/2]);
1208     dirs.push(Math.atan2(B[1]-A[1], B[0]-A[0]));
1209   }
1210   for(let i=0;i<m;i++){
1211     const j=(i+1)%m;
1212     const X = rayX(mids[i], dirs[i]+ang, mids[j], dirs[j]+Math.PI-ang);
1213     if(X) straps.push([mids[i], X, mids[j]]);
1214   }
1215   return straps;
1216 }
1217 function strokeStraps(ctx, straps, band, edge, w, weave){
1218   const draw = (sset)=>{
1219     for(const sp of sset){
1220       ctx.beginPath();
1221       ctx.moveTo(sp[0][0],sp[0][1]);
1222       ctx.lineTo(sp[1][0],sp[1][1]);
1223       ctx.lineTo(sp[2][0],sp[2][1]);
1224       weave ? ribbonI(ctx, band, edge, w) : ribbon(ctx, band, edge, w);
1225     }
1226   };
1227   draw(straps);
1228 }
1229 /* 4.8.8 plitkalash geometriyasi (sakkizburchak + kvadrat) */
1230 function tiles488(R, cells){
1231   const s = R*1.9/cells;
1232   const oR = s*0.5/Math.cos(Math.PI/8);
1233   const octs=[], sqs=[];
1234   const n2 = Math.ceil(cells/2)+1;
1235   for(let gy=-n2; gy<=n2; gy++){
1236     for(let gx=-n2; gx<=n2; gx++){
1237       const x=gx*s, y=gy*s;
1238       if(Math.hypot(x,y) < R*1.15){
1239         const o=[]; for(let i=0;i<8;i++) o.push([x+Math.cos(Math.PI/8+TAU*i/8)*oR, y+Math.sin(Math.PI/8+TAU*i/8)*o
R]);
1240         octs.push(o);
1241       }
1242       const qx=x+s/2, qy=y+s/2;
1243       if(Math.hypot(qx,qy) < R*1.15){
1244         const q = oR*Math.sin(Math.PI/8)*Math.SQRT2;
1245         sqs.push([[qx,qy-q],[qx+q,qy],[qx,qy+q],[qx-q,qy]]);
1246       }
1247     }
1248   }
1249   return {octs, sqs, s, oR};
1250 }
1251 /* GIRIH HANKIN 4.8.8 — yulduz-xoch klassikasi (Registon poli uslubi) */
1252 function drawHankin8(ctx,R,p,sw,c0,c1,c2,c3){
1253   const FIELD = !!p._field;
1254   const cells = FIELD ? Math.max(2, Math.round(R*1.9/(p._cellpx||150))) : 2 + Math.round(p.dens*2);
1255   const T = tiles488(R, cells);
1256   const ang = Math.PI*(0.2 + p.spir*0.18); /* kontakt burchagi ≈ 36°–68° */
1257   const edge = shade(c0, 0.5);
1258   const bw = sw*2.0 + 1.2;
1259   ctx.save();
1260   if(!FIELD){

```

```

1261     ctx.beginPath(); circle(ctx,0,0,R*0.985);
1262     if(!ctx._svg) ctx.clip();
1263 }
1264 /* plitka fonlari */
1265 for(let i=0;i<T.octs.length;i++){
1266     ctx.beginPath(); polyPath(ctx, T.octs[i]);
1267     fillA(ctx, i%2? c3 : lerpColor(c3,'#ffffff',0.4), 0.6);
1268 }
1269 for(const q of T.sqs){
1270     ctx.beginPath(); polyPath(ctx, q);
1271     fillA(ctx, c1, 0.55);
1272 }
1273 /* Hankin straplari: oq/och lenta – haqiqiy girih ko'rinishi */
1274 const band = lerpColor(c3,'#ffffff',0.75);
1275 for(const o of T.octs) strokeStraps(ctx, hankinStraps(o, ang), band, edge, bw, false);
1276 for(const q of T.sqs) strokeStraps(ctx, hankinStraps(q, ang), band, edge, bw, true);
1277 ctx.restore();
1278 if(!FIELD){
1279     ctx.beginPath(); circle(ctx,0,0,R*0.985); ribbon(ctx, c1, shade(c1,0.45), sw*2+1.2);
1280     ctx.beginPath(); circle(ctx,0,0,R*0.9); st(ctx, c2, sw*0.5);
1281 }
1282 }
1283 /* GIRIH HANKIN 6.6.6 – geksagonal to'rdan 6-yulduzlar */
1284 function drawHankin6(ctx,R,p,sw,c0,c1,c2,c3){
1285     const FIELD = !!p._field;
1286     const s = FIELD ? Math.max(30,(p._cellpx||150)*0.5) : R/(1.5 + p.dens*1.6);
1287     const dx = s*Math.sqrt(3), dy = s*1.5;
1288     const ang = Math.PI*(0.24 + p.spir*0.16);
1289     const edge = shade(c0, 0.5);
1290     const bw = sw*2.0 + 1.2;
1291     const band = lerpColor(c3,'#ffffff',0.75);
1292     ctx.save();
1293     if(!FIELD){
1294         ctx.beginPath(); circle(ctx,0,0,R*0.985);
1295         if(!ctx._svg) ctx.clip();
1296     }
1297     const hexes=[];
1298     const NN = FIELD ? Math.ceil(R/(s*1.5))+1 : 5;
1299     for(let gy=-NN; gy<=NN; gy++){
1300         for(let gx=-NN; gx<=NN; gx++){
1301             const x = gx*dx + (gy%2 ? dx/2 : 0), y = gy*dy;
1302             if(Math.hypot(x,y) > R*1.15) continue;
1303             const h=[]; for(let i=0;i<6;i++) h.push([x+Math.cos(Math.PI/6+TAU*i/6)*s, y+Math.sin(Math.PI/6+TAU*i/6)*s]);
1304             hexes.push({h, gx, gy});
1305         }
1306     }
1307     for(const {h,gx,gy} of hexes){
1308         ctx.beginPath(); polyPath(ctx, h);
1309         fillA(ctx, ((gx+gy*2)%3+3)%3===0 ? c1 : ((gx+gy*2)%3+3)%3===1 ? c3 : lerpColor(c3,'#ffffff',0.4), 0.55);
1310     }
1311     for(const {h} of hexes) strokeStraps(ctx, hankinStraps(h, ang), band, edge, bw, false);
1312     ctx.restore();
1313     if(!FIELD){
1314         ctx.beginPath(); circle(ctx,0,0,R*0.985); ribbon(ctx, c1, shade(c1,0.45), sw*2+1.2);
1315     }
1316 }
1317 /* SAKKIZ QIRRALI GIRIH – klassik kvadrat o'quv paneli
1318 (markaziy 8-yulduz rozetta + kvadrat ramka, to'liq bo'yalgan) */
1319 function drawPanelGirih(ctx,R,p,sw,c0,c1,c2,c3){
1320     const H = R*0.94;
1321     const edge = shade(c0, 0.5);
1322     const bw = sw*2.4 + 1.6;
1323     const band = lerpColor(c3,'#ffffff',0.8);
1324     /* tashqi ramka bandlari */
1325     ctx.beginPath(); polyPath(ctx, [[-H,-H],[H,-H],[H,H],[-H,H]]);
1326     ribbon(ctx, c1, shade(c1,0.45), bw*1.1);
1327     const H2 = H*0.88;
1328     ctx.beginPath(); polyPath(ctx, [[-H2,-H2],[H2,-H2],[H2,H2],[-H2,H2]]);
1329     fillA(ctx, c3, 0.45);
1330     ribbon(ctx, c0, edge, bw*0.7);
1331     /* ramka orasidagi mini-rozetkalar */
1332     const mm = 4 + Math.round(p.dens*3);
1333     const rimR = (H+H2)/2;
1334     for(let side=0; side<4; side++){
1335         for(let i=1;i<mm;i++){
1336             const t = i/mm*2 - 1;
1337             let x,y;
1338             if(side===0){ x=t*rimR; y=-rimR; } else if(side===1){ x=rimR; y=t*rimR; }
1339             else if(side===2){ x=t*rimR; y=rimR; } else { x=-rimR; y=t*rimR; }
1340             dot(ctx,x,y,(H-H2)*0.22, i%2? c0 : band);
1341         }
1342     }
1343     /* burchak choraklari */
1344     for(const [sx,sy] of [[-1,-1],[1,-1],[1,1],[-1,1]]){
1345         ctx.beginPath();
1346         ctx.moveTo(sx*H2, sy*H2);
1347         ctx.lineTo(sx*H2, sy*H2*0.55);
1348         ctx.bezierCurveTo(sx*H2*0.8, sy*H2*0.6, sx*H2*0.6, sy*H2*0.8, sx*H2*0.55, sy*H2);
1349         ctx.closePath();
1350         fillA(ctx, c0, 0.6);
1351         st(ctx, edge, sw*0.5);

```

```

1352 }
1353 /* MARKAZIY 8-YULDUZ ROZETTA: ikki kvadrat + kite gulbarglar */
1354 const r = H2*0.66;
1355 /* kite gulbarglar (yulduz uchlaridan tashqariga) */
1356 const g = {R0:H2*0.92, r2:r*0.78, rm:r*0.9, w:Math.PI/8*0.8};
1357 for(let i=0;i<8;i++){
1358   const a = TAU*i/8 - Math.PI/2;
1359   ctx.beginPath(); kitePath(ctx, a, g);
1360   fillA(ctx, i%2? c1 : c3, 0.75);
1361   st(ctx, shade(c2,0.2), sw*0.4);
1362 }
1363 /* yulduz maydoni */
1364 ctx.beginPath(); starPath(ctx, 8, r, r*0.765, -Math.PI/2);
1365 fillA(ctx, c1, 0.92);
1366 /* ikki kvadrat — oq lenta straplar (haqiqiy panel uslubi) */
1367 const sq=(rot)=>{ const q=[]; for(let i=0;i<4;i++) q.push(pol(rot+TAU*i/4, r)); return q; };
1368 ctx.beginPath(); polyPath(ctx, sq(-Math.PI/2));
1369 ribbon(ctx, band, edge, bw);
1370 ctx.beginPath(); polyPath(ctx, sq(-Math.PI/2+Math.PI/4));
1371 ribbonI(ctx, band, edge, bw);
1372 /* birinchi kvadratning ikki qarama-qarshi tomoni qayta ustidan — weave */
1373 const A = sq(-Math.PI/2);
1374 for(const j of [0,2]){
1375   const P1=A[j], P2=A[(j+1)%4];
1376   ctx.beginPath();
1377   ctx.moveTo(P1[0]+(P2[0]-P1[0])*0.3, P1[1]+(P2[1]-P1[1])*0.3);
1378   ctx.lineTo(P1[0]+(P2[0]-P1[0])*0.7, P1[1]+(P2[1]-P1[1])*0.7);
1379   ribbonI(ctx, band, edge, bw);
1380 }
1381 /* markaziy sakkizburchak + gul */
1382 const oct=[]; for(let i=0;i<8;i++) oct.push(pol(-Math.PI/2+Math.PI/8+TAU*i/8, r*0.545));
1383 ctx.beginPath(); polyPath(ctx, oct);
1384 fillA(ctx, c0, 0.9);
1385 st(ctx, edge, sw*0.5);
1386 for(let i=0;i<8;i++){
1387   ctx.beginPath();
1388   petalPath(ctx, TAU*i/8 - Math.PI/2, r*0.1, r*0.42, Math.PI/8*0.62);
1389   fillA(ctx, band, 0.9);
1390   st(ctx, shade(c1,0.3), sw*0.35);
1391 }
1392 dot(ctx,0,0,r*0.09,c1);
1393 }
1394 /* STAKAN uchun FRIZ kompozitsiyasi — gorizontal band, chokka mos takror */
1395 function drawFriezeComposition(ctx,S,p){
1396   const pal = PAL[p.pal] || PAL.gold;
1397   const line=(y,w,col)=>{
1398     ctx.beginPath(); ctx.moveTo(-6,y); ctx.lineTo(S+6,y);
1399     ribbon(ctx, col, shade(col,0.45), w);
1400   };
1401   line(S*0.14, p.sw*2.4+1.6, pal.c[1]);
1402   line(S*0.225, p.sw*1.2+0.6, pal.c[0]);
1403   line(S*0.775, p.sw*1.2+0.6, pal.c[0]);
1404   line(S*0.86, p.sw*2.4+1.6, pal.c[1]);
1405   /* band foni — naqsh tagida yengil tus */
1406   ctx.save();
1407   ctx.globalAlpha = 0.38;
1408   ctx.fillStyle = pal.c[4];
1409   ctx.fillRect(0, S*0.27, S, S*0.46);
1410   ctx.restore();
1411   /* motif qatori — zichroq, teng qadam: chokda uzilmaydi */
1412   const m = Math.max(6, Math.min(12, p.sym));
1413   for(let i=0;i<m;i++){
1414     drawPattern(ctx, (i+0.5)*S/m, S*0.5, (S/m)*0.42, {...p, lay:1});
1415   }
1416   /* ajratkich romblar (chok nuqtasida ham) */
1417   for(let i=0;i<m;i++){
1418     const x = i*S/m;
1419     ctx.beginPath();
1420     polyPath(ctx,[[x,S*0.5-S*0.045],[x+S*0.026,S*0.5],[x,S*0.5+S*0.045],[x-S*0.026,S*0.5]]);
1421     fillHard(ctx, pal.c[1], 0.95);
1422     st(ctx, shade(pal.c[1],0.4), p.sw*0.4);
1423   }
1424   /* yuqori nuqta zanjiri */
1425   for(let i=0;i<m*3;i++){
1426     const x = (i+0.5)*S/(m*3);
1427     dot(ctx, x, S*0.185, p.sw*0.7, pal.c[0]);
1428     dot(ctx, x, S*0.815, p.sw*0.7, pal.c[0]);
1429   }
1430 }
1431 /* =====
1432 C. KOMBINATSIYALAR (4) — tashqi uslub + ichki uslub (0.5R)
1433 ===== */
1434 function makeCombo(fA,fB){
1435   return function(ctx,R,p,sw,c0,c1,c2,c3){
1436     fA(ctx,R,p,sw,c0,c1,c2,c3);
1437     ctx.save();
1438     ctx.scale(0.5,0.5);
1439     ctx.rotate(Math.PI/Math.max(3,p.sym));
1440     fB(ctx,R,p,sw*1.7,c1,c0,c3,c2);
1441     ctx.restore();
1442   };

```

```

1443 }
1444 function drawGirihIslimiy(ctx,R,p,sw,c0,c1,c2,c3){ makeCombo(drawGirih,drawIslimiy)(ctx,R,p,sw,c0,c1,c2,c3); }
1445 function drawShamsaGirih(ctx,R,p,sw,c0,c1,c2,c3){ makeCombo(drawShamsa,drawGirih)(ctx,R,p,sw,c0,c1,c2,c3); }
1446 function drawMuqarnasIslimiy(ctx,R,p,sw,c0,c1,c2,c3){ makeCombo(drawMuqarnas,drawIslimiy)(ctx,R,p,sw,c0,c1,c2,c3); }
1447 }
1448 function drawXatamRumi(ctx,R,p,sw,c0,c1,c2,c3){ makeCombo(drawXatam,drawRumi)(ctx,R,p,sw,c0,c1,c2,c3); }
1449
1450 /* =====
1451 D. DUNYO NAQSHLARI (8)
1452 ===== */
1453
1454 /* 21. Celtic – 2 tolali torus o'rmasi: haqiqiy navbatlashgan to'qish */
1455 function drawCeltic(ctx,R,p,sw,c0,c1,c2,c3){
1456   const k = Math.max(4, p.sym);
1457   const rm = R*0.66, a = R*(0.14 + p.dens*0.1);
1458   const bw = sw*2.6 + 2;
1459   const strandSeg = (phase, j)=>{ /* j-segment:  $\theta \in [j\pi/k, (j+1)\pi/k]$  */
1460     const pts=[];
1461     const steps = 14;
1462     for(let i=0;i<=steps;i++){
1463       const th = (j + i/steps)*Math.PI/k;
1464       pts.push(pol(th - Math.PI/2, rm + a*Math.sin(k*th + phase)));
1465     }
1466     ctx.beginPath(); smoothPath(ctx, pts, false);
1467   };
1468   /* ikki tola, 2k segment; naqsh navbatlashadi: (tola + j) juft → ostda, toq → ustda */
1469   for(let pass=0; pass<2; pass++){
1470     for(let strand=0; strand<2; strand++){
1471       const phase = strand*Math.PI;
1472       const col = strand? c1 : c0;
1473       for(let j=0; j<2*k; j++){
1474         if(((strand + j) % 2) !== pass) continue;
1475         strandSeg(phase, j);
1476         if(pass===0) ribbon(ctx, col, shade(col,0.5), bw);
1477         else ribbonI(ctx, col, shade(col,0.5), bw);
1478       }
1479     }
1480   }
1481   /* ichki/tashqi chegara bandlari */
1482   ctx.beginPath(); circle(ctx,0,0, rm - a - bw*0.9); ribbon(ctx, c2, shade(c2,0.4), sw*1.2+0.6);
1483   ctx.beginPath(); circle(ctx,0,0, rm + a + bw*0.9); ribbon(ctx, c2, shade(c2,0.4), sw*1.2+0.6);
1484   /* markaziy tugma */
1485   if(p.cmp>=3){
1486     ctx.beginPath(); starPath(ctx, k, (rm-a)*0.5, (rm-a)*0.28, -Math.PI/2);
1487     fillA(ctx, c3, 0.8); st(ctx, shade(c0,0.35), sw*0.5);
1488     dot(ctx,0,0,(rm-a)*0.12,c0);
1489   }
1490   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
1491 }
1492
1493 /* 22. Mandala – konsentrik halqalar, har halqada boshqa element */
1494 function drawMandala(ctx,R,p,sw,c0,c1,c2,c3){
1495   const rings = Math.max(6, Math.min(12, 5 + p.cmp + Math.round(p.dens*2)));
1496   const cols = [c0,c1,c2,c3];
1497   for(let k=0;k<rings;k++){
1498     const t = k/(rings-1);
1499     const r = R*(0.12 + 0.85*t);
1500     const m = Math.max(6, Math.round(p.sym*(1 + t*1.5)));
1501     const col = cols[k%4];
1502     const kind = k%5;
1503     if(kind===0){
1504       for(let i=0;i<m;i++){ const c=pol(TAU*i/m, r); dot(ctx,c[0],c[1], R*0.014+sw*0.5, col); }
1505     } else if(kind===1){
1506       const step = R*0.85/(rings-1);
1507       for(let i=0;i<m;i++){
1508         ctx.beginPath();
1509         petalPath(ctx, TAU*i/m, r-step*0.5, r+step*0.55, (Math.PI/m)*0.75);
1510         fillA(ctx,col,0.5); st(ctx, c0, sw*0.5);
1511       }
1512     } else if(kind===2){
1513       ctx.beginPath();
1514       const pts=[];
1515       for(let i=0;i<=m*6;i++){
1516         const a = TAU*i/(m*6);
1517         pts.push(pol(a, r*(1 + 0.035*Math.sin(a*m + k))));
1518       }
1519       smoothPath(ctx,pts,true);
1520       st(ctx,col,sw*0.7);
1521     } else if(kind===3){
1522       const step = R*0.85/(rings-1);
1523       for(let i=0;i<m;i++){
1524         const a = TAU*(i+0.5)/m, c = pol(a,r);
1525         ctx.beginPath(); leaf(ctx, c[0], c[1], a, step*0.9, step*0.28*(1+p.spir));
1526         fillA(ctx,col,0.7); st(ctx,c0,sw*0.4);
1527       }
1528     } else {
1529       ctx.beginPath(); circle(ctx,0,0,r); st(ctx,col,sw*0.55);
1530     }
1531   }
1532   dot(ctx,0,0,R*0.05,c0);
1533 }

```

```

1533
1534 /* 23. Penrose – kite/dart aperiodik subdivide, oltin nisbat */
1535 function drawPenrose(ctx,R,p,sw,c0,c1,c2,c3){
1536   const PHI = 1.6180339887;
1537   let tris = [];
1538   for(let i=0;i<10;i++){
1539     let B = pol(TAU*(2*i-1)/20 - Math.PI/2, R*0.98);
1540     let C = pol(TAU*(2*i+1)/20 - Math.PI/2, R*0.98);
1541     if(i%2===0){ const t=B; B=C; C=t; }
1542     tris.push([0,[0,0],B,C]);
1543   }
1544   const depth = Math.max(1, Math.min(6, p.cmp));
1545   const div = (x1,x2,f)=>[x1[0]+(x2[0]-x1[0])/f, x1[1]+(x2[1]-x1[1])/f];
1546   for(let d=0; d<depth; d++){
1547     const next=[];
1548     for(const T of tris){
1549       const [ty,A,B,C] = T;
1550       if(ty===0){
1551         const Pp = div(A,B,PHI);
1552         next.push([0,C,Pp,B],[1,Pp,C,A]);
1553       } else {
1554         const Q = div(B,A,PHI), Rr = div(B,C,PHI);
1555         next.push([1,Rr,C,A],[0,Q,Rr,B],[1,Rr,Q,A]);
1556       }
1557     }
1558     tris = next;
1559   }
1560   for(const T of tris){
1561     const [ty,A,B,C] = T;
1562     ctx.beginPath(); polyPath(ctx,[A,B,C]);
1563     fillA(ctx, ty? c1:c3, 0.75);
1564     st(ctx, c0, sw*0.4);
1565   }
1566   ctx.beginPath(); circle(ctx,0,0,R*0.99); st(ctx,c2,sw);
1567 }
1568
1569 /* 24. Zellige – Marokash kafel: egri tomonli romblar, 5 rang */
1570 function drawZellige(ctx,R,p,sw,c0,c1,c2,c3){
1571   const FIELD = !!p._field;
1572   const palc = PAL[p.pal] ? PAL[p.pal].c : [c0,c1,c2,c3,c0];
1573   const s = FIELD ? Math.max(30,(p._cellpx||150)*0.55) : R/(1.6 + p.dens*2.2);
1574   const bow = s*0.2*(0.4 + p.spir);
1575   const curvedQuad = (pts)=>{
1576     ctx.moveTo(pts[0][0],pts[0][1]);
1577     for(let i=0;i<4;i++){
1578       const A=pts[i], B=pts[(i+1)%4];
1579       const mx=(A[0]+B[0])/2, my=(A[1]+B[1])/2;
1580       const nx=-(B[1]-A[1]), ny=(B[0]-A[0]);
1581       const L=Math.hypot(nx,ny)||1;
1582       const sgn = i%2? 1:-1;
1583       const cx2=mx+nx/L*bow*sgn, cy2=my+ny/L*bow*sgn;
1584       ctx.bezierCurveTo(
1585         A[0]+(cx2-A[0])*0.7, A[1]+(cy2-A[1])*0.7,
1586         B[0]+(cx2-B[0])*0.7, B[1]+(cy2-B[1])*0.7,
1587         B[0],B[1]);
1588     }
1589     ctx.closePath();
1590   };
1591   const NZ = FIELD ? Math.ceil(R/(s*0.5))+2 : 4;
1592   for(let gy=-NZ; gy<=NZ; gy++){
1593     for(let gx=-NZ; gx<=NZ; gx++){
1594       const x = (gx+gy)*s*0.5*1.6, y = (gy-gx)*s*0.5;
1595       if(Math.hypot(x,y) > R*(FIELD?1.5:1.03)) continue;
1596       const q=[[x,y-s*0.5],[x+s*0.8,y],[x,y+s*0.5],[x-s*0.8,y]];
1597       ctx.beginPath(); curvedQuad(q);
1598       const ci = ((gx%5)+(gy%2*5)+10)%5;
1599       fillA(ctx, palc[ci], 0.85);
1600       st(ctx, c0, sw*0.6);
1601       if(p.cmp>=5){ dot(ctx,x,y,s*0.07,palc[(ci+2)%5]); }
1602     }
1603   }
1604   if(!FIELD){ ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c0,sw); }
1605 }
1606
1607 /* 25. Batik – to'lqinli konsentrik halqalar + gul motivlari */
1608 function drawBatik(ctx,R,p,sw,c0,c1,c2,c3){
1609   const rings = 3 + Math.min(5, p.cmp);
1610   const n = Math.max(4,p.sym);
1611   const A = R*0.035*(0.6 + p.dens);
1612   for(let k=0;k<rings;k++){
1613     const rk = R*(0.18 + 0.8*k/(rings-1||1));
1614     const pts=[];
1615     const steps = Math.max(48, n*10);
1616     for(let i=0;i<steps;i++){
1617       const a = TAU*i/steps;
1618       pts.push(pol(a, rk + A*Math.sin(n*a + k*1.3 + p.skew)));
1619     }
1620     ctx.beginPath(); smoothPath(ctx,pts,true);
1621     st(ctx, k%2? c1:c0, sw*(0.7 + 0.3*(k%2)));
1622     if(k%2===1 && k<rings-1){
1623       const m = n + k;

```

```

1624     for(let i=0;i<m;i++){
1625         const c = pol(TAU*(i+0.3)/m, rk + R*0.35/rings);
1626         for(let j=0;j<5;j++){
1627             const q = pol(TAU*j/5, R*0.028*(1+p.spir));
1628             dot(ctx, c[0]+q[0], c[1]+q[1], R*0.014, c3);
1629         }
1630         dot(ctx,c[0],c[1],R*0.012,c2);
1631     }
1632 }
1633 }
1634 dot(ctx,0,0,R*0.05*p.ir*2+2,c0);
1635 }
1636
1637 /* 26. Paisley – buta/tomchi: 4 Bezier teardrop, ichida spiral */
1638 function drawPaisley(ctx,R,p,sw,c0,c1,c2,c3){
1639     const n = Math.max(3,p.sym);
1640     for(let i=0;i<n;i++){
1641         ctx.save();
1642         ctx.rotate(TAU*i/n);
1643         ctx.translate(R*0.52, 0);
1644         ctx.rotate(Math.PI/2 + p.skew*0.6);
1645         const L = R*0.42*(0.8+p.dens*0.4), W = L*0.58;
1646         /* boteh: 4 Bezier – asimmetrik tomchi, uchi bukilgan */
1647         ctx.beginPath();
1648         ctx.moveTo(0, L*0.5);
1649         ctx.bezierCurveTo(-W, L*0.5, -W, -L*0.35, -W*0.25, -L*0.62);
1650         ctx.bezierCurveTo(-W*0.05, -L*0.75, W*0.3, -L*0.78, W*0.38, -L*0.6);
1651         ctx.bezierCurveTo(W*0.44, -L*0.46, W*0.1, -L*0.42, W*0.12, -L*0.28);
1652         ctx.bezierCurveTo(W*0.55, -L*0.05, W*0.62, L*0.32, 0, L*0.5);
1653         ctx.closePath();
1654         fillA(ctx, i%2?c1:c3, 0.55);
1655         ribbonI(ctx, c0, shade(c0,0.5), sw*1.8+1);
1656         /* ichki hoshiya konturi */
1657         ctx.save(); ctx.scale(0.82,0.82);
1658         ctx.beginPath();
1659         ctx.moveTo(0, L*0.5);
1660         ctx.bezierCurveTo(-W, L*0.5, -W, -L*0.35, -W*0.25, -L*0.62);
1661         ctx.bezierCurveTo(-W*0.05, -L*0.75, W*0.3, -L*0.78, W*0.38, -L*0.6);
1662         ctx.bezierCurveTo(W*0.44, -L*0.46, W*0.1, -L*0.42, W*0.12, -L*0.28);
1663         ctx.bezierCurveTo(W*0.55, -L*0.05, W*0.62, L*0.32, 0, L*0.5);
1664         ctx.closePath();
1665         ctx.restore();
1666         st(ctx, shade(c2,0.2), sw*0.45);
1667         /* ichki spiral */
1668         const sp = spiralPts(0.16+p.spir*0.2, Math.PI*2.1, L*0.42, i%2===1, 0);
1669         ctx.beginPath(); smoothPath(ctx, sp.map(q=>[q[0], q[1]+L*0.05]), false);
1670         st(ctx, c2, sw*0.6);
1671         /* uchidagi 5-barg gul */
1672         for(let j=0;j<5;j++){
1673             const q = pol(TAU*j/5, L*0.1);
1674             dot(ctx, q[0]-W*0.05, q[1]-L*0.7, L*0.055, c1);
1675         }
1676         dot(ctx, -W*0.05, -L*0.7, L*0.045, c0);
1677         /* chekka nuqta bezaklari */
1678         if(p.cmp>=4){
1679             for(let j=0;j<7;j++){
1680                 const t = j/6;
1681                 dot(ctx, -W*(1.12), L*0.4 - t*L*0.8, sw*0.8, c2);
1682             }
1683         }
1684         ctx.restore();
1685     }
1686     ctx.beginPath(); circle(ctx,0,0,R*p.ir*0.4+2); st(ctx,c0,sw*0.7);
1687 }
1688
1689 /* 27. Greek Key – meander labirint, konsentrik kvadrat ramkalar */
1690 function drawGreek(ctx,R,p,sw,c0,c1,c2,c3){
1691     const frames = 2 + Math.min(4, Math.round(p.cmp/2));
1692     const meanderSide = (x0,y0,x1,y1,units,h)=>{
1693         const dx=(x1-x0)/units, dy=(y1-y0)/units;
1694         const nx=-dy, ny=dx; /* ichkariga normal */
1695         const L=Math.hypot(nx,ny)||1;
1696         const hx=nx/L*h, hy=ny/L*h;
1697         for(let u=0;u<units;u++){
1698             const bx=x0+dx*u, by=y0+dy*u;
1699             const s=[[0,0],[0.72,0],[0.72,0.72],[0.24,0.72],[0.24,0.3],[0.48,0.3],[0.48,0.52],[1,0.52]];
1700             ctx.moveTo(bx,by);
1701             for(const q of s){
1702                 ctx.lineTo(bx+dx*q[0]+hx*q[1], by+dy*q[0]+hy*q[1]);
1703             }
1704         }
1705     };
1706     for(let k=0;k<frames;k++){
1707         const half = R*0.68*(1 - k*0.24);
1708         if(half < R*0.1) break;
1709         const units = Math.max(3, Math.round(p.sym*(0.5+p.dens*0.5)) - k);
1710         const h = half*0.28;
1711         const c=[[-half,-half],[half,-half],[half,half],[-half,half]];
1712         ctx.beginPath();
1713         for(let e=0;e<4;e++){
1714             const A=c[e], B=c[(e+1)%4];

```

```

1715     meanderSide(A[0],A[1],B[0],B[1],units,h);
1716   }
1717   st(ctx, k%2?c1:c0, sw*(1-k*0.1));
1718   ctx.beginPath(); polyPath(ctx,c);
1719   st(ctx, c2, sw*0.5);
1720 }
1721 ctx.beginPath(); starPath(ctx, 4, R*p.ir*0.35, R*p.ir*0.15, Math.PI/4);
1722 fillA(ctx,c3,0.8); st(ctx,c0,sw*0.7);
1723 }
1724
1725 /* 28. Xitoy panjara – fretwork kataklar, T-shakl bo'linmalar */
1726 function drawXitoy(ctx,R,p,sw,c0,c1,c2,c3){
1727   const FIELD = !!p._field;
1728   const half = FIELD ? R*1.42 : R*0.92;
1729   const cells = FIELD ? Math.max(2, Math.round(half*2/(p._cellpx||150))) : Math.max(2, Math.round(2 + p.dens*3));
1730   const s = half*2/cells;
1731   ctx.beginPath();
1732   polyPath(ctx,[[ -half,-half],[half,-half],[half,half],[ -half,half]]);
1733   st(ctx,c0,sw*1.4);
1734   ctx.beginPath();
1735   for(let i=1;i<cells;i++){
1736     ctx.moveTo(-half+i*s,-half); ctx.lineTo(-half+i*s,half);
1737     ctx.moveTo(-half,-half+i*s); ctx.lineTo(half,-half+i*s);
1738   }
1739   st(ctx,c0,sw*0.9);
1740   /* har katakda T-motivlar navbatlashgan yo'nalishda */
1741   const tLen = s*0.34*(0.6+p.ir), tBar = s*0.24;
1742   for(let gy=0; gy<cells; gy++){
1743     for(let gx=0; gx<cells; gx++){
1744       const cx = -half + gx*s + s/2, cy = -half + gy*s + s/2;
1745       const rot = ((gx+gy)%4) * Math.PI/2 + p.skew*0.4;
1746       ctx.save(); ctx.translate(cx,cy); ctx.rotate(rot);
1747       ctx.beginPath();
1748       /* T ning oyog'i katak CHEGARASIGACHA boradi – panjara bilan
1749          tutashadi. Ilgari u faqat tLen gacha chizilar va motiv katak
1750          ichida "suzib" qolardi: naqsh uzilgan ko'rinardi, lazerda
1751          kesilganda esa bunday bo'lak paneldan tushib ketardi. */
1752       ctx.moveTo(0,-s/2); ctx.lineTo(0,tLen*0.2);
1753       ctx.moveTo(-tBar,tLen*0.2); ctx.lineTo(tBar,tLen*0.2);
1754       ctx.moveTo(-tBar,tLen*0.2); ctx.lineTo(-tBar,tLen*0.62);
1755       ctx.moveTo(tBar,tLen*0.2); ctx.lineTo(tBar,tLen*0.62);
1756       ctx.restore();
1757       st(ctx, (gx+gy)%2? c1:c2, sw*0.75);
1758       /* Burchak qavslari – T0'RTALA burchakda, katakdan biroz ichkarida.
1759          Ilgari faqat ikki qarama-qarshi burchakda chizilar va ular
1760          panjara chizig'i ustiga tushib, tasodifiy chiziqchalardek
1761          ko'rinardi. To'rttasi birgalikda ichki ramka hosil qiladi –
1762          xitoy fretworkidagi klassik motiv. */
1763       if(p.cmp>=4){
1764         ctx.beginPath();
1765         const q = s*0.15, inn = s*0.06, h2 = s/2 - inn;
1766         for(const [sx2, sy2] of [[-1,-1],[1,-1],[1,1],[-1,1]]){
1767           const px = cx + sx2*h2, py = cy + sy2*h2;
1768           ctx.moveTo(px - sx2*q, py); ctx.lineTo(px, py); ctx.lineTo(px, py - sy2*q);
1769         }
1770         st(ctx,c3,sw*0.6);
1771       }
1772     }
1773   }
1774 }
1775
1776 /* =====
1777 E. MATEMATIK FRAKTALLAR (7)
1778 ===== */
1779
1780 /* 29. Koch qor parchasi */
1781 function drawKoch(ctx,R,p,sw,c0,c1,c2,c3){
1782   const depth = Math.max(1, Math.min(5, p.cmp));
1783   let pts = [];
1784   for(let i=0;i<3;i++) pts.push(pol(TAU*i/3 - Math.PI/2, R*0.85));
1785   for(let d=0; d<depth; d++){
1786     const next=[];
1787     for(let i=0;i<pts.length;i++){
1788       const A=pts[i], B=pts[(i+1)%pts.length];
1789       const dx=(B[0]-A[0])/3, dy=(B[1]-A[1])/3;
1790       const P1=[A[0]+dx, A[1]+dy], P3=[A[0]+2*dx, A[1]+2*dy];
1791       const ang = Math.atan2(dy,dx) - Math.PI/3;
1792       const L = Math.hypot(dx,dy);
1793       const P2=[P1[0]+Math.cos(ang)*L, P1[1]+Math.sin(ang)*L];
1794       next.push(A,P1,P2,P3);
1795     }
1796     pts = next;
1797   }
1798   ctx.beginPath(); polyPath(ctx,pts);
1799   fillA(ctx,c3,0.35); iStroke(ctx,c0,sw*0.8);
1800   ctx.beginPath(); circle(ctx,0,0,R*p.ir*0.5);
1801   st(ctx,c2,sw*0.6);
1802   dot(ctx,0,0,R*0.03,c1);
1803 }
1804
1805 /* 30. Sierpinski uchburchagi */

```

```

1806 function drawSierpinski(ctx,R,p,sw,c0,c1,c2,c3){
1807   const depth = Math.max(1, Math.min(6, p.cmp));
1808   const A=pol((-Math.PI/2,R*0.92), B=pol(Math.PI/6,R*0.92), C=pol(Math.PI*5/6,R*0.92));
1809   const mid=(u,v)=>[(u[0]+v[0])/2,(u[1]+v[1])/2];
1810   const rec=(a,b,c,d)=>{
1811     if(d===0){
1812       ctx.beginPath(); polyPath(ctx,[a,b,c]);
1813       fillA(ctx, d%2?c1:c3, 0.7); st(ctx,c0,sw*0.35);
1814       return;
1815     }
1816     rec(a,mid(a,b),mid(a,c),d-1);
1817     rec(mid(a,b),b,mid(b,c),d-1);
1818     rec(mid(a,c),mid(b,c),c,d-1);
1819   };
1820   rec(A,B,C,depth);
1821   ctx.beginPath(); polyPath(ctx,[A,B,C]); st(ctx,c0,sw);
1822 }
1823
1824 /* 31. Dragon curve – L-sistema: F→F+G, G→F-G */
1825 function drawDragon(ctx,R,p,sw,c0,c1,c2,c3){
1826   const iters = 8 + Math.min(5, p.cmp);
1827   let seq = 'F';
1828   for(let d=0; d<iters; d++){
1829     let next='';
1830     for(const ch of seq){
1831       if(ch==='F') next += 'F+G';
1832       else if(ch==='G') next += 'F-G';
1833       else next += ch;
1834     }
1835     seq = next;
1836     if(seq.length > 40000) break;
1837   }
1838   let x=0,y=0,ang=0;
1839   const pts=[[0,0]];
1840   let minX=0,maxX=0,minY=0,maxY=0;
1841   for(const ch of seq){
1842     if(ch==='F' || ch==='G'){
1843       x += Math.cos(ang); y += Math.sin(ang);
1844       pts.push([x,y]);
1845       if(x<minX)minX=x; if(x>maxX)maxX=x;
1846       if(y<minY)minY=y; if(y>maxY)maxY=y;
1847     }
1848     else if(ch==='+') ang += Math.PI/2;
1849     else if(ch==='-') ang -= Math.PI/2;
1850   }
1851   const w=maxX-minX||1, h=maxY-minY||1;
1852   const sc = R*1.7/Math.max(w,h);
1853   const ox=(minX+maxX)/2, oy=(minY+maxY)/2;
1854   ctx.beginPath();
1855   ctx.moveTo((pts[0][0]-ox)*sc, (pts[0][1]-oy)*sc);
1856   for(let i=1;i<pts.length;i++) ctx.lineTo((pts[i][0]-ox)*sc, (pts[i][1]-oy)*sc);
1857   st(ctx, c0, sw*0.6);
1858   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
1859 }
1860
1861 /* 32. Hilbert space-filling curve */
1862 function drawHilbert(ctx,R,p,sw,c0,c1,c2,c3){
1863   const order = Math.max(2, Math.min(6, 1 + Math.round(p.cmp*0.8)));
1864   const N = 1<<order;
1865   const d2xy = (d)=>{
1866     let rx, ry, t=d, x=0, y=0;
1867     for(let s=1; s<N; s*=2){
1868       rx = 1 & (t/2); ry = 1 & (t ^ rx);
1869       if(ry===0){
1870         if(rx===1){ x=s-1-x; y=s-1-y; }
1871         const tmp=x; x=y; y=tmp;
1872       }
1873       x += s*rx; y += s*ry;
1874       t = Math.floor(t/4);
1875     }
1876     return [x,y];
1877   };
1878   const total = N*N;
1879   const sc = R*1.7/(N-1);
1880   ctx.beginPath();
1881   for(let d=0; d<total; d++){
1882     const q = d2xy(d);
1883     const px = (q[0]-(N-1)/2)*sc, py=(q[1]-(N-1)/2)*sc;
1884     if(d===0) ctx.moveTo(px,py); else ctx.lineTo(px,py);
1885   }
1886   st(ctx, c0, sw*0.55);
1887   ctx.beginPath();
1888   polyPath(ctx,[[ -R*0.92, -R*0.92],[R*0.92, -R*0.92],[R*0.92,R*0.92],[ -R*0.92,R*0.92]]);
1889   st(ctx,c2,sw*0.6);
1890 }
1891
1892 /* 33. Spirograph – epitrochoid */
1893 function drawSpirograph(ctx,R,p,sw,c0,c1,c2,c3){
1894   const R0 = R*0.62;
1895   /* Gipotroxoida: r = R0·a/b (a va b o'zaro tub) → egri b ta cho'qqili
1896     bo'lib, a ta aylanishdan keyin yopiladi.

```

```

1897     DIQQAT – DEGENERATSIYA:  $b = 2a$  bo'lganda  $(R0-r)/r = 1$  va egri
1898     ELLIPSGA aylanadi;  $d \approx r$  bo'lsa esa TO'G'RI CHIZIQA siqiladi.
1899     Eski formulada  $kNum = sym-2$ ,  $kDen = sym+cmp$  edi va standart
1900     qiymatlarda ( $sym = 8$ ,  $cmp = 4$ ) aynan  $kDen = 2 \cdot kNum$  chiqib qolardi –
1901     shuning uchun spirograf o'rniga to'g'ri chiziqlar chizilardi.
1902     Endi bu holat aniq chetlab o'tiladi. */
1903     const gcd = (x, y) => { x = Math.abs(x); y = Math.abs(y); while(y) { const t = y; y = x % y; x = t; } return x || 1
; };
1904     let b = Math.max(3, p.sym | 0);
1905     let a = 1 + ((p.cmp | 0) % Math.max(1, b - 1));
1906     const g0 = gcd(a, b); a = a/g0; b = b/g0;
1907     if(b === 2*a) b += 1; /* ellips/chiziqa aylanmasin */
1908     if(a >= b) a = 1;
1909     const r = R0*a/b;
1910     const d = r*(0.45 + p.spir*1.1);
1911     const loops = a;
1912     const steps = Math.max(480, 180*b);
1913     const pts=[];
1914     let maxR=0;
1915     for(let i=0; i<=steps; i++){
1916         const t = TAU*loops*i/steps;
1917         const x=(R0-r)*Math.cos(t) + d*Math.cos((R0-r)*t/r);
1918         const y=(R0-r)*Math.sin(t) - d*Math.sin((R0-r)*t/r);
1919         pts.push([x,y]);
1920         const rr=Math.hypot(x,y); if(rr>maxR) maxR=rr;
1921     }
1922     const sc = R*0.95/(maxR||1);
1923     ctx.beginPath();
1924     smoothPath(ctx, pts.map(q=>[q[0]*sc, q[1]*sc]), false);
1925     st(ctx, c0, sw*0.55);
1926     ctx.beginPath(); circle(ctx, 0, 0, R*0.98); st(ctx, c2, sw*0.5);
1927     dot(ctx, 0, 0, R*0.03, c1);
1928 }
1929
1930 /* 34. Lissajous */
1931 function drawLissajous(ctx, R, p, sw, c0, c1, c2, c3){
1932     const a = Math.max(1, p.sym-3), b = a + Math.max(1, Math.round(p.cmp/2));
1933     const del = Math.PI/2 * (1 + p.skew);
1934     const steps = 400;
1935     const pts=[];
1936     for(let i=0; i<=steps; i++){
1937         const t = TAU*i/steps;
1938         pts.push([R*0.88*Math.sin(a*t+del), R*0.88*Math.sin(b*t)]);
1939     }
1940     ctx.beginPath(); smoothPath(ctx, pts, true);
1941     st(ctx, c0, sw*0.6);
1942     if(p.dens>0.5){
1943         const pts2 = pts.map(q=>[q[0]*0.7, q[1]*0.7]);
1944         ctx.beginPath(); smoothPath(ctx, pts2, true);
1945         st(ctx, c1, sw*0.45);
1946     }
1947     ctx.beginPath();
1948     polyPath(ctx, [[-R*0.95, -R*0.95], [R*0.95, -R*0.95], [R*0.95, R*0.95], [-R*0.95, R*0.95]]);
1949     st(ctx, c2, sw*0.4);
1950 }
1951
1952 /* 35. Voronoi – seed nuqtalardan hujayra mozaika */
1953 function mulberry32(seed){
1954     let t = seed>>>0;
1955     return function(){
1956         t += 0x6D2B79F5;
1957         let x = Math.imul(t ^ (t>>>15), 1 | t);
1958         x ^= x + Math.imul(x ^ (x>>>7), 61 | x);
1959         return ((x ^ (x>>>14)) >>> 0) / 4294967296;
1960     };
1961 }
1962 function clipHalfPlane(poly, px, py, nx, ny){
1963     /* n·(q - p) <= 0 tomonini saqlaydi */
1964     const out=[];
1965     for(let i=0; i<poly.length; i++){
1966         const A=poly[i], B=poly[(i+1)%poly.length];
1967         const da=(A[0]-px)*nx + (A[1]-py)*ny;
1968         const db=(B[0]-px)*nx + (B[1]-py)*ny;
1969         if(da<=0) out.push(A);
1970         if((da<0 && db>0) || (da>0 && db<0)){
1971             const t = da/(da-db);
1972             out.push([A[0]+(B[0]-A[0])*t, A[1]+(B[1]-A[1])*t]);
1973         }
1974     }
1975     return out;
1976 }
1977 function drawVoronoi(ctx, R, p, sw, c0, c1, c2, c3){
1978     const palc = PAL[p.pal] ? PAL[p.pal].c : [c0, c1, c2, c3, c0];
1979     const rnd = mulberry32(1000 + p.sym*31 + p.cmp*7 + Math.round(p.dens*100)*13 + Math.round(p.spir*100)*3);
1980     const m = 8 + p.cmp*4 + Math.round(p.dens*14);
1981     const seeds=[];
1982     for(let i=0; i<m; i++){
1983         const a = rnd()*TAU, rr = Math.sqrt(rnd())*R*0.92;
1984         seeds.push(pol(a, rr));
1985     }
1986     /* boshlang'ich poligon – R ga chizilgan 16-burchak */

```

```

1987 const base=[]; for(let i=0;i<16;i++) base.push(pol(TAU*i/16, R*0.98));
1988 for(let i=0;i<m;i++){
1989   let cell = base.slice();
1990   const S = seeds[i];
1991   for(let j=0;j<m && cell.length;j++){
1992     if(j===i) continue;
1993     const T = seeds[j];
1994     const mx=(S[0]+T[0])/2, my=(S[1]+T[1])/2;
1995     const nx=T[0]-S[0], ny=T[1]-S[1];
1996     cell = clipHalfPlane(cell, mx,my, nx,ny);
1997   }
1998   if(cell.length<3) continue;
1999   ctx.beginPath(); polyPath(ctx,cell);
2000   fillA(ctx, palc[i%5], 0.75);
2001   st(ctx, c0, sw*0.55);
2002   if(p.cmp>=5) dot(ctx, S[0], S[1], sw*0.9, c2);
2003 }
2004 ctx.beginPath(); circle(ctx,0,0,R*0.99); st(ctx,c0,sw*0.9);
2005 }
2006
2007 /* =====
2008 NaqshAI — PIC GEOMETRIYA YADROSI (Polygons-In-Contact / Hankin usuli)
2009 -----
2010 Klassik g'irih qurilishi. Bosqichlar:
2011 1. PLITKALASH — davriy (yoki radial) plitka to'ri, qat'iy qirra-qirraga
2012 2. KONTAKT — har qirra o'rtasidan 6 burchagi ostida ikki nur
2013 3. GLOBAL GRAF — barcha segmentlar bitta grafda, dedup + tugun birlashtirish
2014 4. IP (STRAND) — kesishishlarda davomiylik bo'yicha uzluksiz ip ajratish
2015 5. TO'QISH — har ip bo'ylab ust/ost navbatlashishi
2016 Chiqishi sof ma'lumot: {tiles, strands, crossings, cores, period}
2017 Chizish bu ma'lumotdan keyin, alohida bosqichda (picRender).
2018 ===== */
2019 var TAU2 = Math.PI * 2;
2020 var SQ3 = Math.sqrt(3);
2021
2022 /* ----- kichik vektor yordamchilari ----- */
2023 function vAdd(a,b){ return [a[0]+b[0], a[1]+b[1]]; }
2024 function vSub(a,b){ return [a[0]-b[0], a[1]-b[1]]; }
2025 function vMul(a,k){ return [a[0]*k, a[1]*k]; }
2026 function vLen(a){ return Math.hypot(a[0],a[1]); }
2027 function vNorm(a){ var l=vLen(a)||1; return [a[0]/l, a[1]/l]; }
2028 function vDot(a,b){ return a[0]*b[0]+a[1]*b[1]; }
2029 function polyCentroid(p){
2030   var x=0,y=0;
2031   for(var i=0;i<p.length;i++){ x+=p[i][0]; y+=p[i][1]; }
2032   return [x/p.length, y/p.length];
2033 }
2034 function ngon(cx,cy,r,n,rot){
2035   var q=[];
2036   for(var i=0;i<n;i++){ var a=rot+TAU2*i/n; q.push([cx+Math.cos(a)*r, cy+Math.sin(a)*r]); }
2037   return q;
2038 }
2039 /* Ikki nur kesishmasi (nuqta + yo'nalish vektori) */
2040 function rayCross(p,d,q,e){
2041   var den = d[0]*e[1] - d[1]*e[0];
2042   if(Math.abs(den) < 1e-12) return null;
2043   var t = ((q[0]-p[0])*e[1] - (q[1]-p[1])*e[0]) / den;
2044   return [p[0]+d[0]*t, p[1]+d[1]*t];
2045 }
2046 /* bbox ni qoplaydigan panjara nuqtalari: P = i.v1 + j.v2 */
2047 function latticePoints(v1,v2,bb,margin){
2048   var det = v1[0]*v2[1] - v1[1]*v2[0];
2049   if(Math.abs(det) < 1e-9) return [];
2050   var m = margin||0;
2051   var x0=bb.x0-m, y0=bb.y0-m, x1=bb.x1+m, y1=bb.y1+m;
2052   var corners=[[x0,y0],[x1,y0],[x1,y1],[x0,y1]];
2053   var iMin=Infinity, iMax=-Infinity, jMin=Infinity, jMax=-Infinity;
2054   for(var k=0;k<4;k++){
2055     var c=corners[k];
2056     var i=( c[0]*v2[1] - c[1]*v2[0])/det;
2057     var j=(-c[0]*v1[1] + c[1]*v1[0])/det;
2058     iMin=Math.min(iMin,i); iMax=Math.max(iMax,i);
2059     jMin=Math.min(jMin,j); jMax=Math.max(jMax,j);
2060   }
2061   var out=[];
2062   for(var jj=Math.floor(jMin)-1; jj<=Math.ceil(jMax)+1; jj++)
2063     for(var ii=Math.floor(iMin)-1; ii<=Math.ceil(iMax)+1; ii++)
2064       out.push({i:ii, j:jj, p:[ii*v1[0]+jj*v2[0], ii*v1[1]+jj*v2[1]]});
2065   return out;
2066 }
2067 function inBB(p,bb,pad){
2068   var q=pad||0;
2069   return p[0]>=bb.x0-q && p[0]<=bb.x1+q && p[1]>=bb.y0-q && p[1]<=bb.y1+q;
2070 }
2071
2072 /* =====
2073 1) PLITKALASHLAR
2074 Har biri {tiles:[{poly,kind}], period:[w,h]|null, unit:<o'lcham>}
2075 qaytaradi. `kind` — rang guruhi (0 = asosiy, 1 = ikkilamchi).
2076 Barchasi QIRRA-QIRRAGA: har ichki qirra aynan ikki plitkaga tegishli,
2077 shu sabab tugunlar bir-biriga aniq tushadi va iplar uzilmaydi.

```

```

2078  */
2079  var PIC_TILINGS = {
2080
2081      /* 4.4.4.4 – kvadrat to'r (6=45° klassik sakkizburchak-xoch beradi) */
2082      sq: {
2083          nm: "Kvadrat to'r (4.4.4.4)", deltas:[30,45,60,22.5], sides:4,
2084          build: function(s, bb){
2085              var tiles=[], pts=latticePoints([s,0],[0,s], bb, s*1.2);
2086              for(var k=0;k<pts.length;k++){
2087                  var x=pts[k].p[0], y=pts[k].p[1];
2088                  tiles.push({poly:[x,y],[x+s,y],[x+s,y+s],[x,y+s]], kind:(pts[k].i+pts[k].j)&1});
2089              }
2090              return {tiles:tiles, period:[s,s], unit:s};
2091          }
2092      },
2093
2094      /* 4.8.8 – sakkizburchak + kvadrat: Registon panellarining asosi */
2095      t488: {
2096          nm: "Sakkizburchak + kvadrat (4.8.8)", deltas:[45,67.5,22.5], sides:8,
2097          build: function(s, bb){
2098              var oR = (s*0.5)/Math.cos(Math.PI/8);          /* sakkizburchak tashqi radiusi */
2099              var a = 2*oR*Math.sin(Math.PI/8);            /* qirra uzunligi */
2100              var qR = a/Math.SQRT2;                        /* kvadrat tashqi radiusi (45° burilgan) */
2101              var tiles=[], pts=latticePoints([s,0],[0,s], bb, s*1.6);
2102              for(var k=0;k<pts.length;k++){
2103                  var x=pts[k].p[0], y=pts[k].p[1];
2104                  tiles.push({poly:ngon(x,y,oR,8,Math.PI/8), kind:0});
2105                  tiles.push({poly:ngon(x+s/2,y+s/2,qR,4,-Math.PI/2), kind:1});
2106              }
2107              return {tiles:tiles, period:[s,s], unit:s};
2108          }
2109      },
2110
2111      /* 6.6.6 – asal uyasi (p6m) */
2112      t666: {
2113          nm: "Olti burchak to'ri (6.6.6)", deltas:[30,60,45,15], sides:6,
2114          build: function(s, bb){
2115              var v1=[SQ3*s,0], v2=[SQ3*s/2, 1.5*s];
2116              var tiles=[], pts=latticePoints(v1,v2, bb, s*2);
2117              for(var k=0;k<pts.length;k++){
2118                  var c=pts[k].p;
2119                  tiles.push({poly:ngon(c[0],c[1],s,6,Math.PI/6), kind:((pts[k].i+2*pts[k].j)%3+3)%3===0?0:1});
2120              }
2121              return {tiles:tiles, period:[SQ3*s, 3*s], unit:s};
2122          }
2123      },
2124
2125      /* 3.6.3.6 – olti burchak + uchburchak (trigeksagonal) */
2126      t3636: {
2127          nm: "Olti burchak + uchburchak (3.6.3.6)", deltas:[45,30,60], sides:6,
2128          build: function(a, bb){
2129              var d=2*a, v1=[d,0], v2=[d/2, d*SQ3/2];
2130              var tiles=[], pts=latticePoints(v1,v2, bb, d*2);
2131              var mid=function(A,B){ return [(A[0]+B[0])/2,(A[1]+B[1])/2] };
2132              for(var k=0;k<pts.length;k++){
2133                  var P=pts[k].p;
2134                  tiles.push({poly:ngon(P[0],P[1],a,6,0), kind:0});
2135                  /* bo'shliq uchburchaklari = panjara uchburchagining qirra o'rtalari */
2136                  var A=P, B=vAdd(P,v1), C=vAdd(P,v2), D=vAdd(B,v2);
2137                  tiles.push({poly:[mid(A,B),mid(B,C),mid(A,C)], kind:1});
2138                  tiles.push({poly:[mid(B,D),mid(C,D),mid(B,C)], kind:1});
2139              }
2140              return {tiles:tiles, period:[d, d*SQ3], unit:a};
2141          }
2142      },
2143
2144      /* 3.12.12 – o'n ikki burchak + uchburchak: Temuriylar 12-nurli yulduzi */
2145      t3122: {
2146          nm: "O'n ikki burchak (3.12.12)", deltas:[45,75,30,60,15], sides:12,
2147          build: function(a, bb){
2148              var R12 = a/(2*Math.sin(Math.PI/12));
2149              var r12 = a/(2*Math.tan(Math.PI/12));
2150              var d = 2*r12, v1=[d,0], v2=[d/2, d*SQ3/2];
2151              var tR = a/SQ3;          /* uchburchak tashqi radiusi */
2152              var tiles=[], pts=latticePoints(v1,v2, bb, d*1.6);
2153              for(var k=0;k<pts.length;k++){
2154                  var P=pts[k].p;
2155                  tiles.push({poly:ngon(P[0],P[1],R12,12,Math.PI/12), kind:0});
2156                  var Gu=vAdd(P, vMul(vAdd(v1,v2), 1/3));
2157                  var Gd=vAdd(P, vMul(vAdd(v1,v2), 2/3));
2158                  tiles.push({poly:ngon(Gu[0],Gu[1],tR,3,-Math.PI/2), kind:1});
2159                  tiles.push({poly:ngon(Gd[0],Gd[1],tR,3, Math.PI/2), kind:1});
2160              }
2161              return {tiles:tiles, period:[d, d*SQ3], unit:a};
2162          }
2163      },
2164
2165      /* SHAMSA-10 – HAQIQIY GIRIH TILE TO'PLAMI (o'n burchakli medalyon).
2166      Markazda o'n burchak (ichki burchagi 144°), uning har qirrasiga tashqi
2167      tomondan muntazam beshburchak (108°) ulanadi. O'n burchak uchida
2168      burchaklar yig'indisi 144° + 108° + 108° = 360° – ya'ni beshburchaklar

```

```

2169     halqasi BO'SHLIQSIZ yopiladi. Bu Registon va Shohi Zinda panellaridagi
2170     klassik shamsa qurilishi; barcha qirralar teng (a), barcha burchaklar
2171     36° ga karrali. Aynan shu sabab PIC lentalarini butun medalyon bo'ylab
2172     uzluksiz oqadi va 10 nurli yulduz markazda aniq yopiladi. */
2173     shamsa10: {
2174         nm: "Shamsa — o'n burchak + beshburchak", deltas:[72,54,63,36], sides:10, radial:true,
2175         build: function(unit, bb, opt){
2176             opt = opt || {};
2177             var R = opt.R || unit*4;
2178             /* R — medalyonning tashqi radiusi. Beshburchak halqasining tashqi
2179             chegarasi markazdan rOut = rd + 2·rp masofada (rd — o'n burchak
2180             ichki radiusi, rp — beshburchak ichki radiusi) → a ni shundan topamiz. */
2181             var rdK = 1/(2*Math.tan(Math.PI/10)); /* o'n burchak ichki radiusi / a */
2182             var rpK = 1/(2*Math.tan(Math.PI/5)); /* beshburchak ichki radiusi / a */
2183             var RpK = 1/(2*Math.sin(Math.PI/5)); /* beshburchak tashqi radiusi / a */
2184             var a = R/(rdK + rpK + RpK);
2185             var rd = rdK*a, rp = rpK*a, Rp = RpK*a;
2186             var Rd = a/(2*Math.sin(Math.PI/10));
2187             var tiles=[], i, j;
2188             /* markaziy o'n burchak: uchlari 18°+36k, qirra o'rtalari 36k */
2189             tiles.push({poly: ngon(0,0,Rd,10,Math.PI/10), kind:0});
2190             for(i=0;i<10;i++){
2191                 var th = TAU2*i/10; /* qirra normalining yo'nalishi */
2192                 var C = [Math.cos(th)*(rd+rp), Math.sin(th)*(rd+rp)];
2193                 /* beshburchakning ichki qirrasini o'rtasi C dan (th+180°) tomonda →
2194                 uchlari th + 180° + 36° + 72°j burchaklarda */
2195                 var poly=[];
2196                 for(j=0;j<5;j++){
2197                     var aa = th + Math.PI + Math.PI/5 + TAU2*j/5;
2198                     poly.push([C[0]+Math.cos(aa)*Rp, C[1]+Math.sin(aa)*Rp]);
2199                 }
2200                 tiles.push({poly:poly, kind:1});
2201             }
2202             return {tiles:tiles, period:null, unit:a, radius:rd+rp+Rp, radial:true};
2203         }
2204     }
2205 };
2206
2207 /* =====
2208 2) KONTAKT NURLARI + 3) GLOBAL GRAF (dedup, tugun birlashtirish)
2209 =====
2210 Har plitkani har qirrasini o'rtasidan ikki nur chiqadi:
2211     d1 = +cosδ·u + sinδ·nIn     d2 = -cosδ·u + sinδ·nIn
2212     (u — qirra yo'nalishi, nIn — plitka ichiga qaragan normal)
2213     Qo'shni plitkada u va nIn ikkisi ham teskari bo'ladi, shuning uchun
2214     uning nurlari -d1, -d2 ga teng → qirra o'rtasida chiziqlar TO'G'RI
2215     davom etadi. Aynan shu xossa "yamoq" effektini yo'q qiladi.
2216 ===== */
2217 function picBuild(tg, deltaDeg, opt){
2218     opt = opt || {};
2219     var tiles = tg.tiles, unit = tg.unit || 1;
2220     var EPS = unit * 1e-3; /* tugun birlashtirish toleransi */
2221     var delta = deltaDeg * Math.PI/180;
2222     var cs = Math.cos(delta), sn = Math.sin(delta);
2223
2224     var nodes=[], nodeMap={}, edges=[], edgeSet={};
2225     /* Tugun turi muhim:
2226     mid — plitka qirrasining o'rtasi (ichkarida 4 darajali KESISHISH,
2227     patch chegarasida 2 darajali TUGATUVCHI)
2228     x — plitka ichidagi uchrashuv nuqtasi (lentaning haqiqiy BURCHAGI) */
2229     function nodeAt(p, isMid){
2230         var kx = Math.round(p[0]/EPS), ky = Math.round(p[1]/EPS);
2231         var key = kx+'|'+ky;
2232         var id = nodeMap[key];
2233         if(id === undefined){
2234             /* qo'shni kataklarni ham tekshiramiz — yaxlitlash chegarasida bo'linib
2235             ketmasligi uchun (aks holda bir tugun ikkiga ajralib, ip uziladi) */
2236             for(var dx=-1; dx<=1 && id===undefined; dx++){
2237                 for(var dy=-1; dy<=1 && id===undefined; dy++){
2238                     var alt = nodeMap[(kx+dx)+'|'+(ky+dy)];
2239                     if(alt !== undefined && Math.hypot(nodes[alt].p[0]-p[0], nodes[alt].p[1]-p[1]) < EPS) id = alt;
2240                 }
2241             }
2242             if(id === undefined){ id = nodes.length; nodes.push({p:p, e:[], mid:false}); }
2243             nodeMap[key] = id;
2244         }
2245         if(isMid) nodes[id].mid = true;
2246         return id;
2247     }
2248     function addEdge(a,b){
2249         if(a===b) return;
2250         var key = a<b ? a+'-'+b : b+'-'+a;
2251         if(edgeSet[key] !== undefined) return; /* DEDUP: umumiy qirradagi takror segment */
2252         edgeSet[key] = edges.length;
2253         edges.push({a:a, b:b});
2254         nodes[a].e.push(edges.length-1);
2255         nodes[b].e.push(edges.length-1);
2256     }
2257
2258     /* Konveks plitka ichidaligini tekshirish (yarim tekislik testi) */
2259     function insideConvex(X, poly, cen, tol){
2260         for(var i=0;i<poly.length;i++){

```

* * *

Listingning 26–77 betlari (52 bet) tushirib qoldirildi.

So'rovnoma muvofiq boshlang'ich matnning fragmentlari taqdim etilmoqda:
birinchi 25 bet va oxirgi 25 bet.

To'liq listing (102 bet) talab qilinsa, darhol taqdim etiladi.

```

6927   ctx.restore();
6928 }
6929 function arLoop(){
6930   if(!arActive) return;
6931   const overlay = $('ar-overlay');
6932   const stage = $('ar-stage');
6933   if(overlay && stage){
6934     if(overlay.width !== stage.clientWidth || overlay.height !== stage.clientHeight){
6935       overlay.width = stage.clientWidth||300;
6936       overlay.height = stage.clientHeight||300;
6937     }
6938     const ctx = overlay.getContext('2d');
6939     if(ctx){
6940       if(arNeedsRedraw || !arPatCanvas) renderArPattern();
6941       ctx.clearRect(0,0,overlay.width,overlay.height);
6942       if(AR.simMode) drawArBackdrop(ctx, overlay.width, overlay.height);
6943       drawArSurface(ctx, overlay.width, overlay.height);
6944     }
6945   }
6946   requestAnimationFrame(arLoop);
6947 }
6948 function startAR(){
6949   const errBox = $('ar-err');
6950   if(errBox) errBox.textContent = '';
6951   if(!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia){
6952     /* Brauzer kamerani faqat XAVFSIZ MUHITDA beradi (https yoki localhost).
6953     Oddiy http:// da navigator.mediaDevices umuman mavjud bo'lmaydi va
6954     ilgari bu "brauzeringiz qo'llab-quvvatlamaydi" deb ko'rsatilardi -
6955     sabab noto'g'ri aytilgan bo'lardi. */
6956     const secure = (typeof isSecureContext !== 'undefined') ? isSecureContext
6957       : (location.protocol === 'https:' || location.hostname === 'localhost');
6958     const msg = secure
6959       ? 'Brauzeringiz kamerani qo'llab-quvvatlamaydi.'
6960       : 'Kamera faqat HTTPS orqali ishlaydi. Saytni https:// bilan oching '
6961       + '(GitHub Pages allaqachon HTTPS). Hozircha «Simulyatsiya» tugmasidan foydalaning.';
6962     if(errBox) errBox.textContent = msg;
6963     ntf(msg, 'err');
6964     return;
6965   }
6966   navigator.mediaDevices.getUserMedia({video:{facingMode:'environment'}})
6967   .catch(()=>navigator.mediaDevices.getUserMedia({video:true}))
6968   .then(stream=>{
6969     arStream = stream;
6970     const video = $('ar-video');
6971     if(video){
6972       video.srcObject = stream;
6973       video.style.display = ''; /* simulyatsiyadan keyin qaytariladi */
6974       /* autoplay atributi bor, lekin ba'zi brauzerlar aniq play() talab
6975       qiladi - xatosi jimgina yutilmasin */
6976       const pr = video.play();
6977       if(pr && pr.catch) pr.catch(e=>{
6978         console.warn('[ar] video.play', e);
6979         if(errBox) errBox.textContent = 'Videoni boshlab bo'lmadi: ' + e.name;
6980       });
6981     }
6982     arActive = true;
6983     arNeedsRedraw = true;
6984     const card = $('ar-start-card'); if(card) card.style.display = 'none';
6985     const bar = $('ar-bar'); if(bar){ bar.hidden = false; bar.style.display = 'flex'; }
6986     arLoop();
6987     ntf('AR rejim yoqildi ☑', 'ok');
6988   })
6989   .catch(err=>{
6990     let msg = 'Kamera xatosi: ' + err.name;
6991     if(err.name === 'NotAllowedError') msg = 'Kameraga ruxsat berilmadi – brauzer sozlamalaridan ruxsat bering'
+ (location.protocol==='file:' ? ' (agar file:// da bloklangan bo'lsa, HTTPS/GitHub Pages orqali oching)' : '') +
+ '.';
6992     else if(err.name === 'NotFoundError') msg = 'Kamera topilmadi.';
6993     else if(err.name === 'NotReadableError') msg = 'Kamera band – boshqa ilova ishlatayotgan bo'lishi mumkin.';
6994     if(errBox) errBox.textContent = msg;
6995     ntf(msg, 'err');
6996   });
6997 }
6998 function stopAR(){
6999   arActive = false;
7000   AR.simMode = false;
7001   if(arStream){
7002     arStream.getTracks().forEach(tr=>tr.stop()); /* streamni to'xtatish */
7003     arStream = null;
7004   }
7005   const video = $('ar-video');
7006   if(video){ video.srcObject = null; video.style.display = ''; }
7007   const overlay = $('ar-overlay');
7008   if(overlay){
7009     /* keyingi safar boshlanguncha eski kadr ko'rinib qolmasin */
7010     const octx = overlay.getContext('2d');
7011     if(octx) octx.clearRect(0, 0, overlay.width, overlay.height);
7012   }
7013   const card = $('ar-start-card'); if(card) card.style.display = 'flex';
7014   /* .ar-bar{display:flex} qoidasi [hidden] ustidan ustunlik qiladi –
7015   shuning uchun inline style bilan ham aniq yashiramiz */

```

```

7016   const bar = $('ar-bar'); if(bar){ bar.hidden = true; bar.style.display = 'none'; }
7017 }
7018 function captureAR(){
7019   const video = $('ar-video'), overlay = $('ar-overlay');
7020   if(!overlay || !arActive){ ntf('Avval kamerani yoqing yoki simulyatsiyani boshlang', 'err'); return; }
7021   const W = (!AR.simMode && video && video.videoWidth) || overlay.width || 640;
7022   const H = (!AR.simMode && video && video.videoHeight) || overlay.height || 480;
7023   const off = document.createElement('canvas');
7024   off.width = W; off.height = H;
7025   const octx = off.getContext('2d');
7026   if(!octx) return;
7027   if(AR.simMode){
7028     drawArBackdrop(octx, W, H);
7029   } else if(video){
7030     octx.drawImage(video, 0, 0, W, H);
7031   }
7032   /* overlay object-fit:cover ga mos masshtabda ustiga chiziladi */
7033   const sc = Math.max(W/overlay.width, H/overlay.height);
7034   const dw = overlay.width*sc, dh = overlay.height*sc;
7035   octx.drawImage(overlay, (W-dw)/2, (H-dh)/2, dw, dh);
7036   if(off.toBlob){
7037     off.toBlob(b=>{ if(b) _d1Blob(b, 'image/png', 'naqsh_ar.png', 'AR surat saqlandi '); }, 'image/png');
7038   }
7039 }
7040
7041 /* =====
7042   AI CHAT — lokal NLP parser, rasm-natija markazli
7043   ===== */
7044 const CHAT_TYPES = [
7045   ['girihi', 'girihi'], ['xatam', 'xatam'], ['rozetta', 'rozetta'], ['yulduz', 'yulduz'],
7046   ['muqarnas', 'muqarnas'], ['katak', 'katak8'], ['olti burchak', 'hexa'], ['asal', 'hexa'], ['zanjira', 'zanjira'],
7047   ['hankin 6', 'hankin6'], ['hankin olti', 'hankin6'], ['hankin', 'hankin8'], ['sakkiz qirrali', 'panelgirihi'], ['panel', 'panelgirihi'],
7048   ['islamiy', 'islamiy'], ['rumi', 'rumi'], ['hatayi', 'hatayi'], ['palak', 'palak'], ['shamsa', 'shamsa'],
7049   ['arabesk', 'arabesk'], ['arabesque', 'arabesk'], ['novda', 'scroll'], ['scroll', 'scroll'], ['medalyon', 'medallion'], ['medallion', 'medallion'],
7050   ['celtic', 'celtic'], ['kelt', 'celtic'], ['mandala', 'mandala'], ['penrose', 'penrose'], ['zellige', 'zellige'], ['zelij', 'zellige'],
7051   ['batik', 'batik'], ['paisley', 'paisley'], ['buta', 'paisley'], ['greek', 'greek'], ['meander', 'greek'], ['xitoy', 'xitoy'],
7052   ['koch', 'koch'], ['qor parcha', 'koch'], ['sierpinski', 'sierpinski'], ['dragon', 'dragon'], ['ajdar', 'dragon'],
7053   ['hilbert', 'hilbert'], ['spirograf', 'spirograph'], ['spirograph', 'spirograph'], ['lissajous', 'lissajous'], ['lissajous', 'lissajous'], ['lissajou', 'lissajous'],
7054 ];
7055 const CHAT_COLORS = [
7056   ['oltin', 'gold'], ['tilla', 'gold'], ['feruza', 'turquoise'], ['turkuaz', 'turquoise'], ['registon', 'turquoise'],
7057   ['zumrad', 'emerald'], ['yashil', 'emerald'], ['yoqut', 'ruby'], ['qizil', 'ruby'],
7058   ['ko'k', 'navy'], ['kok', 'navy'], ['navy', 'navy'], ['tuproq', 'earth'], ['jigarrang', 'earth']
7059 ];
7060 /* Matn -> barqaror hash (bir xil matn = bir xil naqsh) */
7061 function hashStr(str){
7062   let h = 5381;
7063   for(let i=0; i<str.length; i++){ h = ((h<<5)+h) + str.charCodeAt(i) >>> 0; }
7064   return h >>> 0;
7065 }
7066 function parseChat(txt){
7067   const q = (txt||'').toLowerCase();
7068   const np = {...P}; delete np._bg;
7069   /* 1) HAR QANDAY MATN naqsh tug'diradi: matn hashi -> deterministik parametrlar
7070      (nano-banana uslubi: "salom dunyo" ham, ism ham o'z noyob naqshiga ega) */
7071   const seed = hashStr(q);
7072   const rnd = mulberry32(seed);
7073   const ids = PATTERNS.map(x=>x.id);
7074   const pals = Object.keys(PAL);
7075   np.type = ids[Math.floor(rnd()*ids.length)];
7076   np.pal = pals[Math.floor(rnd()*pals.length)];
7077   np.sym = 5 + Math.floor(rnd()*10);
7078   np.cmp = 2 + Math.floor(rnd()*6);
7079   np.sw = +(1.2 + rnd()*2.6).toFixed(1);
7080   np.ir = +(0.2 + rnd()*0.5).toFixed(2);
7081   np.spir = +(rnd()).toFixed(2);
7082   np.dens = +(0.3 + rnd()*0.6).toFixed(2);
7083   np.lay = 1 + Math.floor(rnd()*3);
7084   np.skew = 0;
7085   np.rot = Math.floor(rnd()*360);
7086   np._seed = seed % 100000;
7087   /* 2) Kalit so'zlar seedni ANIQLASHTIRADI (nomi, rangi, soni aytilsa — shu bo'ladi) */
7088   for(const [kw, id] of CHAT_TYPES){ if(q.indexOf(kw)>=0){ np.type = id; break; } }
7089   /* rang */
7090   for(const [kw, id] of CHAT_COLORS){ if(q.indexOf(kw)>=0){ np.pal = id; break; } }
7091   /* sonlar: "8 simmetriya", "12 ta" */
7092   const m = q.match(/(\d+)\s*(ta|simmetriya|simmetriyal|burchak|qirra)/);
7093   if(m){ np.sym = Math.max(3, Math.min(16, parseInt(m[1], 10))); }
7094   else { const m2 = q.match(/b{1,2}\b/); if(m2){ const v=parseInt(m2[1], 10); if(v>=3 && v<=16) np.sym = v; } }
7095 }
7096 /* sifatlar */
7097 if(q.indexOf('zich')>=0) np.dens = 0.8;
7098 if(q.indexOf('siyrak')>=0) np.dens = 0.3;
7099 if(q.indexOf('murakkab')>=0) np.cmp = 6;
7100 if(q.indexOf('sodda')>=0) np.cmp = 2;
7101 if(q.indexOf('ingichka')>=0) np.sw = 1;

```

```

7101   if(q.indexOf('qalin')>=0)      np.sw   = 4;
7102   if(q.indexOf('spiral')>=0)     np.spir = 0.8;
7103   if(q.indexOf('qatlam')>=0)     np.lay   = 4;
7104   if(q.indexOf('tessell')>=0 || q.indexOf('tiling')>=0 || q.indexOf('takror')>=0) np._tess = true;
7105   if(q.indexOf('p4m')>=0) np._grid='p4m'; else if(q.indexOf('p6m')>=0) np._grid='p6m';
7106   else if(q.indexOf('p6')>=0) np._grid='p6'; else if(q.indexOf('p4')>=0) np._grid='p4';
7107   return np;
7108 }
7109 function chatCard(np, query){
7110   const log = $('chat-log');
7111   if(!log) return;
7112   const card = document.createElement('div');
7113   card.className = 'chat-row';
7114   const inner = document.createElement('div');
7115   inner.className = 'ai-card';
7116   const cnv = document.createElement('canvas');
7117   cnv.width = cnv.height = 600; /* 2x – CorelDraw kabi tiniq preview */
7118   inner.appendChild(cnv);
7119   const octx = cnv.getContext('2d');
7120   if(octx){
7121     const pal = PAL[np.pal] || PAL.gold;
7122     const keepBg = P._bg;
7123     P._bg = pal.bg;
7124     octx.fillStyle = pal.bg;
7125     octx.fillRect(0,0,600,600);
7126     if(np._tess){
7127       const keepG = TESS.group, keepC = TESS.cell, keepS = TESS.showCell;
7128       TESS.group = np._grid || TESS.group;
7129       TESS.cell = 200; TESS.showCell = false;
7130       drawTessellation(octx, 600, 600, np);
7131       TESS.group = keepG; TESS.cell = keepC; TESS.showCell = keepS;
7132     } else {
7133       drawPattern(octx, 300, 300, 276, np);
7134     }
7135     P._bg = keepBg;
7136   }
7137   const meta = document.createElement('div');
7138   meta.className = 'meta';
7139   const pt = getPattern(np.type), pal2 = PAL[np.pal] || PAL.gold;
7140   meta.innerHTML = 'Turi: <b>'+pt.nm+'</b> · Simmetriya: <b>'+np.sym+'</b> · Palitra: <b>'+pal2.nm+'</b> · Murakka
blik: <b>'+np.cmp+'</b>' + (np._tess ? ' · <b>Tessellatsiya '+ (np._grid || TESS.group) + '</b>' : '') + (np._seed !== un
defined ? ' · Seed <b>'+np._seed+'</b>' : '');
7141   inner.appendChild(meta);
7142   const acts = document.createElement('div');
7143   acts.className = 'acts';
7144   const bOpen = document.createElement('button');
7145   bOpen.className = 'btn sm'; bOpen.textContent = 'Studio da ochish';
7146   bOpen.addEventListener('click', ()=>{
7147     const clean = {...np}; delete clean._tess; delete clean._grid; delete clean._seed;
7148     Object.assign(P, clean);
7149     if(np._tess){
7150       TESS.on = true;
7151       if(np._grid) TESS.group = np._grid;
7152       const chk = $('chk-tess'); if(chk) chk.checked = true;
7153       const sel = $('sel-tess'); if(sel) sel.value = TESS.group;
7154     }
7155     syncSliders(); redraw();
7156     showPage('studio');
7157     ntf('Naqsh Studio ga yuklandi', 'ok');
7158   });
7159   const bVar = document.createElement('button');
7160   bVar.className = 'btn sm ghost'; bVar.textContent = 'Yana variant';
7161   bVar.addEventListener('click', ()=>{
7162     const v = {...np};
7163     v.spir = +(Math.random()).toFixed(2);
7164     v.dens = +(0.2+Math.random()*0.7).toFixed(2);
7165     v.ir   = +(0.15+Math.random()*0.6).toFixed(2);
7166     v.cmp  = 1+Math.floor(Math.random()*8);
7167     v.rot  = Math.floor(Math.random()*360);
7168     v.lay  = 1+Math.floor(Math.random()*4);
7169     chatCard(v, query);
7170     saveChatHistory();
7171   });
7172   const bDL = document.createElement('button');
7173   bDL.className = 'btn sm gold'; bDL.textContent = 'Yuklab olish';
7174   bDL.addEventListener('click', ()=>{
7175     if(cnv.toBlob) cnv.toBlob(b=>{ if(b) _dlBlob(b,'image/png','naqsh_chat_'+np.type+'.png', 'Rasm yuklab olindi
'); }, 'image/png');
7176   });
7177   acts.appendChild(bOpen); acts.appendChild(bVar); acts.appendChild(bDL);
7178   inner.appendChild(acts);
7179   card.appendChild(inner);
7180   card._np = np;
7181   log.appendChild(card);
7182   log.scrollTop = log.scrollHeight;
7183 }
7184 function chatUserMsg(txt){
7185   const log = $('chat-log');
7186   if(!log) return;
7187   const row = document.createElement('div');
7188   row.className = 'chat-row user';

```

```

7189     const b = document.createElement('div');
7190     b.className = 'bubble-user';
7191     b.textContent = txt;
7192     row.appendChild(b);
7193     row._q = txt;
7194     log.appendChild(row);
7195     log.scrollTop = log.scrollHeight;
7196   }
7197   function saveChatHistory(){
7198     const log = $('chat-log');
7199     if(!log) return;
7200     const hist = [];
7201     Array.prototype.forEach.call(log.children, ch=>{
7202       if(ch._q) hist.push({q:ch._q});
7203       else if(ch._np) hist.push({p:ch._np});
7204     });
7205     lsSet('naqsh_chat', hist.slice(-40));
7206   }
7207   function restoreChatHistory(){
7208     const hist = lsGet('naqsh_chat', []);
7209     hist.forEach(item=>{
7210       if(item.q) chatUserMsg(item.q);
7211       else if(item.p) chatCard(item.p, '');
7212     });
7213   }
7214   /* =====
7215     AI ERKIN TASVIR – Nano Banana (Gemini) orqali, Cloudflare Worker
7216     proksisi bilan. API kalit hech qachon shu faylda saqlanmaydi –
7217     faqat foydalanuvchi o'zi joylashtirgan Worker manzili + token
7218     localStorage'da turadi.
7219     ===== */
7220   let CHAT_MODE = 'exact'; /* 'exact' | 'ai' */
7221   const AI_MODEL_DEFAULT = 'gemini-2.5-flash-image';
7222   const lsGetStr = (k, d) => { try{ return (localStorage.getItem(k) || d || '').trim(); }catch(e){ return d || ''; }
7223   };
7224   const lsSetStr = (k, v) => { try{ localStorage.setItem(k, (v||'').trim()); return true; }catch(e){ return false; }
7225   };
7226   function getAiConn(){ return lsGetStr('naqsh_ai_conn', 'direct') === 'worker' ? 'worker' : 'direct'; }
7227   function setAiConn(v){ return lsSetStr('naqsh_ai_conn', v === 'worker' ? 'worker' : 'direct'); }
7228   function getAiWorkerUrl(){ return lsGetStr('naqsh_ai_worker', ''); }
7229   function setAiWorkerUrl(v){ return lsSetStr('naqsh_ai_worker', v); }
7230   function getAiAppToken(){ return lsGetStr('naqsh_ai_token', ''); }
7231   function setAiAppToken(v){ return lsSetStr('naqsh_ai_token', v); }
7232   function getAiKey(){ return lsGetStr('naqsh_ai_key', ''); }
7233   function setAiKey(v){ return lsSetStr('naqsh_ai_key', v); }
7234   function getAiModel(){ return lsGetStr('naqsh_ai_model', AI_MODEL_DEFAULT) || AI_MODEL_DEFAULT; }
7235   function setAiModel(v){ return lsSetStr('naqsh_ai_model', v); }
7236   /* AI rasm rejimi ishlashga tayyormi? */
7237   function aiReady(){
7238     return getAiConn() === 'worker' ? !!getAiWorkerUrl() : !!getAiKey();
7239   }
7240   function aiShowConn(which){
7241     const w = (which === 'worker');
7242     const seg = $('ai-conn-seg');
7243     if(seg) seg.querySelectorAll('button').forEach(b=> b.classList.toggle('sel', (b.dataset.v === (w?'worker':'direc
7244     t'))));
7245     const d = $('ai-direct-box'), wb = $('ai-worker-box');
7246     if(d) d.hidden = w;
7247     if(wb) wb.hidden = !w;
7248   }
7249   function openAiSetup(){
7250     const u=$('ai-worker-url'), t=$('ai-app-token'), k=$('ai-key'), m=$('ai-model');
7251     if(u) u.value = getAiWorkerUrl();
7252     if(t) t.value = getAiAppToken();
7253     if(k) k.value = getAiKey();
7254     if(m) m.value = getAiModel();
7255     aiShowConn(getAiConn());
7256     const md=$('modal-ai-setup'); if(md) md.classList.add('open');
7257   }
7258   function closeAiSetup(){ const m=$('modal-ai-setup'); if(m) m.classList.remove('open'); }
7259   function aiLoadingCard(){
7260     const log = $('chat-log');
7261     if(!log) return null;
7262     const card = document.createElement('div');
7263     card.className = 'ai-card ai-loading';
7264     card.innerHTML = '<div class="ai-spin"></div><div> Nano Banana tasvirni chizyapti... (2\u2011soniya)</div>';
7265     log.appendChild(card);
7266     log.scrollTop = log.scrollHeight;
7267     return card;
7268   }
7269   /* Gemini javobidan birinchi rasmni ajratib olish.
7270     Worker ham, to'g'ridan-to'g'ri Google ham shu funksiyaga tushadi:
7271     Worker {image,mime} qaytaradi, Google esa candidates[].content.parts[]
7272     ichida inlineData beradi. Ikkalasini ham qo'llab-quvvatlaymiz. */
7273   function aiExtractImage(data){
7274     if(!data) return null;
7275     if(data.image) return { b64: data.image, mime: data.mime || 'image/png' };
7276     const cand = data.candidates;

```

```

7277     if(Array.isArray(cands)){
7278         for(const c of cands){
7279             const parts = c && c.content && c.content.parts;
7280             if(!Array.isArray(parts)) continue;
7281             for(const p of parts){
7282                 const inl = p.inlineData || p.inline_data;
7283                 if(inl && inl.data) return { b64: inl.data, mime: inl.mimeType || inl.mime_type || 'image/png' };
7284             }
7285         }
7286     }
7287     return null;
7288 }
7289 /* Google xato javobini tushunarli xabarga aylantirish */
7290 function aiApiErr(status, data){
7291     const msg = (data && data.error && data.error.message) || '';
7292     console.error(['ai'], status, data);
7293     const m = msg.toLowerCase();
7294
7295     if(status === 400 && /api key not valid/.test(m))
7296         return { short:'API kalit noto\'g\'ri – tekshirib qayta kiriting', detail:msg };
7297     if(status === 403)
7298         return { short:'Kalitga ruxsat yo\'q – Google AI Studio\'da kalitni tekshiring', detail:msg };
7299     if(status === 404)
7300         return { short:'Model topilmadi – «Model» maydonidagi nomni tekshiring', detail:msg };
7301
7302     if(status === 429){
7303         /* Google 429 ni ikki xil holatda qaytaradi va ular butunlay boshqacha:
7304             1) haqiqiy tezlik chegarasi – kutib qayta urinsa bo'ladi;
7305             2) bu model bepul rejada UMUMAN yo'q (quota_limit_value: 0) –
7306                kutish yordam bermaydi, to'lov (billing) yoqilishi kerak.
7307             Ikkinchisini "birozdan so'ng urining" deb ko'rsatish – chalg'itish. */
7308         const freeTier = /free[_ ]tier|billing|plan and billing|quota_limit_value.*?0\b/.test(m);
7309         if(freeTier) return {
7310             short:'Bu model bepul rejada mavjud emas – Google hisobingizda to'lov (billing) yoqilishi kerak',
7311             detail:msg, hint:'billing'
7312         };
7313         return { short:'So'rovlar chegarasi – 1–2 daqiqadan so'ng qayta urining', detail:msg };
7314     }
7315     if(status >= 500)
7316         return { short:'Google serverida vaqtincha nosozlik – qayta urining', detail:msg };
7317     return { short: msg ? ('Xato: ' + msg.slice(0,120)) : ('Xato: HTTP ' + status), detail:msg };
7318 }
7319
7320 async function aiGenerate(prompt){
7321     if(!aiReady()){
7322         ntf('Avval o AI sozlash – API kalitini kiriting', 'err');
7323         openAiSetup();
7324         return;
7325     }
7326     const card = aiLoadingCard();
7327     /* e – matn yoki {short, detail, hint}. Google'ning O'Z xabari yashirilmaydi:
7328         aynan o'sha matn muammoning haqiqiy sababini aytadi. */
7329     const fail = (e)=>{
7330         const short = (typeof e === 'string') ? e : e.short;
7331         const detail = (typeof e === 'string') ? '' : (e.detail || '');
7332         const hint = (typeof e === 'string') ? '' : (e.hint || '');
7333         if(card){
7334             card.classList.remove('ai-loading');
7335             let html = '<div class="ai-err">▲ ' + escHTML(short) + '</div>';
7336             if(hint === 'billing'){
7337                 html += '<div class="meta" style="line-height:1.6">Nima qilish kerak:<br> +
7338                     '1. <b>aistudio.google.com/apikey</b> → kalitingiz qaysi loyihaga tegishli ekanini ko'ring<br> +
7339                     '2. <b>console.cloud.google.com/billing</b> → o'sha loyihaga to'lov usulini ulang<br> +
7340                     '3. Yoki « AI sozlash → Model» da bepul rejada mavjud boshqa modelni sinang<br> +
7341                     '<i>Eslatma: «Aniq naqsh» rejimi bepul va cheksiz ishlaydi.</i></div>';
7342             }
7343             if(detail){
7344                 html += '<details class="meta"><summary>Google javobi (texnik)</summary>' +
7345                     '<div style="white-space:pre-wrap;word-break:break-word;margin-top:6px">' +
7346                     escHTML(detail.slice(0, 600)) + '</div></details>';
7347             }
7348             card.innerHTML = html;
7349         }
7350         ntf(short, 'err');
7351     };
7352     try{
7353         let res, data;
7354         if(getAiConn() === 'worker'){
7355             res = await fetch(getAiWorkerUrl(), {
7356                 method: 'POST',
7357                 headers: { 'Content-Type': 'application/json', 'X-App-Token': getAiAppToken() },
7358                 body: JSON.stringify({ prompt })
7359             });
7360             data = await res.json().catch(()=>({}));
7361             if(!res.ok){
7362                 /* Worker xatoni matn sifatida uzatadi ({error:"..."}), Google esa
7363                 obyekt sifatida ({error:{message}}). Ikkalasini ham aiApiErr
7364                 tushunadigan bitta shaklga keltiramiz – shunda bepul reja/billing
7365                 holati proksi orqali ham to'g'ri aniqlanadi. */
7366                 const e0 = data && data.error;
7367                 fail(aiApiErr(res.status, (typeof e0 === 'string') ? {error:{message:e0}} : data));

```

```

7368         return;
7369     }
7370 } else {
7371     const model = encodeURIComponent(getAiModel());
7372     const url = 'https://generativelanguage.googleapis.com/v1beta/models/' + model + ':generateContent';
7373     res = await fetch(url, {
7374         method: 'POST',
7375         headers: { 'Content-Type': 'application/json', 'x-goog-api-key': getAiKey() },
7376         body: JSON.stringify({ contents: [{ parts: [{ text: prompt }] }] });
7377     });
7378     data = await res.json().catch(() => ({}));
7379     if(!res.ok){ fail(aiApiErr(res.status, data)); return; }
7380 }
7381
7382 const img0 = aiExtractImage(data);
7383 if(!img0){
7384     /* model matn qaytargan bo'lishi mumkin – uni ko'rsatamiz */
7385     let txt = '';
7386     try{
7387         const parts = data.candidates[0].content.parts;
7388         txt = parts.map(p=>p.text).filter(Boolean).join(' ');
7389     }catch(e){}
7390     fail(txt ? ('Model rasm o\'rniga matn qaytardi: ' + txt.slice(0,160))
7391           : 'Javobda rasm topilmadi – «Model» nomi rasm chizadigan model ekanini tekshiring');
7392     return;
7393 }
7394
7395 if(card){
7396     card.classList.remove('ai-loading');
7397     const img = document.createElement('img');
7398     img.src = 'data:' + img0.mime + ';base64,' + img0.b64;
7399     card.innerHTML = '';
7400     card.appendChild(img);
7401     const meta = document.createElement('div');
7402     meta.className = 'meta';
7403     meta.innerHTML = '□ <b>Nano Banana</b> · "' + escHTML(prompt) + '"';
7404     card.appendChild(meta);
7405     const acts = document.createElement('div');
7406     acts.className = 'acts';
7407     const bdl = document.createElement('button');
7408     bdl.className = 'btn sm gold'; bdl.textContent = 'Yuklab olish';
7409     bdl.addEventListener('click', ()=>{
7410         dlBlob(_dataURLtoBlob(img.src), img0.mime, 'naqshai_ai_' + Date.now() + '.png', 'Rasm yuklab olindi □');
7411     });
7412     acts.appendChild(bdl);
7413     card.appendChild(acts);
7414 }
7415 const log = $('chat-log'); if(log) log.scrollTop = log.scrollHeight;
7416 }catch(e){
7417     console.error('[ai]', e);
7418     fail(/failed to fetch|networkerror/i.test(e.message) ?
7419         'Serverga ulanib bo\'lmadi – internetni tekshiring'
7420         : ('Xatolik: ' + e.message));
7421 }
7422 }
7423 function wireAiSetup(){
7424     const seg = $('chat-mode-seg');
7425     if(seg) seg.addEventListener('click', e=>{
7426         const b = e.target.closest('button');
7427         if(!b) return;
7428         CHAT_MODE = b.dataset.v;
7429         seg.querySelectorAll('button').forEach(x=>x.classList.remove('sel'));
7430         b.classList.add('sel');
7431         if(CHAT_MODE === 'ai' && !aiReady()){
7432             ntf('AI rasm rejimi uchun API kalit kerak (bir marta)', 'err');
7433             openAiSetup();
7434         }
7435         const inp = $('chat-input');
7436         if(inp) inp.placeholder = CHAT_MODE === 'ai'
7437             ? "Masalan: quyosh nurida oltin girih naqshli chinni laganning fotosi"
7438             : "Masalan: 8 simmetriyali oltin girih chiz";
7439     });
7440     const setupBtn = $('btn-ai-setup'); if(setupBtn) setupBtn.addEventListener('click', openAiSetup);
7441     const cancel = $('ai-setup-cancel'); if(cancel) cancel.addEventListener('click', closeAiSetup);
7442     const back = $('modal-ai-setup'); if(back) back.addEventListener('click', e=>{ if(e.target===back) closeAiSetup() });
7443     const cseg = $('ai-conn-seg');
7444     if(cseg) cseg.addEventListener('click', e=>{
7445         const b = e.target.closest('button');
7446         if(b) aiShowConn(b.dataset.v);
7447     });
7448     const save = $('ai-setup-save'); if(save) save.addEventListener('click', ()=>{
7449         const chosen = $('ai-worker-box') && !$('ai-worker-box').hidden ? 'worker' : 'direct';
7450         setAiConn(chosen);
7451         const u = $('ai-worker-url'), t = $('ai-app-token'), k = $('ai-key'), m = $('ai-model');
7452         setAiWorkerUrl(u?u.value: ''); setAiAppToken(t?t.value: '');
7453         setAiKey(k?k.value: ''); setAiModel(m?m.value: '');
7454         closeAiSetup();
7455         ntf(aiReady() ? 'Saqlandi ✓ – AI rasm rejimi tayyor' : 'Saqlandi, lekin kalit/manzil bo\'sh', aiReady()?'ok':'err');
7456     });

```

```

7457 }
7458 function sendChat(){
7459   const inp = $('chat-input');
7460   if(!inp) return;
7461   const txt = (inp.value||'').trim();
7462   if(!txt) return;
7463   inp.value = '';
7464   chatUserMsg(txt);
7465   if(CHAT_MODE === 'ai'){
7466     aiGenerate(txt);
7467     return;
7468   }
7469   const np = parseChat(txt);
7470   setTimeout(()=>{ chatCard(np, txt); saveChatHistory(); }, 120);
7471 }
7472 const CHAT_CHIPS = [
7473   '8 simmetriyali oltin girih chiz',
7474   'Feruza islamiy, zich',
7475   'Ismingizni yozing – shaxsiy naqsh',
7476   '12 ta mandala, murakkab',
7477   "Ko'k penrose, sodda",
7478   'Oltin girih tessellatsiya p6m',
7479   'Samarqand 2026'
7480 ];
7481 function buildChips(){
7482   const box = $('chat-chips');
7483   if(!box) return;
7484   box.innerHTML = '';
7485   CHAT_CHIPS.forEach(txt=>{
7486     const b = document.createElement('button');
7487     b.textContent = txt;
7488     b.addEventListener('click', ()=>{
7489       const inp = $('chat-input');
7490       if(txt.indexOf('Ismingizni')===0){
7491         if(inp){ inp.value=''; inp.placeholder='Ismingizni yozing...'; inp.focus() && inp.focus(); }
7492         return;
7493       }
7494       if(inp){ inp.value = txt; }
7495       sendChat();
7496     });
7497     box.appendChild(b);
7498   });
7499 }
7500
7501 /* =====
7502 NAZARIYA SAHIFASI – INTERAKTIV DARSLIK
7503 Har naqsh: jonli preview + model + formulalar + algoritm + kod
7504 ===== */
7505 function escHTML(t){
7506   return String(t).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
7507 }
7508 const THEORY = {
7509   girih:{
7510     model:`Kanonik islomiy rozetta: {n/k} yulduz o'zagi + kite-gulbarglar halqasi. Chiziqlar lenta (ribbon) sifatida
7511     – ikki qoramtir kontur orasidagi band – chiziladi va navbatlashib to'qiladi.` ,
7512     frm:`P_i = (R*cos θ_i, R*sin θ_i), θ_i = 2πi/n – π/2`,
7513     `k = {n/2} – 1 (vatar qadami: P_i → P_{i+k})`,
7514     `r_2 = R(0.36 + i*r_0.28) – yulduz uchlari; r_1 = r_2(0.66 – spir*0.28)`,
7515     `Kite: A(θ, r_2) → L(θ-w, r_m) → T(θ, R_0) → R_r(θ+w, r_m), w = (π/n)*0.86*(0.72+dens*0.55)`],
7516     algo:`Aylanada n ta nuqta hisoblanadi`,
7517     `Har yo'nalishda kite-gulbarg to'ldiriladi (4 bo'rtgan Bezier yon)`,
7518     `starPath(n, r_2, r_1) – yulduz o'zagi to'ldiriladi`,
7519     `{n/k} vatarlar IKKI O'TISHDA lenta qilib chiziladi: juftlari ostda, toqlari fon-uzilish (ribbonI) bilan us
7520     tida → to'qish`,
7521     `Juft vatarlarning markaziy 42–58% segmenti yana ustidan – weave yakunlanadi`,
7522     `Tashqi halqa lentasi + tugunlarda mini 8-yulduzlar`,
7523     code:`const pts=[]; for(let i=0;i<n;i++) pts.push(pol(TAU*i/n-Math.PI/2, R0));
7524     const chord=(i)=>{ const A=pts[i], B=pts[(i+k)%n];
7525     ctx.beginPath(); ctx.moveTo(A[0],A[1]); ctx.lineTo(B[0],B[1]); };
7526     for(let pass=0; pass<2; pass++)
7527     for(let i=pass; i<n; i+=2){ chord(i); ribbonI(ctx,c0,edge,bw); }`,
7528     par:`<b>sym</b>→n, <b>ir</b>→r_2 (yulduz o'lchami), <b>dens</b>→gulbarg eni w, <b>spir</b>→yulduz o'tkirligi r_1, <
7529     b>cmp>5</b>→ichki mini-rozetta`,
7530     hankin8:{
7531       model:`Hankin usuli (polygons-in-contact): tekislik 4.8.8 Arximed plitkalashi (sakkizburchak + kvadrat) bilan qop
7532       lanadi; strapwork plitkalardan avtomatik "tug'iladi" – hech bir chiziq qo'lda joylanmaydi.` ,
7533       frm:`oR = s/2 / cos(π/8) – oktagon radiusi (s – panjara qadami)`,
7534       `Qirra o'rtasi M_i dan ikki nur: yo'nalish d_i ± δ, δ = π(0.2 + spir*0.18)`,
7535       `Kesishma: M_i + t*(cos(d_i+δ), sin(d_i+δ)) = M_{i+1} + u*(cos(d_{i+1}+π-δ), ...)`],
7536       `Kvadrat uchlari = 4 qo'shni oktagonning eng yaqin burchaklari`,
7537       algo:`Panjara bo'ylab oktagon va oraliq kvadrat plitkalar quriladi`,
7538       `Har plitkada: qirralar o'rtalari M_i va yo'nalishlari d_i topiladi`,
7539       `Qo'shni nurlar juftlari kesishtiriladi (rayX) → [M_i, X, M_{i+1}] strap`,
7540       `Plitka fonlari navbat rangda to'ldiriladi`,
7541       `Oktagon straplari oddiy lenta, kvadratniki ribbonI (to'qish) bilan ustidan`,
7542       code:`function hankinStraps(poly, ang){
7543       const m=poly.length, mids=[], dirs=[], straps=[];
7544       for(let i=0;i<m;i++){ const A=poly[i],B=poly[(i+1)%m];
7545       mids.push([(A[0]+B[0])/2,(A[1]+B[1])/2]);
7546       dirs.push(Math.atan2(B[1]-A[1],B[0]-A[0])); }
7547       for(let i=0;i<m;i++){ const j=(i+1)%m;

```

```

7544     const X=rayX(mids[i],dirs[i]+ang, mids[j],dirs[j]+Math.PI-ang);
7545     if(X) straps.push([mids[i],X,mids[j]]); }
7546     return straps; }`,
7547     par: `<b>dens</b>->panjara zichligi (katak soni), <b>spir</b>->kontakt burchagi 6 (yulduz o'tkirligi)`},
7548     hankin6:{
7549     model: `Hankin usuli geksagonal 6.6.6 panjarada: har muntazam olti burchakdan kontakt burchagiga qarab 6 qirrali y
7550     ulduz hosil bo'ladi.` ,
7551     frm: `Markazlar:  $x = g_x \cdot \sqrt{3} \cdot s + (g_y \bmod 2) \cdot \sqrt{3} \cdot s/2$ ,  $y = g_y \cdot 1.5 \cdot s$ ,
7552     `Hex uchlari:  $C + s \cdot (\cos(\pi/6 + 2\pi i/6), \sin(\pi/6 + 2\pi i/6))$ `,
7553     `6 =  $\pi(0.24 + \text{spir} \cdot 0.16)$ `},
7554     algo: `Asal uyasi panjarasi quriladi (qatorlar yarim qadam siljigan)`,
7555     `Har hexga hankinStraps(6) qo'llanadi`,
7556     `Fonlar  $(g_x + 2g_y) \bmod 3$  bo'yicha 3 rangda almashadi`,
7557     code: `for(let gy=-N;gy<=N;gy++) for(let gx=-N;gx<=N;gx++){
7558     const x=gx*dx+(gy%2*dx/2:0), y=gy*dy;
7559     const h=[]; for(let i=0;i<6;i++){
7560     h.push([x+Math.cos(Math.PI/6+TAU*i/6)*s, y+Math.sin(...)*s]);
7561     strokeStraps(ctx, hankinStraps(h,ang), band, edge, bw); }`,
7562     par: `<b>dens</b>->hex o'lchami s, <b>spir</b>->6 burchagi`,
7563     panelgirih:{
7564     model: `Klassik kvadrat o'quv paneli: ikki kvadrat 45° buralib 8 qirrali yulduz beradi; oq lenta straplar, ramka b
7565     andlari va burchak choraklari – o'zbek naqqoshlik maktabining "sakkiz qirrali girih" darsligi.` ,
7566     frm: `Yulduz: kvadrat1( $\theta_0 = -\pi/2$ ) u kvadrat2( $\theta_0 = -\pi/4$ ), uchlari radiusi r`,
7567     `Ichki oktagon radiusi  $\approx 0.545r$  (kvadratlar kesishmasi)`,
7568     `starPath(8, r, 0.765r) – {8} yulduz maydoni`,
7569     algo: `Tashqi ramka: 2 kvadrat lenta band + orasida mini-rozetka nuqtalari`,
7570     `Burchak choraklari Bezier bilan to'ldiriladi`,
7571     `8 kite-gulbarg + yulduz maydoni fill`,
7572     `Kvadrat1 ostda, kvadrat2 ribbonI ustida, keyin kvadrat1 ning 2 qarama-qarshi tomoni yana ustidan → haqiqiy
7573     weave`,
7574     code: `const sq=(rot)=>{const q=[];for(let i=0;i<4;i++)q.push(pol(rot+TAU*i/4,r));return q};
7575     ctx.beginPath(); polyPath(ctx, sq(-Math.PI/2)); ribbon(ctx,band,edge,bw);
7576     ctx.beginPath(); polyPath(ctx, sq(-Math.PI/2+Math.PI/4)); ribbonI(ctx,band,edge,bw);`,
7577     par: `<b>dens</b>->ramka mini-rozetkalar soni`,
7578     xatam:{
7579     model: `Xatam yulduzi: ikki kvadrat 45° ustma-ust – 8 qirrali yulduz; to'ldirilgan yulduz maydoni, oktagon o'zak v
7580     a to'qilgan lenta kvadratlar.` ,
7581     frm: `Kvadrat uchlari:  $\text{pol}(\theta_0 + 2\pi i/4, r)$ ,  $\theta_0 \in \{-\pi/4, 0\}$ ,
7582     `Yulduz ichki nisbat:  $r_{in}/r_{out} = 0.765 \approx \cos(\pi/8)^2$ `,
7583     algo: `8-yulduz maydoni fill → ichki oktagon fill`,
7584     `Kvadrat A lenta, kvadrat B ribbonI, A ning 28–72% segmentlari qayta ustidan`,
7585     `Uchlardan tashqi halqaga nurlar + halqa band`,
7586     code: `ctx.beginPath(); starPath(ctx, 8, r, r*0.765, rot); fillA(ctx,c1,0.55);
7587     ctx.beginPath(); polyPath(ctx,A); ribbon(ctx,c0,edge,bw);
7588     ctx.beginPath(); polyPath(ctx,B); ribbonI(ctx,c1,shade(c1,.5),bw);`,
7589     par: `<b>cmp</b>->5</b>->ikkinchi ichki daraja (0.5 masshtab, 22.5° burilgan)`},
7590     rozetta:{
7591     model: `Gulbarg halqalari: har barg ikki kubik Bezier yon + o'tkir uch; bandlar konsentrik – tashqi katta barglar,
7592     oraliqda kichiklari, markazda halqa+yulduzcha.` ,
7593     frm: `Barg asosi: ( $\theta \neq w$ ,  $r_{in}$ ), uchi: ( $\theta$ ,  $r_{out}$ ),
7594     `Yon nazorat nuqtalari: ( $\theta \neq 1.12w$ ,  $r_{in} + 0.42\Delta r$ ) va ( $\theta \neq 0.5w$ ,  $0.93r_{out}$ ),
7595     `w =  $(\pi/n)(0.55 + \text{dens} \cdot 0.35)$ `,
7596     algo: `n ta tashqi barg: petalPath fill + kontur + o'rta tomir`,
7597     `n ta oraliq kichik barg (yarim qadam siljigan)`,
7598     `Markaz: lenta halqa (radius ir ga bog'liq) + n-yulduzcha + cmp bo'yicha nuqta halqalari`,
7599     code: `function petalPath(ctx,a,rIn,rOut,hw){
7600     const bl=pol(a-hw,rIn), tip=pol(a,rOut), br=pol(a+hw,rIn);
7601     ctx.moveTo(bl[0],bl[1]);
7602     ctx.bezierCurveTo(...c1,...c2, tip[0],tip[1]); // chap yon
7603     ctx.bezierCurveTo(...c1,...c2, br[0],br[1]); // o'ng yon
7604     ctx.closePath(); }`,
7605     par: `<b>sym</b>->barglar n, <b>dens</b>->barg eni, <b>ir</b>->markaz halqasi, <b>cmp</b>->nuqta halqalari`,
7606     yulduz:{
7607     model: `n/k yulduz poligoni to'qilgan lentalarda: har uch Pi dan P{i+k} ga vatar; qo'shni vatarlar kesishmasi
7608     markaziy n-burchakni beradi va u to'ldiriladi.` ,
7609     frm: `Vatar: Pi → P{(i+k) mod n},  $k = \lfloor n/2 \rfloor - 1 + \lfloor \text{spir} \cdot 1.5 \rfloor$ ,
7610     `Ichki uch:  $X = \text{segX}(P_i, P_{i+k}, P_{i+1}, P_{i+1-k})$ `,
7611     algo: `Vatarlarning ketma-ket kesishmalari hisoblanib ichki poligon fill qilinadi`,
7612     `Vatarlar 2 o'tishda ribbonI bilan to'qiladi`,
7613     `Juftlarning o'rta segmenti ustidan; halqa band + tugun nuqtalar`,
7614     code: `const inner=[];
7615     for(let i=0;i<n;i++){
7616     const X = segX(pts[i], pts[(i+k)%n],
7617     pts[(i+1)%n], pts[(i+1-k)%n]);
7618     if(X) inner.push(X); }
7619     ctx.beginPath(); polyPath(ctx, inner); fillA(ctx,c1,0.85);`,
7620     par: `<b>sym</b>->n, <b>spir</b>->k qadami (yulduz "o'ramliligi")`,
7621     muqarnas:{
7622     model: `Pog'onali arkada modeli: har daraja radiusi geometrik kamayadi (0.74 koeffitsiyent), darajada m ta tokcha
7623     (nisha) – har biri ikki kubik Bezier ravoq, uchida osilgan tomchi.` ,
7624     frm: `rl =  $0.94R \cdot 0.74^L$ ,  $l = 0 \dots (\text{cmp}+1)$ ,
7625     `ml = n + 2l tokcha; osilish:  $M = \text{pol}(\theta_{mid}, r_l - \text{drop})$ ,  $\text{drop} = r_l(0.14 + 0.2 \cdot \text{ir})$ `,
7626     algo: `Har darajada tokcha fonlari (ravoq + yoy bilan yopiq) navbat rangda`,
7627     `Ravoq chizig'i yaxlit lenta sifatida`,
7628     `Osilgan tomchi nuqtalari; rang darajalar bo'ylab lerpColor bilan o'tadi`,
7629     `Markazda yulduz o'zak`,
7630     code: `for(let l=0;l<levels;l++){
7631     const r=R*0.94*Math.pow(0.74,l), m=n+l*2, drop=r*(0.14+ir*0.2);
7632     for(let i=0;i<m;i++){
7633     const A=pol(a0,r), B=pol(a1,r), M=pol((a0+a1)/2, r-drop);
7634     ctx.bezierCurveTo(...,-M...); // ikki Bezier ravoq A-M-B

```

```

7628   } ribbon(ctx, lerpColor(c0,c1,l/levels), ...); }`,
7629   par: <b>sym</b>->bazaviy tokcha soni, <b>cmp</b>->darajalar, <b>ir</b>->osilish chuqurligi`,
7630   katak8:{
7631     model:`4.8.8 Arximed tessellatsiyasi: har uchda bitta kvadrat + ikki oktagon uchrashadi. Kvadrat uchlari oktagon
burchaklaridan olinadi – plitkalar matematik zich yopishadi.` ,
7632     frm:[`s – qadam; oR = s/2/cos(π/8)` ,
7633           `Kvadrat yarim dioganali q ≈ 1.35·(s/2 – oR·cos(π/8))`],
7634     algo:[`Panjara bo'ylab oktagonlar (fill + lenta kontur, ichida 8-yulduzcha)` ,
7635           `Oraliq nuqtalarda 45° kvadratlar fill` ,
7636           `Doira clip + tashqi halqa (yakka rejimda)`],
7637     code:`for(let gy=0;gy<=cells;gy++) for(let gx=0;gx<=cells;gx++){
7638       const x=-half+gx*s, y=-half+gy*s;
7639       const oct=[]; for(let i=0;i<8;i++){
7640         oct.push([x+Math.cos(Math.PI/8+TAU*i/8)*oR, y+Math.sin(...)*oR]);
7641         ctx.beginPath(); polyPath(ctx,oct); fillA(...); ribbon(...); }`,
7642     par: <b>dens</b>->katak soni, <b>cmp</b>->ichki yulduzchalar`,
7643     hexa:{
7644       model:`Asal uyasi (p6m panjara): qatorlar dx/2 siljigan hex to'r; har katak ichida 6 barg rozetkasi.` ,
7645       frm:[`dx = √3·s, dy = 1.5·s; x = g_x·dx + (g_y mod 2)·dx/2`],
7646       algo:[`Hex panjara quriladi, kataklar 3 rangda davriy bo'yaladi` ,
7647             `Lenta konturlar; cmp≥4 da har katakka 6 ta barg + markaz nuqta`],
7648       code:`const pts=[]; for(let i=0;i<6;i++){
7649         pts.push([x+Math.cos(Math.PI/6+TAU*i/6)*s, y+Math.sin(...)*s]);
7650         polyPath(ctx,pts); fillA(...); ribbon(ctx,c0,edge,bw);` ,
7651       par: <b>dens</b>->katak o'lchami, <b>cmp</b>->ichki barglar`,
7652       zanjira:{
7653         model:`Interlock halqalar zanjiri: n ta halqa aylana bo'ylab, radiusi qo'shnilar kesishadigan qilib tanlanadi; har
n keyingi halqa ribbonI bilan oldingisini "kesib" o'tadi, oxirida bitta yoy zanjirni yopadi.` ,
7654         frm:[`r_r = (π·r_c/n)(0.92 + dens·0.35) – halqa radiusi` ,
7655               `Markazlar: pol(2πi/n – π/2, r_c), r_c = 0.68R`],
7656         algo:[`Halqalar ketma-ket ribbonI bilan chiziladi – har biri oldingisining ustidan` ,
7657               `1-halqaning oxirgi halqa tomonidagi yoyi (arcB) qayta ustidan → zanjir yopiladi` ,
7658               `Markazda medalyon: lenta halqa + yulduz`],
7659         code:`for(let i=0;i<n;i++){
7660           const c=centers[i];
7661           ctx.beginPath(); circle(ctx,c[0],c[1],rr);
7662           ribbonI(ctx, i%2?c1:c0, shade(col,.5), bw); }
7663         arcB(ctx, centers[0][0], centers[0][1], rr, dir-0.7, dir+0.7);
7664         ribbonI(ctx, c0, edge, bw); // zanjirni yopish` ,
7665         par: <b>sym</b>->halqalar soni, <b>dens</b>->halqa kesishuvi`,
7666         islimiy:{
7667           model:`Logarifmik spiral novdalar: o'sish qonuni r(θ)=a·e^{bθ}; poya torayuvchi (3 o'tishli qalinlik), barglar ta
ngens burchagida navbatma-navbat, uchida uch bargli palmetta kurtak.` ,
7668           frm:[`r(θ) = a·e^{bθ}, b = 0.1 + spir·0.28, a = L·e^{-b·θ_max}` ,
7669                 `θ_max = π(1.2 + cmp·0.18); barg burchagi = atan2(Δy,Δx) ± π/2.5`],
7670           algo:[`SpiralPts: θ bo'ylab namuna nuqtalar → Catmull-Rom → kubik Bezier` ,
7671                 `stem(): bir yo'l 3 marta kamayuvchi qalinlikda – taper illyuziyasi` ,
7672                 `Har k-nuqtada fineLeaf (fill + kontur + o'rta tomir), o'lcham uchga qarab o'sadi` ,
7673                 `Oxirida budAt – 3 barg fan; cmp≥3 da qarshi kichik gajak`],
7674           code:`function spiralPts(b,thMax,L,mir){
7675             const a=L/Math.exp(b*thMax), pts=[];
7676             for(let i=0;i<=steps;i++){
7677               const th=thMax*i/steps, r=a*Math.exp(b*th);
7678               pts.push([Math.cos(mir*th)*r, Math.sin(mir*th)*r]); }
7679             return pts; }`,
7680           par: <b>sym</b>->novdalar, <b>spir</b>->b (o'ralish), <b>cmp</b>->θ_max, <b>dens</b>->barg zichligi`,
7681           rumi:{
7682             model:`Klassik rumi (buralgan barg) motivi: 4 kubik Bezierdan iborat ilmoqli yopiq shakl, tok halqasi bo'ylab tas
hqi-katta / ichki-kichik juftlikda; barglar orasidan chetga S-novdalar chiqadi – tessellatsiyada qo'shnilar bilan
tutashadi.` ,
7683             frm:[`Motiv: (0,0)→(0.94L,-0.1L) yuqori yon, ilmoq uchi (0.7L, 0.13L), ichki qaytish → yopiq` ,
7684                   `Bog'lovchi novda: r(t)=0.96r_c+(1.06R-0.96r_c)t, θ(t)=θ_0+0.16·sin(πt)·(±1)`],
7685             algo:[`Tok halqasi lenta bilan` ,
7686                   `Har pozitsiyada katta rumi barg (tashqariga) + kichigi (ichkariga), ichida ilmoq chizig'i` ,
7687                   `Barglar orasidan chetga S-novda + o'rta va uch barglar` ,
7688                   `Markazda yulduz rozetkasi`],
7689             code:`function rumiLeafPath(ctx,L){
7690               ctx.moveTo(0,0);
7691               ctx.bezierCurveTo(L*.12,-L*.36, L*.68,-L*.42, L*.94,-L*.1);
7692               ctx.bezierCurveTo(L*.04,L*.04, L*.9,L*.2, L*.7,L*.13); // ilmoq
7693               ctx.bezierCurveTo(L*.52,L*.06, L*.42,L*.2, L*.28,L*.16);
7694               ctx.bezierCurveTo(L*.12,L*.1, L*.04,L*.05, 0,0);
7695               ctx.closePath(); }`,
7696             par: <b>sym</b>->motivlar soni, <b>dens</b>->barg o'lchami L, <b>skew</b>->burilish`,
7697             hatayi:{
7698               model:`Stilizatsiyalangan lotus: har gulbarg 3 Bezier bosqich – keng asos → bel (58% balandlikda) → o'tkir uch; i
kki band + urug'don halqasi.` ,
7699               frm:[`Bel nuqtalari: (θ±0.62w, r_in + 0.58Δr); uch: (θ, r_out)` ,
7700                     `m = 5 + [cmp≥3] + [cmp≥5] gulbarg`],
7701               algo:[`Tashqi m gulbarg (3-segment yo'l) + o'rta tomirlar` ,
7702                     `Oraliq kichik gulbarglar (yarim qadam)` ,
7703                     `Urug'don: lenta halqa + nuqtalar doirasi + markaz`],
7704               code:`ctx.moveTo(bl...); // asos chap
7705               ctx.bezierCurveTo(c1..., wl..., wl...); // asos-bel
7706               ctx.bezierCurveTo(wl..., c2..., tip...); // bel-uch
7707               ctx.bezierCurveTo(c3..., wr..., wr...); // uch-bel (o'ng)
7708               ctx.bezierCurveTo(wr..., c4..., br...); // bel-asos` ,
7709               par: <b>cmp</b>->gulbarg soni, <b>dens</b>->barg eni, <b>sym</b>->urug'don nuqtalari`,
7710               palak:{
7711               model:`Nilufar: 3 konsentrik gulbarg halqasi jadval bilan beriladi – har halqaning (r_in, r_out, en, siljish, ran
g) parametrlari; halqalar yarim qadam navbatlashadi.` ,

```

```

7712 frm:[`Halqa1: 0.55R-0.95R; Halqa2: 0.32R-0.6R (siljish ½); Halqa3: 0.14R-0.36R`],
7713 algo:[`Har halqada n barg petalPath bilan, fill ranglari navbat`,
7714 `cmp halqalar sonini boshqaradi; markaz lenta + nuqta`],
7715 code:`const rings=[{rIn:.55*R,rOut:.95*R,off:0},
7716 {rIn:.32*R,rOut:.6*R, off:.5},
7717 {rIn:.14*R,rOut:.36*R,off:0}];
7718 for(const rg of rings) for(let i=0;i<n;i++)
7719 petalPath(ctx, TAU*(i+rg.off)/n - Math.PI/2, rg.rIn, rg.rOut, rg.w);`,
7720 par:`<b>sym</b>->barglar, <b>cmp</b>->halqalar soni`,
7721 shamsa:{
7722 model:`Quyosh medalyoni – 3 band kompozitsiya: markaziy to'ldirilgan yulduz, 2n alanga-nur (uzun/qisqa navbat, to
mchi-barg shaklida), tashqi scallop-gulbarg bandi.`,
7723 frm:[`Nurlar: boshi (0, 0.4R), uzunligi R.{0.36, 0.22} navbat`,
7724 `Tashqi band: 2n gulbarg 0.8R-0.955R`],
7725 algo:[`Tashqi halqa lentasi + gulbarg bandi`,
7726 `2n nur-barg (leaf, 2 Bezier) – juftlari uzun`,
7727 `Markaz: lenta halqa + katta yulduz fill + ichki qarama-qarshi yulduzcha`],
7728 code:`for(let i=0;i<n*2;i++){
7729 const a=TAU*i/(n*2)-Math.PI/2, long=i%2===0;
7730 const s0=pol(a, R*0.4), len=R*(long?0.36:0.22);
7731 ctx.beginPath(); leaf(ctx,s0[0],s0[1],a,len,len*0.14);
7732 fillA(ctx, long?c0:c2, 0.9); }`,
7733 par:`<b>sym</b>->n (nurlar = 2n)`,
7734 arabesk:{
7735 model:`Ogee (S-egri) to'ri: ichki n tugundan tashqi (yarim qadam siljigan) n tugunga ikki oila S-egri – nazorat n
uqtalari o'rta chiziqning normal bo'ylab offset siljiriladi; tutashuvlarda palmetta.`,
7736 frm:[`S-egri nazorat: 0.6A+0.4M ± n`off, off = 0.09R(0.6+spir)`,
7737 `Tugunlar: ichki (0_i, 0.4R), tashqi (0_i+π/n, 0.88R)`,
7738 algo:[`Birinci oila (+off) lenta, ikkinchi oila (-off) ribbonI – kesishuvda to'qiladi`,
7739 `Tashqi tugunlarda budAt palmetta, ichkilarida barg juftligi`,
7740 `Markaz halqa + yulduz`,
7741 code:`ctx.moveTo(A[0],A[1]);
7742 ctx.bezierCurveTo(
7743 A[0]*.6+mx*.4 + nx/L*off, A[1]*.6+my*.4 + ny/L*off,
7744 B[0]*.6+mx*.4 - nx/L*off, B[1]*.6+my*.4 - ny/L*off,
7745 B[0], B[1]);`,
7746 par:`<b>sym</b>->tugunlar, <b>spir</b>->S-egri chuqurligi`,
7747 scroll:{
7748 model:`Bargli novda: asosiy S-poya bitta kubik Bezier; barglar egri bo'ylab B(t) formulasidan namuna olinib, tang
ens burchagiga ±π/2.3 da navbatlashadi; uchida log-spiral gajak.`,
7749 frm:[`B(t) = (1-t)³P₀ + 3(1-t)²tP₁ + 3(1-t)t²P₂ + t³P₃`,
7750 `Barg burchagi: atan2(B(t+ε)-B(t)) ± π/2.3`],
7751 algo:[`P₀...P₃ nazorat nuqtalari (S-shakl) → stem taper bilan`,
7752 `t_j = j/(m+1) da cubicAt namunalari, fineLeaf navbat tomonlarda, o'lcham t bilan o'sadi`,
7753 `Uchida spiralPts gajak`,
7754 code:`function cubicAt(p0,p1,p2,p3,t){ const u=1-t;
7755 return [u*u*u*p0[0]+3*u*u*t*p1[0]+3*u*t*t*p2[0]+t*t*t*p3[0],
7756 u*u*u*p0[1]+3*u*u*t*p1[1]+3*u*t*t*p2[1]+t*t*t*p3[1]];`,
7757 par:`<b>sym</b>->novdalar, <b>dens</b>->barg soni, <b>spir</b>->barg o'lchami, <b>skew</b>->S chuqurligi`,
7758 medallion:{
7759 model:`Ko'p bandli medalyon – kompozitsion model: tashqi lenta halqa → scallop (chig'anoq) band → gulbarg band →
markazda 0.5 masshtabda to'liq girih rozetta chaqiriladi.`,
7760 frm:[`Scallop: har segment A-M(0.82R)→B ikki Bezier; gulbarglar 0.5R-0.8R`],
7761 algo:[`4 band tashqaridan ichkariga ketma-ket`,
7762 `drawGirih(ctx, R*0.95, ...) 0.5 masshtabli kontekstda qayta ishlatiladi – kod kompozitsiyasi`],
7763 code:`ctx.save(); ctx.scale(0.5,0.5);
7764 drawGirih(ctx, R*0.95, {...p, sym:n}, sw*1.6, c0,c1,c2,c3);
7765 ctx.restore();`,
7766 par:`girih parametrlarining barchasi ichki o'zakka o'tadi`,
7767 gi:{model:`Yuqori tartibli funksiya kompozitsiyasi: makeCombo(fA,fB) – tashqi uslub to'liq R da, ichki uslub 0.5 m
ashtab + yarim qadam burilish bilan chiziladi. Bu yerda: Girih (tashqi) + Islimiy (ichki).`,
7768 frm:[`f(ctx) = fA(ctx,R) ∘ [scale(0.5)·rot(π/n)] fB(ctx,R)`],
7769 algo:[`fA chaqiriladi`, `Kontekst 0.5 ga masshtablanadi, π/n ga buriladi`, `fB rang juftlari almashgan holda chaqir
iladi`],
7770 code:`function makeCombo(fA,fB){
7771 return function(ctx,R,p,sw,c0,c1,c2,c3){
7772 fA(ctx,R,p,sw,c0,c1,c2,c3);
7773 ctx.save(); ctx.scale(0.5,0.5); ctx.rotate(Math.PI/p.sym);
7774 fB(ctx,R,p,sw*1.7, c1,c0,c3,c2);
7775 ctx.restore(); }; },
7776 par:`ikkala uslubning parametrlari birga ishlaydi`,
7777 sg:{model:`makeCombo(Shamsa, Girih): tashqi quyosh medalyoni ichida 0.5 masshtabda girih yulduz-tugun.`,frm:[`f =
Shamsa ∘ scale(0.5)·Girih`],algo:[`Shamsa to'liq R da`, `Girih 0.5R da yarim qadam burilib`],code:`makeCombo(drawSh
amsa, drawGirih)`},par:`ikkala uslub parametrlari`,
7778 mi:{model:`makeCombo(Muqarnas, Islimiy): pog'onali arkada ichida spiral novdalar.`,frm:[`f = Muqarnas ∘ scale(0.5)
·Islimiy`],algo:[`Muqarnas darajalari`, `Islimiy spirallari markazda`],code:`makeCombo(drawMuqarnas, drawIslimiy)`},
par:`ikkala uslub parametrlari`,
7779 xr:{model:`makeCombo(Xatam, Rumi): xatam yulduzi ichida rumi barglari halqasi.`,frm:[`f = Xatam ∘ scale(0.5)·Rumi`
],algo:[`Xatam ramkasi`, `Rumi halqasi markazda`],code:`makeCombo(drawXatam, drawRumi)`},par:`ikkala uslub parametrl
ari`,
7780 celtic:{
7781 model:`Ikki tolali torus o'rmasi (plait): tolalar r(θ)=r_m + a·sin(kθ+φ) sinusoid halqalar, φ∈{0,π}; kesishishlar
sin=0 nuqtalarida; har tola 2k segmentga bo'linib, (tola+j) juftligi bo'yicha ost/ust tartibida chiziladi – matem
atik aniq navbatlashgan to'qish.`,
7782 frm:[`r_A(θ)=r_m+a·sin(kθ), r_B(θ)=r_m+a·sin(kθ+π)`,
7783 `Kesishishlar: 0_j = jπ/k, j = 0..2k-1`,
7784 `Ust/ost: (strand + j) mod 2`],
7785 algo:[`Har tola 0∈[jπ/k,(j+1)π/k] segmentlarga bo'linadi (smoothPath)`,
7786 `1-o'tish: juft paritetli segmentlar oddiy lenta (ostda)`,
7787 `2-o'tish: toq paritetli ribbonI (fon-uzilish) bilan ustida`,
7788 `Ichki/tashqi chegara bandlari`],

```

```

7789 code:`for(let pass=0; pass<2; pass++)
7790 for(let strand=0; strand<2; strand++)
7791   for(let j=0; j<2*k; j++){
7792     if(((strand+j)%2)!==pass) continue;
7793     strandSeg(strand*Math.PI, j);
7794     pass ? ribbonI(ctx,col,edge,bw) : ribbon(ctx,col,edge,bw); `},
7795 par:`<b>sym</b>->k (kesishishlar), <b>dens</b>->amplituda a va band eni`,
7796 mandala:{
7797   model:`Konsentrik halqalar generatori: halqa indeksi k bo'yicha element turi kind = k mod 5 davriy almashadi – nu
qtalar, gulbarglar, to'liqli kontur, barglar, doira.`;
7798   frm:`r_k = R(0.12 + 0.85*k/(K-1)); m_k = n(1 + 1.5k/K)`;
7799   `To'liqli: r(θ) = r_k(1 + 0.035*sin(mθ + k))`},
7800 algo:`K = 6..12 halqa (cmp+dens dan)`; `Har halqada kind bo'yicha element chiziladi`,`Ranglar 4 lik siklda`,
7801 code:`for(let k=0;k<rings;k++){
7802   const r=R*(0.12+0.85*k/(rings-1)), kind=k%5;
7803   if(kind===1) /* gulbarglar */ petalPath(...);
7804   else if(kind===2) /* to'liqli */ smoothPath(ctx, wavePts, true);
7805   ... `},
7806 par:`<b>sym</b>->bazaviy m, <b>cmp,dens</b>->halqalar soni, <b>spir</b>->barg o'lchami`,
7807 penrose:{
7808   model:`P2 aperiodik qoplama – Robinson uchburchaklari subdividesi: har "qizil" uchburchak 2 ga, har "ko'k" 3 ga b
o'linadi, bo'lish nuqtalari oltin nisbat φ bilan; boshlanish – 10 uchburchakli g'ildirak.`;
7809   frm:`φ = (1+√5)/2 ≈ 1.618; P = A + (B-A)/φ`;
7810   `Qizil → {(C,P,B), ko'k(P,C,A)}; Ko'k → {ko'k(R,C,A), qizil(Q,R,B), ko'k(R,Q,A)}`},
7811 algo:`10 ta yarim-uchburchak g'ildiragi (har ikkinchisi aks)`,
7812   `depth = cmp marta subdivide`,
7813   `Turi bo'yicha 2 rangda fill + ingichka kontur`,
7814 code:`for(const [ty,A,B,C] of tris){
7815   if(ty===0){ const P=div(A,B,PHI);
7816     next.push([0,C,P,B],[1,P,C,A]); }
7817   else { const Q=div(B,A,PHI), R2=div(B,C,PHI);
7818     next.push([1,R2,C,A],[0,Q,R2,B],[1,R2,Q,A]); } `},
7819 par:`<b>cmp</b>->rekursiya chuqurligi (uchburchaklar ~ 3.6^cmp)`},
7820 zellige:{
7821   model:`Marokash kafeli: romb panjara, har rombnning 4 tomoni navbatma-navbat ichkari/tashqariga bo'rtgan Bezier; 5
rang (g_x + 2g_y) mod 5 davriyligida.`;
7822   frm:`Markazlar: x=(g_x+g_y)·0.8s, y=(g_y-g_x)·0.5s`;
7823   `Bo'rtish: nazorat = o'rta + n·bow·(-1)^i, bow = 0.2s(0.4+spir)`},
7824 algo:`Romb panjara quriladi`,`curvedQuad: 4 Bezier tomon, ishorasi almashib`,`5 rangli fill + kontur`,
7825 code:`for(let i=0;i<4;i++){ const A=q[i], B=q[(i+1)%4];
7826   const sgn = i%2 ? 1 : -1;
7827   const cx2 = mx + nx/L*bow*sgn, cy2 = my + ny/L*bow*sgn;
7828   ctx.bezierCurveTo(A+0.7(c-A)..., B+0.7(c-B)..., B[0],B[1]); `},
7829 par:`<b>dens</b>->panjara zichligi, <b>spir</b>->bo'rtish, <b>cmp≥5</b>->markaz nuqtalari`,
7830 batik:{
7831   model:`To'liqli modulyatsiyali konsentrik halqalar: r(θ) = r_k + A·sin(nθ + φ_k); nuqtalar Catmull-Rom orqali kubi
k Bezierga aylantiriladi; oraliqlarda 5-nuqtali gul motivlari.`;
7832   frm:`r(θ) = r_k + A·sin(nθ + 1.3k + skew), A = 0.035R(0.6+dens)`},
7833 algo:`K = 3+cmp halqa, har biri ≥48 namunali smoothPath`,`Toq halqalar orasida gul-nuqtalar`,`Ranglar navbat`,
7834 code:`const pts=[];
7835 for(let i=0;i<steps;i++){ const a=TAU*i/steps;
7836   pts.push(pol(a, rk + A*Math.sin(n*a + k*1.3))); }
7837 ctx.beginPath(); smoothPath(ctx, pts, true); st(ctx,col,w);`,
7838 par:`<b>sym</b>->to'liqli chastotasi n, <b>cmp</b>->halqalar, <b>dens</b>->amplituda, <b>skew</b>->faza`,
7839 paisley:{
7840   model:`Boteh (buta): 4 kubik Bezierdan asimetrik, uchi ilmoqli tomchi; ichida log-spiral, ichki hoshiya konturi
(0.82 masshtab), uchida 5-nuqta gul, chetida nuqta bezaklar.`;
7841   frm:`Kontur: (0, .5L)→(-W,-.35L)→ilmoq(.38W,-.6L)→(.12W,-.28L)→yopiq`,
7842   `Ichki spiral: r=a·e^{(0.16+0.2spir)θ}`},
7843 algo:`n pozitsiyaga buriladi/siljiydi`,`Boteh fill + ribbonI + 0.82 masshtabli ichki kontur`,`Spiral, uch guli,
cmp≥4 da chet nuqtalari`,
7844 code:`ctx.moveTo(0, L*0.5);
7845 ctx.bezierCurveTo(-W, L*.5, -W, -L*.35, -W*.25, -L*.62);
7846 ctx.bezierCurveTo(-W*.05,-L*.75, W*.3,-L*.78, W*.38,-L*.6);
7847 ctx.bezierCurveTo(W*.44,-L*.46, W*.1,-L*.42, W*.12,-L*.28);
7848 ctx.bezierCurveTo(W*.55,-L*.05, W*.62, L*.32, 0, L*.5);`,
7849 par:`<b>sym</b>->butalar, <b>dens</b>->o'lcham, <b>spir</b>->ichki spiral, <b>cmp≥4</b>->chet nuqtalari`,
7850 greek:{
7851   model:`Meander (labirint): birlik "kalit" yo'li 8 nuqtali lokal koordinatalarda beriladi va affin almashtirish bi
lan har tomonga u marta takrorlanadi; ramkalar konsentrik.`;
7852   frm:`Birlik: [(0,0),(.72,0),(.72,.72),(.24,.72),(.24,.3),(.48,.3),(.48,.52),(1,.52)]`,
7853   `Nuqta = B + d·x + n·h·y (d – tomon yo'nalishi, n – ichki normal)`,
7854 algo:`K ramka, har birida 4 tomon`,`Har tomonda u birlik meander affin ko'chiriladi`,`Markazda 4-yulduz`,
7855 code:`for(const q of unit){
7856   ctx.lineTo(bx + dx*q[0] + hx*q[1],
7857     by + dy*q[0] + hy*q[1]); `},
7858 par:`<b>sym,dens</b>->birliklar soni, <b>cmp</b>->ramkalar, <b>ir</b>->markaz yulduzi`,
7859 xitoy:{
7860   model:`Fretwork panjara: kvadrat kataklar to'ri, har katakda T-motiv (gx+gy) mod 4 · 90° buralgan; burchak qavsla
r bilan boyitiladi.`;
7861   frm:`T: tik (0,-t)→(0,.2t), gorizontal (±b,.2t), oyoqlar (±b,.2t)→(±b,.62t)`},
7862 algo:`Tashqi ramka + panjara chiziqlari`,`Har katak markazida buralgan T`,`cmp≥4 da burchak qavslar`,
7863 code:`ctx.save(); ctx.translate(cx,cy);
7864 ctx.rotate(((gx+gy)%4)*Math.PI/2);
7865 ctx.moveTo(0,-tLen); ctx.lineTo(0,tLen*0.2);
7866 ctx.moveTo(0,tBar,tLen*0.2); ctx.lineTo(tBar,tLen*0.2); ...`,
7867 par:`<b>dens</b>->kataklar, <b>ir</b>->T o'lchami, <b>skew</b>->burilish, <b>cmp≥4</b>->qavslar`,
7868 koch:{
7869   model:`Koch qor parchasi – IFS/rekursiv almashtirish: har segment 4 ga bo'linadi, o'rtadagi ikkitasi 60° uchburch
ak yasaydi. Fraktal o'lcham D = log4/log3.`;
7870   frm:`P2 = P1 + Rot(-60°)·(B-A)/3; D = log 4 / log 3 ≈ 1.262`,

```

```

7871 algo:[`Teng tomonli uchburchakdan boshlanadi`,`depth marta: har segment → 4 segment`,`Yopiq kontur fill + lenta`]
7872 ,
7873 code:`const ang = Math.atan2(dy,dx) - Math.PI/3;
7874 const P2 = [P1[0]+Math.cos(ang)*L, P1[1]+Math.sin(ang)*L];
7875 next.push(A, P1, P2, P3);`,
7876 par:`<b>cmp</b>→rekursiya (segmentlar 3·4^cmp)`},
7877 sierpinski:{
7878 model:`Sierpinski uchburchagi: o'rta nuqtalar orqali 3 kichik uchburchakka rekursiv bo'linish; D = log3/log2`,`
7879 frm:[`mid(U,V) = (U+V)/2; D = log 3 / log 2 ≈ 1.585`],
7880 algo:[`depth=0 da uchburchak fill`,`Aks holda 3 burchak uchburchagiga rekursiya`],
7881 code:`const rec=(a,b,c,d)=>{
7882   if(d===0){ polyPath(ctx,[a,b,c]); fillA(...); return; }
7883   rec(a, mid(a,b), mid(a,c), d-1);
7884   rec(mid(a,b), b, mid(b,c), d-1);
7885   rec(mid(a,c), mid(b,c), c, d-1); }`,`
7886 par:`<b>cmp</b>→chuqurlik (3^cmp uchburchak)`},
7887 dragon:{
7888 model:`Heighway ajdar egri – L-sistema: F→F+G, G→F-G, burilish ±90°; satr iteratsiyadan keyin chelakdek yig'ilib,
7889 chegara qutisiga masshtablanadi`,`
7890 frm:[`Uzunlik: 2^N segment; burchak ±π/2`,`
7891   `Fit: masshtab = 1.7R / max(Δx, Δy)`],
7892 algo:[`Satr N marta qayta yoziladi`,`Toshbaqa-yurish: F/G oldinga, +/- burilish`,`Bounding box → markazlash va ma
7893 sshtab`],
7894 code:`for(const ch of seq){
7895   if(ch==='F' || ch==='G'){ x+=Math.cos(ang); y+=Math.sin(ang);
7896   pts.push([x,y]); }
7897   else if(ch==='+') ang += Math.PI/2;
7898   else if(ch==='-') ang -= Math.PI/2; }`,`
7899 par:`<b>cmp</b>→iteratsiyalar (8+cmp)`},
7900 hilbert:{
7901 model:`Hilbert fazoni to'ldiruvchi egri: d-indeksdan (x,y) ga Gray-kod asosidagi iterativ almashtirish (kvadrant
7902 aniqlash + aks/burish); 4^order nuqta ketma-ket ulanadi`,`
7903 frm:[`N = 2^order; d ∈ [0, N^2)`],
7904 `Har bosqichda: r_x=1&(t/2), r_y=1&(t&tor_x); r_y=0 bo'lsa aks+almashtirish`,`
7905 algo:[`d = 0..N^2-1 uchun d2xy hisoblanadi`,`Nuqtalar polyline bilan ulanadi`,`Markazlash va masshtab`],
7906 code:`const d2xy=(d)=>{ let rx,ry,t=d,x=0,y=0;
7907   for(let s=1;s<N;s*=2){
7908     rx=1&(t/2); ry=1&(t^rx);
7909     if(ry===0){ if(rx===1){x=s-1-x;y=s-1-y;}
7910     const tmp=x; x=y; y=tmp; }
7911     x+=s*rx; y+=s*ry; t=Math.floor(t/4); }
7912   return [x,y]; }`,`
7913 par:`<b>cmp</b>→order (nuqtalar 4^order)`},
7914 spirograph:{
7915 model:`Epitrochoid – g'ildirak ichida g'ildirak: R_0 radiusli halqa ichida r g'ildirak dumalaydi, qalam d masofada
7916 ; kNum/kDen nisbat necha aylanishda yopilishini belgilaydi`,`
7917 frm:[`x = (R_0-r)cos t + d·cos((R_0-r)t/r)`],
7918 `y = (R_0-r)sin t - d·sin((R_0-r)t/r)`],
7919 `r = R_0·kNum/kDen; yopilish: t ∈ [0, 2π·kDen)`],
7920 algo:[`t bo'ylab zich namunalar`,`Max radiusga qarab masshtab`,`smoothPath (Catmull-Bezier) bilan silliq chizilad
7921 i`],
7922 code:`for(let i=0;i<=steps;i++){
7923   const t=TAU*loops*i/steps;
7924   pts.push([(R0-r)*Math.cos(t)+d*Math.cos((R0-r)*t/r),
7925     (R0-r)*Math.sin(t)-d*Math.sin((R0-r)*t/r)]); }`,`
7926 par:`<b>sym,cmp</b>→kNum/kDen nisbati, <b>spir</b>→d (qalam masofasi)`},
7927 lissajous:{
7928 model:`Lissaju figurasi: ikki perpendikulyar garmonik tebranish superpozitsiyasi; a:b chastota nisbati figura tur
7929 ini, δ faza siljishi shaklini belgilaydi`,`
7930 frm:[`x = A·sin(a·t + δ), y = B·sin(b·t)`],
7931 `δ = (π/2)(1 + skew)`],
7932 algo:[`400 namunali yopiq smoothPath`,`dens>0.5 da 0.7 masshtabli ikkinchi kontur`],
7933 code:`for(let i=0;i<=steps;i++){ const t=TAU*i/steps;
7934   pts.push([R*Math.sin(a*t+del), R*Math.sin(b*t)]); }
7935 smoothPath(ctx, pts, true);`,`
7936 par:`<b>sym</b>→a, <b>cmp</b>→b-a, <b>skew</b>→δ, <b>dens</b>→ikkinchi kontur`},
7937 voronoi:{
7938 model:`Voronoi diagrammasi yarim-tekislik kesish usulida: har seed hujayrasi boshlang'ich 16-burchakdan boshlab,
7939 boshqa har bir seed bilan o'rta perpendikulyar bo'ylab Sutherland-Hodgman kesish orqali olinadi. Seedlar determini
7940 stik PRNG (mulberry32) dan – bir xil parametrlar = bir xil mozaika`,`
7941 frm:[`Hujayra(S) = n_T { q : (q-M)·(T-S) ≤ 0 }, M=(S+T)/2`,`
7942   `PRNG urug'i: 1000 + 31·sym + 7·cmp + 13·[100·dens] + ...`],
7943 algo:[`m = 8+4cmp+14dens seed doira ichida (√ taqsimot)`],
7944 `Har seed uchun poligon barcha bisektorlar bilan kesiladi`,`
7945 `5 palitra rangida fill + kontur`],
7946 code:`function clipHalfPlane(poly, px,py, nx,ny){
7947   const out=[];
7948   for(let i=0;i<poly.length;i++){
7949     const A=poly[i], B=poly[(i+1)%poly.length];
7950     const da=(A[0]-px)*nx+(A[1]-py)*ny;
7951     const db=(B[0]-px)*nx+(B[1]-py)*ny;
7952     if(da<=0) out.push(A);
7953     if(da*db<0){ const t=da/(da-db);
7954       out.push([A[0]+(B[0]-A[0])*t, A[1]+(B[1]-A[1])*t]); } }
7955   return out; }`,`
7956 par:`<b>sym,cmp,dens,spir</b>→PRNG urug'i va seedlar soni`
7957 };
7958 function buildTheory(){
7959   const wrap = $('theory-wrap');
7960   if(!wrap) return;
7961   let html = '<h1>Naqsh matematikasi va algoritmlari</h1>' +

```

```

7953     <p class="lead">Har bir naqshni oching: qaysi <b>model</b>, qaysi <b>formulalar</b> va qanday <b>kod</b> uni
        hosil qilayotganini jonli preview bilan birga ko'rasiz. Barcha kod parchalari platformaning haqiqiy manbasidan ol
        ingan.</p>;
7954     /* ===== Umumiy qurilish bloklari ===== */
7955     html += '<div class="th-card"><h3>Umumiy qurilish bloklari – barcha naqshlar shulardan quriladi</h3>' +
7956     '<div class="th-item"><b>Qutb koordinatalari:</b> <span class="formula">pol(θ, r) = (r·cos θ, r·sin θ)</span>
        – n-karra simmetriya θi = 2πi/n burchaklarida takrorlash orqali olinadi.</div>' +
7957     '<div class="th-item"><b>Kubik Bezier:</b> <span class="formula">B(t) = (1-t)3P0 + 3(1-t)2tP1 + 3(1-t)t2P2 + t
        3P3</span> – barcha organik egri chiziqlarning yagona asosi; hatto doiralalar ham 4 Bezier segment (k = 0.5523·r naz
        orat masofasi) bilan chiziladi, shuning uchun SVG eksportda faqat C buyruqlari chiqadi.</div>' +
7958     '<div class="th-item"><b>Catmull-Rom → Bezier:</b> nuqtalar orqali silliq egri: nazorat nuqtalari <span class=
        "formula">C1 = P1 + (P2-P0)/6, C2 = P2 - (P3-P1)/6</span> – spiral, to'liqin va epitrochoidlarda ishlatiladi (smoo
        thPath).</div>' +
7959     '<div class="th-item"><b>Lenta (ribbon):</b> bitta yo'll 3 marta chiziladi – qora hoshiya (w), band rangi (w-m
        ax(1.8, 0.42w)), glazur yaltiroq'i (0.14w) – haqiqiy strapwork ko'rinishi.</div>' +
7960     '<div class="th-item"><b>Interlacing (to'qish):</b> ribbonI avval fon rangida kengroq (w+0.55w) chizadi – ost
        idagi lenta kesishuvda "uziladi", ustidan o'tish effekti tug'iladi. Chizish tartibi (2 o'tish, paritet bo'yich
        a) navbatlashgan weave beradi.</div>' +
7961     '<div class="th-item"><b>Nur kesishmasi:</b> <span class="formula">rayX: P+t·d^ = Q+u·ê → t = ((Q-P)×ê)/(d^×ê)
        </span> – g'irih va Hankin straplarining yuragi.</div>' +
7962     '<div class="th-item"><b>Qatlamlash:</b> qatlamlar ichma-ich medalyonlar: masshtab 0.42°l, burilish (l mod 2)·
        π/n – yarim qadam qatlamlarni interlock qiladi, ustma-ust loyqa bo'lmaydi.</div>' +
7963     '<div class="th-item"><b>Tessellatsiya:</b> <span class="formula">T(n,m) = n·v1 + m·v2</span>; p4: v1=(c,0), v
        2=(0,c); p6: v2=(c/2, c√3/2); p4m/p6m – shaxmat ko'zgusi. Panjara-naqshlar (Hankin, Katak-8, Zellige...) uzluksiz y
        axlit maydon sifatida, medalyonlar bleed (R tile = 0.62c &gt; c/2) bilan chiziladi.</div>' +
7964     '<div class="th-item"><b>SVG/DXF eksport:</b> soxta kontekst (SvgCtx) har bezierCurveTo ni 2×3 affin matritsa
        steki orqali kuzatib SVG C buyrug'iga yozadi; arc() 4/3·tan(Δθ/4) formulasi bilan Bezierga aylanadi; DXF uchun Be
        zier 32 nuqtali LWPOLYLINE ga tabaqalanadi.</div>' +
7965     '</div>;
7966     Object.keys(CATS).forEach(cat=>{
7967         html += '<h2 class="th-cat">' + CATS[cat] + '</h2>';
7968         PATTERNS.filter(pt=>pt.cat===cat).forEach(pt=>{
7969             const T = THEORY[pt.id] || {};
7970             const modelTxt = T.model || pt.th;
7971             html += '<details class="th-pat" data-id="'+pt.id+'"><summary>' +
7972                 '<canvas data-pt="'+pt.id+'" width="200" height="200"></canvas>' +
7973                 '<span class="th-head"><b>'+pt.nm+'</b><em>'+modelTxt+'</em></span>' +
7974                 '<span class="th-open"></span></summary><div class="th-body">';
7975             if(T.frm && T.frm.length){
7976                 html += '<h5>Matematik formulalar</h5>';
7977                 T.frm.forEach(f=>{ html += '<div class="frm">'+escHTML(f)+'</div>'; });
7978             }
7979             if(T.algo && T.algo.length){
7980                 html += '<h5>Algoritmlar</h5><ol class="algo">';
7981                 T.algo.forEach(a=>{ html += '<li>'+a+'</li>'; });
7982                 html += '</ol>';
7983             }
7984             if(T.code){
7985                 html += '<h5>Kod (platforma manbasidan)</h5><pre class="code">'+escHTML(T.code)+'</pre>';
7986             }
7987             if(T.par){
7988                 html += '<div class="th-par"><b>Slayderlar:</b> '+T.par+'</div>';
7989             }
7990             html += '<button class="btn sm gold th-studio" data-id="'+pt.id+'" style="margin-top:12px">+ Studio da sinab
        ko'rish</button>';
7991             html += '</div></details>';
7992         });
7993     });
7994     wrap.innerHTML = html;
7995     renderTheoryCanvases(wrap);
7996     wrap.addEventListener('click', e=>{
7997         const b = e.target && e.target.closest ? e.target.closest('.th-studio') : null;
7998         if(!b) return;
7999         P.type = b.dataset.id;
8000         syncSliders();
8001         redraw(true);
8002         showPage('studio');
8003         ntf(getPattern(P.type).nm + ' – Studio da ochildi', 'ok');
8004     });
8005 }
8006 function renderTheoryCanvases(wrap){
8007     const list = wrap.querySelectorAll ? wrap.querySelectorAll('canvas[data-pt]') : [];
8008     const keepBg = P._bg;
8009     Array.prototype.forEach.call(list, cnv=>{
8010         const octx = cnv.getContext ? cnv.getContext('2d') : null;
8011         if(!octx) return;
8012         const id = cnv.dataset ? cnv.dataset.pt : null;
8013         const pal = PAL.gold;
8014         P._bg = pal.bg;
8015         octx.fillStyle = pal.bg;
8016         octx.fillRect(0,0,200,200);
8017         try{
8018             drawPattern(octx, 100, 100, 92, {
8019                 type:id, sym:8, cmp:4, sw:1.7, ir:0.38, spir:0.4,
8020                 dens:0.55, lay:1, skew:0, rot:0, pal:'gold', fill:'dual'
8021             });
8022         }catch(e){}
8023     });
8024     P._bg = keepBg;
8025 }
8026

```

```

8027  /* =====
8028  NAVIGATSIYA + INIT
8029  ===== */
8030  function showPage(name){
8031    document.querySelectorAll('.page').forEach(pg=>{
8032      pg.classList.toggle('active', pg.id === 'page-'+name);
8033    });
8034    document.querySelectorAll('nav.tabs button').forEach(b=>{
8035      b.classList.toggle('active', b.dataset.page === name);
8036    });
8037    if(name !== 'ar' && arActive) stopAR(); /* sahifadan chiqilganda kamera o'chadi */
8038    if(name === '3d'){ init3D(); if(threeReady) update3DTexture(); }
8039    if(name === 'studio') setTimeout(resizeCanvas, 30);
8040  }
8041  function wireUI(){
8042    const tabs = $('tabs');
8043    if(tabs) tabs.addEventListener('click', e=>{
8044      const b = e.target.closest('button');
8045      if(b && b.dataset.page) showPage(b.dataset.page);
8046    });
8047    /* accordion */
8048    document.querySelectorAll('.acc > .acc-h').forEach(h=>{
8049      h.addEventListener('click', ()=>{
8050        const acc = h.parentNode;
8051        if(acc) acc.classList.toggle('open');
8052      });
8053    });
8054    /* mobil sheet */
8055    const sideT = $('side-toggle'); side = $('side');
8056    if(sideT && side) sideT.addEventListener('click', ()=> side.classList.toggle('open'));
8057    /* studio boshqaruvlari */
8058    const rndB = $('btn-random'); if(rndB) rndB.addEventListener('click', randomize);
8059    const saveB = $('btn-save'); if(saveB) saveB.addEventListener('click', saveCurrent);
8060    const fillSel = $('sel-fill'); if(fillSel) fillSel.addEventListener('change', ()=>{ P.fill = fillSel.value; PIC_
8061    TILE_CACHE = {}; redraw(); });
8062    const tessChk = $('chk-tess'); if(tessChk) tessChk.addEventListener('change', ()=>{ TESS.on = tessChk.checked; r
8063    edraw(); });
8064    const tessSel = $('sel-tess'); if(tessSel) tessSel.addEventListener('change', ()=>{ TESS.group = tessSel.value;
8065    if(TESS.on) redraw(); });
8066    const cellSl = $('sl-cell'); if(cellSl) cellSl.addEventListener('input', ()=>{
8067      TESS.cell = parseInt(cellSl.value,10);
8068      const v = $('v-cell'); if(v) v.textContent = TESS.cell+'px';
8069      if(TESS.on) redraw();
8070    });
8071    const cellB = $('chk-cellb'); if(cellB) cellB.addEventListener('change', ()=>{ TESS.showCell = cellB.checked; if
8072    (TESS.on) redraw(); });
8073    const bleedB = $('chk-bleed'); if(bleedB) bleedB.addEventListener('change', ()=>{ TESS.bleed = bleedB.checked; i
8074    f(TESS.on) redraw(); });
8075    const nrB = $('chk-noring'); if(nrB) nrB.addEventListener('change', ()=>{ TESS.noring = nrB.checked; if(TESS.on)
8076    redraw(); });
8077    const g3 = $('sel-3dgrid'); if(g3) g3.addEventListener('change', ()=>{
8078      if(g3.value === 'off'){ T3D.tess = false; }
8079      else { T3D.tess = true; T3D.group = g3.value; }
8080      if(threeReady) update3DTexture();
8081    });
8082    /* eksport modal */
8083    const expB = $('btn-export'); if(expB) expB.addEventListener('click', openExport);
8084    const expC = $('exp-cancel'); if(expC) expC.addEventListener('click', closeExport);
8085    const expG = $('exp-go'); if(expG) expG.addEventListener('click', doExport);
8086    const back = $('modal-export'); if(back) back.addEventListener('click', e=>{ if(e.target===back) closeExport();
8087    });
8088    segWire('exp-format','format',false);
8089    segWire('exp-format2','format',false);
8090    segWire('exp-size','size',true);
8091    segWire('exp-dpi','dpi',true);
8092    segWire('exp-jqsize','jqsize',true);
8093    segWire('exp-jqgroup','jqgroup',false);
8094    segWire('exp-jqdepth','jqdepth',true);
8095    segWire('exp-mm','mm',true);
8096    segWire('exp-fill','fill',false);
8097    /* Tayyor o'lchamni bosganda raqamli maydonlar ham yangilanadi –
8098    shunda foydalanuvchi qaysi qiymat ketayotganini har doim ko'radi. */
8099    const linkSeg = (segId, aId, bId)>{
8100      const box = $(segId); if(!box) return;
8101      box.addEventListener('click', e=>{
8102        const b = e.target.closest('button'); if(!b) return;
8103        const v = b.dataset.v;
8104        const a = $(aId), c = $(bId);
8105        if(a){ a.value = v; delete a.dataset.touched; }
8106        if(c){ c.value = v; delete c.dataset.touched; }
8107      });
8108    };
8109    linkSeg('exp-mm', 'exp-mm-w', 'exp-mm-h');
8110    linkSeg('exp-size', 'exp-px-w', 'exp-px-h');
8111    linkSeg('exp-jqsize', 'exp-jq-c', null);
8112    /* Raqam qo'lda o'zgartirilsa – tayyor tugmalardan tanlovni olib tashlaymiz,
8113    chunki endi qiymat ularnikiga mos kelmasligi mumkin. */
8114    const unsel = (inputIds, segId)>{
8115      for(const id of inputIds){
8116        const el = $(id); if(!el) continue;
8117        el.addEventListener('input', ()=>{

```

```

8111     el.dataset.touched = '1';
8112     const box = $(segId); if(!box) return;
8113     const a = $(inputIds[0]), b = inputIds[1] ? $(inputIds[1]) : null;
8114     const same = a && (!b || a.value === b.value) ? a.value : null;
8115     box.querySelectorAll('button').forEach(x=>{
8116       x.classList.toggle('sel', same !== null && x.dataset.v === same);
8117     });
8118     if(segId === 'exp-jqsize'){
8119       const v = parseInt(el.value, 10);
8120       if(isFinite(v)){ EXP.jqsize = Math.min(4096, Math.max(32, v)); expJqInfo(); }
8121     }
8122   });
8123 }
8124 };
8125 unsel(['exp-mm-w', 'exp-mm-h'], 'exp-mm');
8126 unsel(['exp-px-w', 'exp-px-h'], 'exp-size');
8127 unsel(['exp-jq-c', 'exp-jqsize');
8128 /* 3D */
8129 const objSel = $('sel-obj'); if(objSel) objSel.addEventListener('change', ()=>{ if(threeReady) setObject(objSel.
value); });
8130 const lightSel = $('sel-light'); if(lightSel) lightSel.addEventListener('change', ()=>{ if(threeReady) setLights
(lightSel.value); });
8131 const rotB = $('btn-rotate'); if(rotB) rotB.addEventListener('click', ()=>{
8132   if(!threeReady){ ntf('3D hali yuklanmadi', 'err'); return; }
8133   T3.auto = !T3.auto;
8134   rotB.textContent = T3.auto ? '□ To\'xtatish' : '□ Avto-aylanish';
8135 });
8136 const shotB = $('btn-shot'); if(shotB) shotB.addEventListener('click', screenshot3D);
8137 const paintB = $('btn-paint'); if(paintB) paintB.addEventListener('click', animatePaint3D);
8138 const photoB = $('btn-photo'); photoF = $('photo-file');
8139 if(photoB && photoF) photoB.addEventListener('click', ()=> photoF.click());
8140 if(photoF) photoF.addEventListener('change', ()=>{
8141   const f = photoF.files && photoF.files[0];
8142   if(!f) return;
8143   const reader = new FileReader();
8144   reader.onload = ()=>{
8145     const img = new Image();
8146     img.onload = ()=>{
8147       PH.img = img;
8148       const panel = $('photo-panel'); if(panel) panel.hidden = false;
8149       if(threeReady) update3DTexture();
8150       ntf('Rasm ramkaga joylandi □', 'ok');
8151     };
8152     img.onerror = ()=> ntf('Rasm ochilmadi', 'err');
8153     img.src = reader.result;
8154   };
8155   reader.onerror = ()=> ntf('Fayl o\'qilmadi', 'err');
8156   reader.readAsDataURL(f);
8157   photoF.value = '';
8158 });
8159 const frameSel = $('sel-frame'); if(frameSel) frameSel.addEventListener('change', ()=>{ PH.shape = frameSel.valu
e; if(threeReady) update3DTexture(); });
8160 const psize = $('sl-psize'); if(psize) psize.addEventListener('input', ()=>{
8161   PH.size = parseInt(psize.value, 10);
8162   const v = $('v-psize'); if(v) v.textContent = PH.size+'%';
8163   if(threeReady) update3DTexture();
8164 });
8165 const pframe = $('sl-pframe'); if(pframe) pframe.addEventListener('input', ()=>{
8166   PH.frame = parseInt(pframe.value, 10);
8167   const v = $('v-pframe'); if(v) v.textContent = PH.frame;
8168   if(threeReady) update3DTexture();
8169 });
8170 const ptext = $('inp-ptext'); if(ptext) ptext.addEventListener('input', ()=>{
8171   PH.text = ptext.value;
8172   if(threeReady) update3DTexture();
8173 });
8174 const pdate = $('inp-pdate'); if(pdate) pdate.addEventListener('input', ()=>{
8175   PH.date = pdate.value;
8176   if(threeReady) update3DTexture();
8177 });
8178 const photoDel = $('btn-photo-del'); if(photoDel) photoDel.addEventListener('click', ()=>{
8179   PH.img = null;
8180   const panel = $('photo-panel'); if(panel) panel.hidden = true;
8181   if(threeReady) update3DTexture();
8182   ntf('Rasm o\'chirildi', 'ok');
8183 });
8184 /* AR */
8185 const arStart = $('btn-ar-start'); if(arStart) arStart.addEventListener('click', startAR);
8186 const arSim = $('btn-ar-sim'); if(arSim) arSim.addEventListener('click', startARSim);
8187 const arStop = $('btn-ar-stop'); if(arStop) arStop.addEventListener('click', stopAR);
8188 const arCap = $('btn-ar-capture'); if(arCap) arCap.addEventListener('click', captureAR);
8189 const arSurf = $('ar-surface'); if(arSurf) arSurf.addEventListener('change', ()=>{ AR.surface = arSurf.value; })
;
8190 const arOp = $('ar-op'); if(arOp) arOp.addEventListener('input', ()=>{ AR.op = parseInt(arOp.value, 10)/100; });
8191 const arSize = $('ar-size'); if(arSize) arSize.addEventListener('input', ()=>{ AR.size = parseInt(arSize.value, 1
0)/100; });
8192 const arRot = $('ar-rot'); if(arRot) arRot.addEventListener('input', ()=>{ AR.rot = parseInt(arRot.value, 10); })
;
8193 const overlay = $('ar-overlay');
8194 if(overlay){
8195   let dragging = false, lx = 0, ly = 0;

```

```

8196     overlay.addEventListener('pointerdown', e=>{ dragging = true; lx = e.clientX; ly = e.clientY; overlay.setPoint
erCapture && overlay.setPointerCapture(e.pointerId); });
8197     overlay.addEventListener('pointermove', e=>{
8198         if(!dragging) return;
8199         AR.x += e.clientX - lx; AR.y += e.clientY - ly;
8200         lx = e.clientX; ly = e.clientY;
8201     });
8202     overlay.addEventListener('pointerup', ()=>{ dragging = false; });
8203     overlay.addEventListener('pointercancel', ()=>{ dragging = false; });
8204 }
8205 window.addEventListener('pagehide', stopAR);
8206 /* Chat */
8207 const sendB = $('#btn-chat-send'); if(sendB) sendB.addEventListener('click', sendChat);
8208 const inp = $('#chat-input'); if(inp) inp.addEventListener('keydown', e=>{ if(e.key==='Enter') sendChat(); });
8209 window.addEventListener('resize', ()=>{
8210     const pg = $('#page-studio');
8211     if(pg && pg.classList.contains('active')) resizeCanvas();
8212 });
8213 }
8214 /* =====
8215     AUTENTIFIKATSIYA — ikki rejimli
8216     =====
8217     1) SUPABASE rejimi (production): haqiqiy PostgreSQL database,
8218     bcrypt parol xeshi (Supabase auth.users ichida), Google OAuth,
8219     rollar, status, parol tiklash, email tasdiqlash, audit log.
8220     Yoqish: quyidagi SUPABASE_URL/SUPABASE_ANON_KEY ni to'ldiring
8221     yoki ilovadan "Server sozlash" orqali kiriting.
8222     2) LOKAL rejim (zaxira): server sozlanmaganda ilova baribir
8223     ishlaydi — hisoblar shu qurilmada localStorage'da qoladi.
8224     Demo/oflayn uchun. Bunda admin panel va parol tiklash yo'q.
8225
8226     anon key MAXFIY EMAS — u brauzerga chiqarilishi Supabase
8227     arxitekturasida normal; haqiqiy himoya Row Level Security (RLS)
8228     orqali serverda amalga oshiriladi (supabase/schema.sql ga qarang).
8229     service_role kalitini esa BU YERGA HECH QACHON yozmang.
8230     ===== */
8231 const SUPABASE_URL = ''; /* masalan: https://abcdefgh.supabase.co */
8232 const SUPABASE_ANON_KEY = ''; /* "anon public" kaliti */
8233
8234 const AUTH = { mode:'local', client:null, user:null, profile:null, ready:false };
8235
8236 function sbCfg(){
8237     let url = SUPABASE_URL, key = SUPABASE_ANON_KEY;
8238     try{
8239         url = (localStorage.getItem('naqsh_sb_url') || url || '').trim();
8240         key = (localStorage.getItem('naqsh_sb_key') || key || '').trim();
8241     }catch(e){}
8242     return { url, key };
8243 }
8244 function sbCfgSet(url, key){
8245     try{
8246         localStorage.setItem('naqsh_sb_url', (url||'').trim());
8247         localStorage.setItem('naqsh_sb_key', (key||'').trim());
8248         return true;
8249     }catch(e){ return false; }
8250 }
8251 function sbEnabled(){ return AUTH.mode === 'supabase' && !!AUTH.client; }
8252
8253 /* Texnik xatoni foydalanuvchiga tushunarli o'zbekcha xabarga aylantiradi.
8254     Batafsil xato faqat konsolga (developer uchun) yoziladi. */
8255 function authErrMsg(err){
8256     const raw = (err && (err.message || err.error_description || '')) || '';
8257     console.error('[auth]', err);
8258     const m = raw.toLowerCase();
8259     if(m.includes('invalid login credentials')) return 'Email yoki parol noto\'g\'ri';
8260     if(m.includes('email not confirmed')) return 'Email tasdiqlanmagan — pochtagizdagi havolani bosing';
8261     if(m.includes('user already registered') || m.includes('already been registered')) return 'Bu email allaqachon ro\'yxatdan o\'tgan';
8262     if(m.includes('password should be')) return 'Parol kamida 6 belgi bo\'lishi kerak';
8263     if(m.includes('rate limit') || m.includes('too many')) return 'Juda ko\'p urinish — bir necha daqiqadan so\'ng qayta urinin
g';
8264     if(m.includes('failed to fetch') || m.includes('networkerror')) return 'Serverga ulanib bo\'lmadi — internetni tekshiring';
8265     return 'Xatolik yuz berdi — birozdan so\'ng qayta urining';
8266 }
8267
8268 }
8269
8270 async function authLogEvent(event, meta){
8271     if(!sbEnabled()) return;
8272     try{
8273         await AUTH.client.from('auth_events').insert({
8274             user_id: AUTH.user ? AUTH.user.id : null,
8275             email: AUTH.user ? AUTH.user.email : null,
8276             event, meta: meta || null
8277         });
8278     }catch(e){ console.error('[auth] log', e); }
8279 }
8280
8281
8282 async function authLoadProfile(){
8283     if(!sbEnabled() || !AUTH.user){ AUTH.profile = null; return null; }
8284     try{

```

```

8285     const { data, error } = await AUTH.client.from('profiles').select('*').eq('id', AUTH.user.id).single();
8286     if(error) throw error;
8287     AUTH.profile = data;
8288     return data;
8289   }catch(e){ console.error('[auth] profile', e); AUTH.profile = null; return null; }
8290 }
8291
8292 /* Bloklangan hisob kira olmasligi kerak. Supabase auth.users status'ni
8293    bilmaydi, shuning uchun kirgandan keyin profil status'ini tekshiramiz. */
8294 async function authEnforceStatus(){
8295   if(!sbEnabled() || !AUTH.profile) return true;
8296   const st = AUTH.profile.status;
8297   if(st === 'blocked' || st === 'deactivated'){
8298     const msg = st === 'blocked' ? 'Hisob bloklangan – administrator bilan bog\'laning'
8299       : 'Hisob faolsizlantirilgan';
8300     await AUTH.client.auth.signOut();
8301     AUTH.user = null; AUTH.profile = null;
8302     ntf(msg, 'err');
8303     return false;
8304   }
8305   return true;
8306 }
8307
8308 async function authBootstrap(){
8309   const { url, key } = sbCfg();
8310   if(url && key && typeof supabase !== 'undefined' && supabase.createClient){
8311     try{
8312       AUTH.client = supabase.createClient(url, key, {
8313         auth:{ persistSession:true, autoRefreshToken:true, detectSessionInUrl:true }
8314       });
8315       AUTH.mode = 'supabase';
8316       const { data } = await AUTH.client.auth.getSession();
8317       AUTH.user = data && data.session ? data.session.user : null;
8318       if(AUTH.user){ await authLoadProfile(); await authEnforceStatus(); await cloudLoadPatterns(); }
8319       AUTH.client.auth.onAuthStateChange(async (evt, session)=>{
8320         const wasUser = AUTH.user;
8321         AUTH.user = session ? session.user : null;
8322         if(AUTH.user){
8323           await authLoadProfile();
8324           /* yangi kirish bo'lsa – mehmon naqshlarini ko'chiramiz, so'ng yuklaymiz */
8325           if(!wasUser){
8326             const moved = await cloudMigrateGuest();
8327             if(moved) ntf(moved + ' ta naqsh hisobingizga ko\'chirildi 🔄', 'ok');
8328           }
8329           await cloudLoadPatterns();
8330         } else {
8331           AUTH.profile = null; CLOUD.patterns = null;
8332         }
8333         updateUserBtn(); renderGallery(); renderProfileCard();
8334         if(evt === 'PASSWORD_RECOVERY') openNewPassword();
8335       });
8336     }catch(e){
8337       console.error('[auth] init', e);
8338       AUTH.mode = 'local'; AUTH.client = null;
8339     }
8340   }
8341   AUTH.ready = true;
8342   updateUserBtn();
8343 }
8344
8345 /* ----- LOKAL rejim (zaxira) ----- */
8346 function usersGet(){ return lsGet('naqsh_users', {}); }
8347 function usersSet(u){ return lsSet('naqsh_users', u); }
8348 function sessionGet(){ return lsGet('naqsh_session', null); }
8349 function sessionSet(v){
8350   try{
8351     if(v===null) localStorage.removeItem('naqsh_session');
8352     else localStorage.setItem('naqsh_session', JSON.stringify(v));
8353   }catch(e){}
8354 }
8355 async function hashPass(pass, salt){
8356   try{
8357     const data = new TextEncoder().encode(salt + '::naqshai::' + pass);
8358     const buf = await crypto.subtle.digest('SHA-256', data);
8359     return Array.from(new Uint8Array(buf)).map(b=>b.toString(16).padStart(2,'0')).join('');
8360   }catch(e){
8361     return 'f' + hashStr(salt + '::naqshai::' + pass).toString(16); /* zaxira yo'l */
8362   }
8363 }
8364 function randSalt(){
8365   let s = '';
8366   for(let i=0;i<16;i++) s += Math.floor(Math.random()*36).toString(36);
8367   return s + Date.now().toString(36);
8368 }
8369
8370 /* ----- Umumiy API (UI shu nomlarni chaqiradi) ----- */
8371 function currentUser(){
8372   if(!sbEnabled()){
8373     if(!AUTH.user) return null;
8374     const p = AUTH.profile || {};
8375     const nm = [p.first_name, p.last_name].filter(Boolean).join(' ');

```

```

8376         || (AUTH.user.email||'').split('@')[0];
8377     return {
8378         login: AUTH.user.email, name: nm,
8379         created: p.created_at ? Date.parse(p.created_at) : Date.now(),
8380         role: p.role || 'user', status: p.status || 'active',
8381         provider: p.provider || 'email', verified: !!p.email_verified
8382     };
8383 }
8384 const s = sessionGet();
8385 if(!s) return null;
8386 const u = usersGet();
8387 return u[s] ? {login:s, name:u[s].name, created:u[s].created, role:'user', status:'active', provider:'email', ve
8388 rified:true} : null;
8389 }
8390 function isAdmin(){
8391     const u = currentUser();
8392     return !!u && (u.role === 'admin' || u.role === 'super_admin');
8393 }
8394 async function registerUser(name, login, pass, pass2, lastName){
8395     name = (name||'').trim(); login = (login||'').trim().toLowerCase();
8396     lastName = (lastName||'').trim();
8397     if(!name || !login){ ntf('Ism va email kiritilishi shart', 'err'); return false; }
8398     if(!/^[^s@]+@[^\s@]+\.[^\s@]{2,}$/.test(login)){ ntf('Email formati noto\'g\'ri', 'err'); return false; }
8399     if((pass||'').length < 6){ ntf('Parol kamida 6 belgi', 'err'); return false; }
8400     if(pass !== pass2){ ntf('Parollar mos emas', 'err'); return false; }
8401 }
8402 if(sbEnabled()){
8403     try{
8404         const { data, error } = await AUTH.client.auth.signUp({
8405             email: login, password: pass,
8406             options:{ data:{ first_name:name, last_name:lastName }, emailRedirectTo: location.href.split('#')[0] }
8407         });
8408         if(error){ ntf(authErrMsg(error), 'err'); return false; }
8409         if(data && data.user && !data.session){
8410             ntf('Email manzilingizni tasdiqlang – pochtagizga havola yuborildi ☐', 'ok');
8411             return true;
8412         }
8413         AUTH.user = data.session ? data.session.user : null;
8414         await authLoadProfile();
8415         return true;
8416     }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8417 }
8418 }
8419 const users = usersGet();
8420 if(users[login]){ ntf('Bu email allaqachon ro\'yxatdan o\'tgan', 'err'); return false; }
8421 const salt = randSalt();
8422 const hash = await hashPass(pass, salt);
8423 const guest = lsGet('naqsh_saved', []);
8424 const full = [name, lastName].filter(Boolean).join(' ');
8425 users[login] = { name: full, salt, hash, created:Date.now(), patterns:guest.slice(0,48) };
8426 if(!usersSet(users)){ ntf('Saqlashda xato (localStorage)', 'err'); return false; }
8427 sessionSet(login);
8428 return true;
8429 }
8430 }
8431 async function loginUser(login, pass){
8432     login = (login||'').trim().toLowerCase();
8433     if(sbEnabled()){
8434         try{
8435             const { data, error } = await AUTH.client.auth.signInWithPassword({ email: login, password: pass||'' });
8436             if(error){
8437                 /* xavfsizlik: hisob bor-yo'qligini oshkor qilmaymiz */
8438                 ntf(authErrMsg(error), 'err');
8439                 try{ await AUTH.client.from('auth_events').insert({ email: login, event:'failed_login' }); }catch(_){}
8440                 return false;
8441             }
8442             AUTH.user = data.user;
8443             await authLoadProfile();
8444             if(!(await authEnforceStatus())) return false;
8445             try{ await AUTH.client.rpc('touch_last_login'); }catch(_){}
8446             await authLogEvent('login');
8447             return true;
8448         }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8449     }
8450 }
8451 const users = usersGet();
8452 const rec = users[login];
8453 if(!rec){ ntf('Email yoki parol noto\'g\'ri', 'err'); return false; }
8454 const hash = await hashPass(pass||'', rec.salt);
8455 if(hash !== rec.hash){ ntf('Email yoki parol noto\'g\'ri', 'err'); return false; }
8456 sessionSet(login);
8457 return true;
8458 }
8459 }
8460 async function logoutUser(){
8461     if(sbEnabled()){
8462         await authLogEvent('logout');
8463         try{ await AUTH.client.auth.signOut(); }catch(e){}
8464         AUTH.user = null; AUTH.profile = null;
8465     } else {

```

```

8466     sessionSet(null);
8467 }
8468 updateUserBtn();
8469 renderGallery();
8470 closeProfile();
8471 ntf('Hisobdan chiqildi', 'ok');
8472 }
8473
8474 async function changePassword(oldP, newP, newP2){
8475     if((newP||'').length < 6){ ntf('Yangi parol kamida 6 belgi', 'err'); return false; }
8476     if(newP !== newP2){ ntf('Yangi parollar mos emas', 'err'); return false; }
8477
8478     if(sbEnabled()){
8479         try{
8480             /* joriy parolni tekshirish uchun qayta kirishga urinamiz */
8481             const email = AUTH.user && AUTH.user.email;
8482             const { error: e1 } = await AUTH.client.auth.signInWithPassword({ email, password: oldP||'' });
8483             if(e1){ ntf('Joriy parol noto\'g\'ri', 'err'); return false; }
8484             const { error: e2 } = await AUTH.client.auth.updateUser({ password: newP });
8485             if(e2){ ntf(authErrMsg(e2), 'err'); return false; }
8486             await authLogEvent('password_changed');
8487             ntf('Parol muvaffaqiyatli o\'zgartirildi', 'ok');
8488             return true;
8489         }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8490     }
8491
8492     const u = currentUser();
8493     if(!u){ ntf('Avval hisobga kiring', 'err'); return false; }
8494     const users = usersGet();
8495     const rec = users[u.login];
8496     const oldHash = await hashPass(oldP||'', rec.salt);
8497     if(oldHash !== rec.hash){ ntf('Joriy parol noto\'g\'ri', 'err'); return false; }
8498     rec.salt = randSalt();
8499     rec.hash = await hashPass(newP, rec.salt);
8500     usersSet(users);
8501     ntf('Parol muvaffaqiyatli o\'zgartirildi', 'ok');
8502     return true;
8503 }
8504
8505 /* ----- Parolni tiklash ----- */
8506 async function requestPasswordReset(email){
8507     email = (email||'').trim().toLowerCase();
8508     if(!/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/.test(email)){ ntf('Email formati noto\'g\'ri', 'err'); return false; }
8509     if(!sbEnabled()){
8510         ntf('Parolni tiklash uchun server (Supabase) sozlanishi kerak', 'err');
8511         return false;
8512     }
8513     try{
8514         const { error } = await AUTH.client.auth.resetPasswordForEmail(email, {
8515             redirectTo: location.href.split('#')[0]
8516         });
8517         if(error){ ntf(authErrMsg(error), 'err'); return false; }
8518         /* xavfsizlik: email ro'yxatda bor-yo'qligini oshkor qilmaymiz */
8519         ntf('Agar bunday hisob mavjud bo\'lsa, tiklash havolasi yuborildi', 'ok');
8520         return true;
8521     }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8522 }
8523
8524 async function applyNewPassword(newP, newP2){
8525     if((newP||'').length < 6){ ntf('Parol kamida 6 belgi', 'err'); return false; }
8526     if(newP !== newP2){ ntf('Parollar mos emas', 'err'); return false; }
8527     if(!sbEnabled()){ ntf('Server sozlanmagan', 'err'); return false; }
8528     try{
8529         const { error } = await AUTH.client.auth.updateUser({ password: newP });
8530         if(error){ ntf(authErrMsg(error), 'err'); return false; }
8531         await authLogEvent('password_reset');
8532         ntf('Parol yangilandi — endi kirishingiz mumkin', 'ok');
8533         return true;
8534     }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8535 }
8536
8537 /* ----- Google orqali kirish (Supabase OAuth) ----- */
8538 async function googleSignIn(){
8539     if(!sbEnabled()){
8540         ntf('Google orqali kirish uchun avval o Server sozlash (Supabase) ni bajaring', 'err');
8541         const box = $('gcid-box'); if(box) box.hidden = false;
8542         return;
8543     }
8544     try{
8545         const { error } = await AUTH.client.auth.signInWithOAuth({
8546             provider: 'google',
8547             options:{ redirectTo: location.href.split('#')[0] }
8548         });
8549         if(error) ntf(authErrMsg(error), 'err');
8550     }catch(e){ ntf(authErrMsg(e), 'err'); }
8551 }
8552
8553 /* =====
8554 ADMIN PANEL — foydalanuvchilarni boshqarish
8555 Diqqat: bu yerdagi tekshiruvlar faqat KO'RINISH uchun. Haqiqiy
8556 ruxsat serverda — Supabase RLS siyosatlari va protect_privileged_columns
8557 triggeri bilan ta'minlanadi (supabase/schema.sql). Oddiy foydalanuvchi
8558 so'rov yuborsa ham server rad etadi.

```

```

8557 ===== */
8558 const ADM = { rows: [], q:'', status:'', role:'' };
8559
8560 function openAdmin(){
8561   if(!isEnabled()){ ntf('Admin panel uchun server (Supabase) sozlanishi kerak', 'err'); return; }
8562   if(!isAdmin()){ ntf('Ruxsat yo\'q', 'err'); return; }
8563   const m=$( 'modal-admin' ); if(m) m.classList.add('open');
8564   admLoad();
8565 }
8566 function closeAdmin(){ const m=$( 'modal-admin' ); if(m) m.classList.remove('open'); }
8567
8568 async function admLoad(){
8569   if(!isEnabled() || !isAdmin()) return;
8570   try{
8571     const [{ data: rows, error: e1 }, { data: stats }] = await Promise.all([
8572       AUTH.client.from('profiles').select('*').order('created_at', { ascending:false }).limit(500),
8573       AUTH.client.rpc('admin_stats')
8574     ]);
8575     if(e1) throw e1;
8576     ADM.rows = rows || [];
8577     admRenderStats(stats);
8578     admRender();
8579   }catch(e){ ntf(authErrMsg(e), 'err'); }
8580 }
8581
8582 function admRenderStats(st){
8583   const box = $( 'adm-stats' ); if(!box) return;
8584   if(!st){ box.innerHTML = ''; return; }
8585   const cell = (v,l) => '<div class="adm-stat"><b>'+(v==null?'-':v)+'</b><span>'+escHTML(l)+'</span></div>';
8586   box.innerHTML =
8587     cell(st.total, 'Jami foydalanuvchi') + cell(st.active, 'Faol') +
8588     cell(st.today, 'Bugun qo\'shilgan') + cell(st.blocked, 'Bloklangan') +
8589     cell(st.via_google, 'Google orqali') + cell(st.via_email, 'Email orqali');
8590 }
8591
8592 function admRender(){
8593   const t = $( 'adm-table' ); if(!t) return;
8594   const q = ADM.q.trim().toLowerCase();
8595   const list = ADM.rows.filter(r=>{
8596     if(ADM.status && r.status !== ADM.status) return false;
8597     if(ADM.role && r.role !== ADM.role) return false;
8598     if(q){
8599       const hay = ((r.email||'')+ '+(r.first_name||'')+ '+(r.last_name||')).toLowerCase();
8600       if(hay.indexOf(q) < 0) return false;
8601     }
8602     return true;
8603   });
8604   const me = currentUser();
8605   const superA = me && me.role === 'super_admin';
8606   const ROLES = ['user', 'researcher', 'teacher', 'admin', 'super_admin'];
8607   const fmt = d => d ? new Date(d).toLocaleString('uz-UZ', {year:'numeric', month:'2-digit', day:'2-digit', hour:'2-digit', minute:'2-digit'}) : '-';
8608
8609   let html = '<thead><tr><th>Email</th><th>Ism</th><th>Rol</th><th>Status</th><th>Provider</th>'
8610     + ' <th>Ro\'yxatdan o\'tgan</th><th>Oxirgi kirish</th><th>Amal</th></tr></thead><tbody>';
8611   if(!list.length){
8612     html += '<tr><td colspan="8" style="text-align:center;padding:18px;color:var(--text-soft)">Hech narsa topilmad</td></tr>';
8613   }
8614   list.forEach(r=>{
8615     const nm = [r.first_name, r.last_name].filter(Boolean).join(' ') || '-';
8616     const canEdit = superA || r.role !== 'super_admin';
8617     const roleSel = '<select data-act="role" data-id="'+r.id+'"+(canEdit?':':disabled)+'>' +
8618       ROLES.filter(x => superA || x !== 'super_admin' || r.role === 'super_admin')
8619       .map(x=>'<option value="'+x+'"+(x===r.role?' selected':'')+>'+x+'</option>').join('') +
8620       '</select>';
8621     const blocked = r.status === 'blocked';
8622     const btn = canEdit
8623       ? '<button class="btn sm "+(blocked?':':'danger')+" data-act="'+(blocked?'unblock':'block')+" data-id="'+r.id+'">'
8624       + (blocked ? 'Blokdan chiqar' : 'Bloklash') + '</button>'
8625       : '<button class="btn sm ghost" data-act="deact" data-id="'+r.id+'"+(r.status===\'deactivated\'?' disabled
8626       :':')>Faolsizlantirish</button>'
8627     : '<span style="color:var(--text-soft);font-size:12px"></span>';
8628     html += '<tr>'
8629       + '<td>'+escHTML(r.email||'')+ (r.email_verified? '<span title="Email tasdiqlangan">'</span>':'')</td>'
8630       + '<td>'+escHTML(nm)</td>'
8631       + '<td>'+roleSel</td>'
8632       + '<td><span class="adm-badge "+escHTML(r.status)+'>'+escHTML(r.status)+'</span></td>'
8633       + '<td>'+escHTML(r.provider||'email')</td>'
8634       + '<td>'+fmt(r.created_at)</td>'
8635       + '<td>'+fmt(r.last_login)</td>'
8636       + '<td>'+btn</td>'
8637     + '</tr>';
8638   });
8639   html += '</tbody>';
8640   t.innerHTML = html;
8641 }
8642
8643 async function admSetStatus(id, status){
8644   try{

```

```

8644     const { error } = await AUTH.client.from('profiles').update({ status }).eq('id', id);
8645     if(error) throw error;
8646     ntf('Status yangilandi: ' + status, 'ok');
8647     await admLoad();
8648   }catch(e){ ntf(authErrMsg(e), 'err'); }
8649 }
8650 async function admSetRole(id, role){
8651   try{
8652     const { error } = await AUTH.client.from('profiles').update({ role }).eq('id', id);
8653     if(error) throw error;
8654     ntf('Rol yangilandi: ' + role, 'ok');
8655     await admLoad();
8656   }catch(e){ ntf(authErrMsg(e), 'err'); await admLoad(); }
8657 }
8658
8659 function wireAdmin(){
8660   const t = $('#adm-table');
8661   if(t){
8662     t.addEventListener('click', e=>{
8663       const b = e.target.closest('button[data-act]');
8664       if(!b) return;
8665       const id = b.dataset.id;
8666       if(b.dataset.act === 'block') admSetStatus(id, 'blocked');
8667       if(b.dataset.act === 'unblock') admSetStatus(id, 'active');
8668       if(b.dataset.act === 'deact') admSetStatus(id, 'deactivated');
8669     });
8670     t.addEventListener('change', e=>{
8671       const sel = e.target.closest('select[data-act="role"]');
8672       if(sel) admSetRole(sel.dataset.id, sel.value);
8673     });
8674   }
8675   const q=$('#adm-q'); if(q) q.addEventListener('input', ()=>{ ADM.q = q.value; admRender(); });
8676   const st=$('#adm-status');if(st) st.addEventListener('change', ()=>{ ADM.status = st.value; admRender(); });
8677   const rl=$('#adm-role'); if(rl) rl.addEventListener('change', ()=>{ ADM.role = rl.value; admRender(); });
8678   const rf=$('#adm-refresh');if(rf) rf.addEventListener('click', admLoad);
8679   const cl=$('#adm-close'); if(cl) cl.addEventListener('click', closeAdmin);
8680   const bk=$('#modal-admin');if(bk) bk.addEventListener('click', e=>{ if(e.target===bk) closeAdmin(); });
8681 }
8682
8683 /* ----- Parolni tiklash oynalari ----- */
8684 function openForgot(){ closeAuth(); const m=$('#modal-forgot'); if(m) m.classList.add('open'); }
8685 function closeForgot(){ const m=$('#modal-forgot'); if(m) m.classList.remove('open'); }
8686 function openNewPassword(){ const m=$('#modal-newpass'); if(m) m.classList.add('open'); }
8687 function closeNewPassword(){ const m=$('#modal-newpass'); if(m) m.classList.remove('open'); }
8688
8689 function wireResetUI(){
8690   const f=$('#btn-forgot'); if(f) f.addEventListener('click', openForgot);
8691   const fc=$('#fg-cancel'); if(fc) fc.addEventListener('click', closeForgot);
8692   const fb=$('#modal-forgot'); if(fb) fb.addEventListener('click', e=>{ if(e.target===fb) closeForgot(); });
8693   const fs=$('#fg-send');
8694   if(fs) fs.addEventListener('click', async ()=>{
8695     const el=$('#fg-email');
8696     if(await requestPasswordReset(el?el.value:')) closeForgot();
8697   });
8698   const nc=$('#np-cancel'); if(nc) nc.addEventListener('click', closeNewPassword);
8699   const ns=$('#np-save');
8700   if(ns) ns.addEventListener('click', async ()=>{
8701     const a=$('#np-1'), b=$('#np-2');
8702     if(await applyNewPassword(a?a.value:'', b?b.value:')){
8703       if(a) a.value=''; if(b) b.value='';
8704       closeNewPassword();
8705     }
8706   });
8707 }
8708
8709 /* Auth modalidagi rejim izohi – foydalanuvchi qaysi rejimda ekanini bilsin */
8710 function updateAuthNote(){
8711   const n = $('#auth-mode-note');
8712   if(!n) return;
8713   n.innerHTML = sbEnabled()
8714     ? '□ Hisoblar serverda (Supabase) saqlanadi; parol bcrypt bilan xeshlanadi va bizga ko\'rinmaydi.'
8715     : '△ Server sozlanmagan – hisob faqat SHU QURILMADA saqlanadi (demo rejimi). Haqiqiy hisoblar, Google kirish
8716 va parol tiklash uchun yuqoridagi «Server sozlash» ni bajaring.';
8717 }
8718
8719 /* ----- Auth UI ----- */
8720 /* Modal har safar KIRISH tabida ochiladi. Aks holda foydalanuvchi bir marta
8721 "Ro'yxatdan o'tish" tabiga o'tsa, tab holati saqlanib qolar va keyingi
8722 ochilishlarda kirish formasi (#auth-login-box) yashirin qolardi –
8723 tashqaridan bu "login umuman ishlaymapti" bo'lib ko'rinadi. */
8724 function authShowTab(which){
8725   const seg = $('#auth-seg');
8726   if(seg) seg.querySelectorAll('button').forEach(b=>{
8727     b.classList.toggle('sel', b.dataset.v === which);
8728   });
8729   const lb=$('#auth-login-box'), rb=$('#auth-reg-box');
8730   if(lb) lb.hidden = which !== 'login';
8731   if(rb) rb.hidden = which !== 'reg';
8732   const err=$('#gcid-box'); if(err) err.hidden = true;
8733 }
8734 function openAuth(tab){

```

```

8734     authShowTab(tab === 'reg' ? 'reg' : 'login');
8735     updateAuthNote();
8736     const m=$( 'modal-auth' ); if(m) m.classList.add('open');
8737 }
8738 function closeAuth(){ const m=$( 'modal-auth' ); if(m) m.classList.remove('open'); }
8739 function openProfile(){ renderProfile(); const m=$( 'modal-profile' ); if(m) m.classList.add('open'); }
8740 function closeProfile(){ const m=$( 'modal-profile' ); if(m) m.classList.remove('open'); }
8741 function updateUserBtn(){
8742     const b = $( 'btn-user' );
8743     if(!b) return;
8744     const u = currentUser();
8745     b.textContent = u ? ( ' ' + u.name ) : ' Kirish';
8746 }
8747 function renderProfile(){
8748     const body = $( 'profile-body' );
8749     if(!body) return;
8750     const u = currentUser();
8751     if(!u){ body.innerHTML = '<p>Hisobga kirilmagan.</p>'; return; }
8752     const pats = getSavedList();
8753     const d = new Date(u.created);
8754     const dateStr = d.getDate()+'.'+(d.getMonth()+1)+'.'+d.getFullYear();
8755     const cloud = (typeof cloudActive === 'function') && cloudActive();
8756     let html = '<div class="prof-row"><span>Ism</span><b>'+escHTML(u.name)+'</b></div>' +
8757     '<div class="prof-row"><span>Email</span><b>'+escHTML(u.login)+'</b></div>' +
8758     '<div class="prof-row"><span>Ro'yxatdan o'tish usuli</span><b>' +
8759     (u.provider === 'google' ? 'Google' : 'Email va parol') + '</b></div>' +
8760     '<div class="prof-row"><span>A'zo bo'lgan sana</span><b>'+dateStr+'</b></div>' +
8761     '<div class="prof-row"><span>Yaratilgan naqshlar</span><b>'+pats.length+' ta</b></div>' +
8762     '<div class="prof-row"><span>Naqshlar qayerda</span><b>' +
8763     (cloud ? ' Hisobingizda – boshqa qurilmada ham ochiladi'
8764     : ' Faqat shu qurilmada') + '</b></div>';
8765     html += '<div class="prof-sec"><h4>Mening naqshlarim</h4><div class="gal-grid" id="prof-gal">';
8766     html += '</div>' + (pats.length ? '' : '<div class="gal-empty">Studio da « Saqlash» bosing – naqshlar shu yerda
to'planadi.</div>') + '</div>';
8767     html += '<div class="prof-sec"><h4>Sozlamalar – parolni o'zgartirish</h4>' +
8768     '<div class="fld"><label>Joriy parol</label><input id="cp-old" type="password" autocomplete="current-password"
></div>' +
8769     '<div class="fld"><label>Yangi parol</label><input id="cp-new" type="password" autocomplete="new-password"></d
iv>' +
8770     '<div class="fld"><label>Yangi parolni tasdiqlang</label><input id="cp-new2" type="password" autocomplete="new
-password"></div>' +
8771     '<button class="btn sm" id="btn-cp"> Parolni o'zgartirish</button></div>';
8772     if(isAdmin()){
8773         html += '<div class="prof-sec"><h4>Administrator</h4>' +
8774         '<button class="btn sm gold" id="btn-admin"> Foydalanuvchilarni boshqarish</button></div>';
8775     }
8776     html += '<div class="prof-sec"><button class="btn sm danger" id="btn-logout"> Hisobdan chiqish</button></div>';
8777     body.innerHTML = html;
8778     /* naqsh thumb lari */
8779     const grid = $( 'prof-gal' );
8780     if(grid){
8781         pats.forEach(item=>{
8782             const dv = document.createElement('div');
8783             dv.className = 'gal-item';
8784             const img = document.createElement('img');
8785             img.src = item.thumb;
8786             img.alt = 'naqsh';
8787             dv.appendChild(img);
8788             dv.addEventListener('click', ()=>{
8789                 Object.assign(P, item.p);
8790                 syncSliders(); redraw();
8791                 closeProfile();
8792                 showPage('studio');
8793                 ntf('Naqsh yuklandi', 'ok');
8794             });
8795             grid.appendChild(dv);
8796         });
8797     }
8798     const cpBtn = $( 'btn-cp' );
8799     if(cpBtn) cpBtn.addEventListener('click', async ()=>{
8800         const o=$( 'cp-old' ), n=$( 'cp-new' ), n2=$( 'cp-new2' );
8801         const ok = await changePassword(o.value, n.value, n2.value);
8802         if(ok){ if(o.value) o.value=''; if(n.value) n.value=''; if(n2.value) n2.value=''; }
8803     });
8804     const lo = $( 'btn-logout' );
8805     if(lo) lo.addEventListener('click', logoutUser);
8806     const ab = $( 'btn-admin' );
8807     if(ab) ab.addEventListener('click', ()=>{ closeProfile(); openAdmin(); });
8808 }
8809 function wireAuth(){
8810     const ub = $( 'btn-user' );
8811     if(ub) ub.addEventListener('click', ()=>{ currentUser() ? openProfile() : openAuth(); });
8812     const seg = $( 'auth-seg' );
8813     if(seg) seg.addEventListener('click', e=>{
8814         const b = e.target.closest('button');
8815         if(!b) return;
8816         authShowTab(b.dataset.v);
8817     });
8818     const c1=$( 'auth-cancel' ); if(c1) c1.addEventListener('click', closeAuth);
8819     const c2=$( 'auth-cancel2' ); if(c2) c2.addEventListener('click', closeAuth);
8820     const back=$( 'modal-auth' ); if(back) back.addEventListener('click', e=>{ if(e.target===back) closeAuth(); });

```

```

8821   const pback=$( 'modal-profile' ); if(pback) pback.addEventListener('click', e=>{ if(e.target===pback) closeProfile
8822   (); });
8823   const pc=$( 'profile-close' ); if(pc) pc.addEventListener('click', closeProfile);
8824   const doL=$( 'btn-do-login' );
8825   if(doL) doL.addEventListener('click', async ()=>{
8826     const l=$( 'al-login' ), p=$( 'al-pass' );
8827     if(await loginUser(l?.value:'', p?.value:')){
8828       closeAuth(); updateUserBtn(); renderGallery();
8829       ntf('Xush kelibsiz, ' + currentUser().name + '! +', 'ok');
8830       if(p) p.value='';
8831     }
8832   });
8833   const doR=$( 'btn-do-register' );
8834   if(doR) doR.addEventListener('click', async ()=>{
8835     const n=$( 'au-name' ), ln=$( 'au-last' ), l=$( 'au-login' ), p=$( 'au-pass' ), p2=$( 'au-pass2' );
8836     if(await registerUser(n?.value:'', l?.value:'', p?.value:'', p2?.value:'', ln?.value:')){
8837       closeAuth(); updateUserBtn(); renderGallery();
8838       ntf('Hisob yaratildi — xush kelibsiz! ☑', 'ok');
8839       if(p) p.value=''; if(p2) p2.value='';
8840     }
8841   });
8842   const alp=$( 'al-pass' );
8843   if(alp) alp.addEventListener('keydown', e=>{ if(e.key==='Enter' && doL) doL.click(); });
8844   const g1=$( 'btn-google-login' ); if(g1) g1.addEventListener('click', googleSignIn);
8845   const g2=$( 'btn-google-reg' ); if(g2) g2.addEventListener('click', googleSignIn);
8846   const gb=$( 'btn-gcid' ); if(gb) gb.addEventListener('click', ()=>{
8847     const box=$( 'gcid-box' );
8848     if(box){
8849       box.hidden = !box.hidden;
8850       if(!box.hidden){
8851         const cfg = sbCfg();
8852         const u=$( 'inp-sburl' ), k=$( 'inp-sbkey' );
8853         if(u) u.value = cfg.url; if(k) k.value = cfg.key;
8854       }
8855     }
8856   });
8857   const gs=$( 'btn-gcid-save' ); if(gs) gs.addEventListener('click', ()=>{
8858     const u=$( 'inp-sburl' ), k=$( 'inp-sbkey' );
8859     const url = u ? u.value.trim() : '', key = k ? k.value.trim() : '';
8860     if(url && !/^https:\/\/[a-z0-9-]+\.\supabase\.co\/?$/i.test(url)){
8861       ntf('Project URL odatda https://xxxx.supabase.co ko\'rinishida bo\'ladi', 'err');
8862     }
8863     if(sbCfgSet(url, key)){
8864       ntf(url && key ? 'Saqlandi ✓ — server ulanmoqda...' : 'Server sozlamalari tozalandi', 'ok');
8865       const box=$( 'gcid-box' ); if(box) box.hidden = true;
8866       authBootstrap().then(()=>{
8867         updateAuthNote();
8868         ntf(sbEnabled() ? 'Serverga ulandi ✓' : 'Lokal (demo) rejimga qaytildi', sbEnabled()?'ok':'err');
8869       });
8870     } else {
8871       ntf('Saqlab bo\'lmadi (localStorage)', 'err');
8872     }
8873   });
8874   const gc=$( 'btn-gcid-close' ); if(gc) gc.addEventListener('click', ()=>{
8875     const box=$( 'gcid-box' ); if(box) box.hidden = true;
8876   });
8877 }
8878 /* — JONLI KURSOR: yumshoq ergashuvchi halqa + bosishda to'liqin — */
8879 function initCursor(){
8880   try{
8881     if(!window.matchMedia || !matchMedia('(pointer:fine)').matches) return;
8882     if(!document.body) return;
8883     const dotEl = document.createElement('div'); dotEl.id = 'cur-dot';
8884     const ringEl = document.createElement('div'); ringEl.id = 'cur-ring';
8885     document.body.appendChild(ringEl);
8886     document.body.appendChild(dotEl);
8887     let mx=-100, my=-100, rx=-100, ry=-100;
8888     document.addEventListener('pointermove', e=>{
8889       mx = e.clientX; my = e.clientY;
8890       dotEl.style.left = mx+'px';
8891       dotEl.style.top = my+'px';
8892       const hot = e.target && e.target.closest ? e.target.closest('button,select,input,a,.pal-chip,.gal-item,canva
8893       s') : null;
8894       if(ringEl.classList) ringEl.classList.toggle('hot', !!hot);
8895     });
8896     const loop = ()=>{
8897       rx += (mx-rx)*0.16; ry += (my-ry)*0.16;
8898       ringEl.style.left = rx+'px';
8899       ringEl.style.top = ry+'px';
8900       requestAnimationFrame(loop);
8901     };
8902     loop();
8903     document.addEventListener('pointerdown', e=>{
8904       const rp = document.createElement('div');
8905       rp.className = 'cur-ripple';
8906       rp.style.left = e.clientX+'px';
8907       rp.style.top = e.clientY+'px';
8908       document.body.appendChild(rp);
8909       setTimeout(()=>{ if(rp.parentNode) rp.parentNode.removeChild(rp); }, 560);
8910     });
8911   }

```

```

8910     }catch(e){}
8911   }
8912   /* == INTRO VIDEO ==
8913   Videoni HTML fayl YONIGA "intro.mp4" nomi bilan qo'ying – avtomatik ijro etiladi.
8914   Boshqa nom/URL uchun quyidagi konstantani o'zgartiring. Bo'sh '' = intro o'chiq. */
8915   const INTRO_VIDEO_SRC = 'intro.mp4';
8916   function initIntro(after){
8917     const ov = $('intro-ov'), v = $('intro-video');
8918     let seen = false;
8919     try{ seen = sessionStorage.getItem('naqsh_intro') === '1'; }catch(e){}
8920     if(!ov || !v || !INTRO_VIDEO_SRC || seen){ after(); return; }
8921     let done = false;
8922     const finish = ()=>{
8923       if(done) return;
8924       done = true;
8925       try{ sessionStorage.setItem('naqsh_intro','1'); }catch(e){}
8926       try{ if(v.pause) v.pause(); }catch(e){}
8927       if(ov.classList) ov.classList.add('done');
8928       setTimeout(()=>{ if(ov.parentNode) ov.parentNode.removeChild(ov); }, 650);
8929       after();
8930     };
8931     v.addEventListener('ended', finish);
8932     v.addEventListener('error', finish); /* video topilmasa – darhol o'tib ketadi */
8933     const skip = $('intro-skip');
8934     if(skip) skip.addEventListener('click', finish);
8935     const mute = $('intro-mute');
8936     if(mute) mute.addEventListener('click', ()=>{
8937       v.muted = !v.muted;
8938       mute.textContent = v.muted ? '□ Ovozni yoqish' : '□ Ovozni o\'chirish';
8939     });
8940     ov.hidden = false;
8941     v.src = INTRO_VIDEO_SRC;
8942     try{
8943       const pr = v.play ? v.play() : null;
8944       if(pr && pr.catch) pr.catch(finish);
8945     }catch(e){ finish(); }
8946     /* fayl yo'q va error kelmasa ham 2s da davom etamiz */
8947     setTimeout(()=>{ if(v.readyState === 0 || v.readyState === undefined) finish(); }, 2000);
8948   }
8949   function initSplash(){
8950     try{
8951       const sp = $('splash');
8952       if(!sp) return;
8953       /* yulduz chizilish animatsiyasini qayta boshlash */
8954       try{
8955         const paths = sp.querySelectorAll ? sp.querySelectorAll('path') : [];
8956         Array.prototype.forEach.call(paths, p=>{
8957           p.style.animation = 'none';
8958           void p.getBoundingClientRect();
8959           p.style.animation = '';
8960         });
8961       }catch(e){}
8962       let seen = false;
8963       try{ seen = sessionStorage.getItem('naqsh_splash') === '1'; }catch(e){}
8964       const delay = seen ? 350 : 1900; /* birinchi tashrif – to'liq sahna, keyin qisqa */
8965       setTimeout(()=>{
8966         if(sp.classList) sp.classList.add('done');
8967         setTimeout(()=>{ if(sp.parentNode) sp.parentNode.removeChild(sp); }, 700);
8968       }, delay);
8969       sp.addEventListener('click', ()=>{
8970         if(sp.classList) sp.classList.add('done');
8971         setTimeout(()=>{ if(sp.parentNode) sp.parentNode.removeChild(sp); }, 700);
8972       });
8973       try{ sessionStorage.setItem('naqsh_splash','1'); }catch(e){}
8974     }catch(e){}
8975   }
8976   function init(){
8977     initIntro(initSplash);
8978     cvEl = $('cv');
8979     buildPatternList();
8980     buildSliders();
8981     buildPalettes();
8982     buildGirihUI();
8983     wireGirihUI();
8984     buildTheory();
8985     buildChips();
8986     renderGallery();
8987     restoreChatHistory();
8988     wireUI();
8989     wireAuth();
8990     wireAdmin();
8991     wireResetUI();
8992     wireAiSetup();
8993     updateUserBtn();
8994     authBootstrap(); /* Supabase sessiyasini tiklash (sozlangan bo'lsa) */
8995     initCursor();
8996     resizeCanvas();
8997     ntf('NaqshAI tayyor – ijodga marhamat! +', 'ok');
8998   }
8999   /* Initialization – try-catch bilan himoyalangan */
9000   document.addEventListener('DOMContentLoaded', function(){

```

```
9001     try{
9002         init();
9003     }catch(e){
9004         console.error('NaqshAI init xatosi:', e);
9005         try{ ntf('Ishga tushirishda xato: ' + e.message, 'err'); }catch(_e){}
9006     }
9007 };
9008
```